

Crossing with Confidence: Formal Analysis and Model Checking of Blockchain Bridges

Pyrros Chaidos*, Pooya Farshim[†], Denis Firsov[‡], Dimitar Jetchev^{†**}
Aggelos Kiayias^{||¶}, Markulf Kohlweiss^{||¶}, Anca Nitulescu[§]

Input Output, *Greece, [†]Switzerland, [‡]Estonia, [¶]UK, [§]France

^{||} University of Edinburgh, UK

^{**} Microsoft Azure AI Platform, US

Abstract—We develop formal code-based security definitions for blockchain bridges and apply them to several bridge architectures deployed in practice. We derive both traditional pen-and-paper proofs and on the other, automated guarantees against bounded counterexamples. The latter is achieved via bounded model checking of our formally specified properties, implemented in Quint, a specification language and model checker closely related to TLA⁺.

Our definitions are expressed in a precise, code-based variant of the Universal Composition (UC) framework. This enables the modular use of hybrid functionalities—even for property-based definitions—and is essential for bounded model checking, where underlying primitives must be idealized.

Accordingly, we idealize and model-check all building blocks used in our protocols. Notably, we formulate a novel UC ideal functionality for Advanced Threshold Signatures (ATS) and model-check it for attacks to ensure its robustness.

Index Terms—Formal Security, Universal Composition, Model Checking, Blockchain Bridges, Advanced Threshold Signatures.

I. INTRODUCTION

Major ledger networks, despite their combined financial worth of \$3T,¹ currently lack native interoperability, relying instead on a variety of blockchain bridges. A blockchain bridge, is a protocol that allows the transfer of information, such as ownership of digital assets or smart contract state, between two distributed ledgers.

Several high-profile blockchain bridges rely on arrangements where a small fixed committee uses a threshold signature scheme to attest to the state of the sending chain. This strongly ties the security of these bridges to trusting the committees and their key management practices, and consequently has led to numerous serious security breaches including the Ronin attack (\$625M stolen) [43]. Moreover, subtle and technically sophisticated attacks on fixed committee bridges, such as the Poly Network attack (\$611M stolen) and Solana’s Wormhole bridge attack (compromised assets of total value \$325M), highlight the need for security to carry through from design to implementation even when trust assumptions hold.

Motivated by the financial value locked in blockchain bridges and the resulting societal risks, we initiate a formal treatment of security for blockchain bridges. Our goals are twofold:

- Existing works on blockchain bridges typically focus on protocol constructions, and often treat security and protocol analysis informally. Our first goal is to complement these lines of work by defining explicit syntax and security models within the UC framework, a sufficiently general model of computation. This brings about a clear and shared understanding of security requirements needed to make the traditional pen-and-paper provable security approach meaningful. Indeed, we carry out such proofs for a number of bridge architectures deployed in practice: direct observation, fixed committee, native consensus certificate, ZK, and advanced threshold signature (ATS) bridges.
- We specify bridge protocols and security properties in a code-based language that is sufficiently precise to render protocols amenable to both easier implementation and analysis. To evidence this, we use Quint [44], a user-friendly language for specifying and model-checking distributed protocols, to automatically analyze protocols and their idealized building blocks (referred to as hybrid functionalities in UC) for logical flaws. This significantly strengthens our trust in the robustness of our definitions, protocol specifications, and proofs. A conceptual insight that emerges is the need for perfectly secure building blocks, as model checkers typically cannot (easily) handle probabilistic or reductionist arguments.

We stress that our pen-and-paper proofs play a central role in validating the correctness and security of the core protocol design, providing conceptual clarity and theoretical assurance. Complementing this, the UC model checking framework we develop in Quint validates these theoretical efforts by providing automated, systematic evidence regarding the sanity and consistency of the formal definitions, protocols, and building blocks. Furthermore, this framework allows for the efficient exploration of protocols and is particularly effective at identifying logical flaws early in the design phase. Together, these two approaches reinforce one another: hand-written proofs guide sound protocol design, while model checking provides scalable, automated validation of the specification.

We next give an overview of our work, starting with the underlying framework, the security definitions formulated

¹As of 26 Jan 2026; see <https://coinmarketcap.com/charts/>.

therein, a provable security treatment of a number of bridge architectures, and concluding with a brief report of our experimental results in Quint.

A. Framework and security definitions

To address the gap in formal definitions of security for bridges, we adopt a code-based language akin to that used in stand-alone (aka. game-based) security models [6]. However, in the setting of distributed protocols, setup assumptions, such as the type of PKI or the underlying network model, are of greater importance. This motivated us to employ the Universal Composability (UC) framework and extend it with a similar code-based approach that is, however, better suited for the distributed setting. (We give an overview in Section II-B.)

Although we operate within the UC framework, we nevertheless formulate our security definitions as trace *properties*. In other words, in our setting, an environment (which is an interactive distinguisher) tries to elicit an execution trace that violates a specific property. There are two main benefits to this approach: first, security properties are easier (often substantially so) to formulate. Second, the specified properties are compatible with model checking: a model checker searches for counterexamples for specific properties. We also retain the composition guarantees of UC when it comes to instantiating idealized building blocks.

We formulate bridge protocols as a collection of interfaces for generating bridge-maintenance keys (e.g., signature keys) and a root-of-trust data (e.g., the genesis blocks of respective chains, and initial public keys), as well as the core bridge maintenance protocol which submits transactions to the ledgers.

We introduce two new interfaces, Certify and the matching Verify. This abstraction allows us to shift our focus from the transfer of tokens or messages (which is commonly viewed as “the bridge”) and instead study general types of statements that can be verified by smart contracts on a receiving ledger about the state of a sending chain. (Statements being about chains facilitates chain introspection.) More concretely, we split the design of bridges into two parts: a protocol for transferring (compressed) state—aka. finalized block headers—to a receiving ledger, and a protocol for certifying and verifying properties of the sending chain using headers.

Relative to this syntax, we identify two core security properties for bridges: *bridge soundness* and *bridge liveness*. Intuitively, under certain trust assumptions, bridge soundness guarantees that if a “property certificate” for a statement about the sending ledger verifies on the receiving ledger, then the property indeed holds on the sending ledger with respect to all honest parties’ ledgers. Bridge liveness, on the other hand, means that a true property of a sending ledger can eventually be proven on the receiving ledger. These properties roughly correspond to the functional and security goals informally stated for the bridge construction in [51]. Our effort consists of stating these properties in a more rigorous language suitable for proofs and model checking and generalizing them to cover a variety of bridge architectures. Although we focus on liveness and soundness, other notions of security can be formulated

using the approach we put forth here. In particular, applications built on top of bridges can be analyzed and model checked, as we do here. To support this modularity, we make the Verify interface callable by the environment.

B. Building bridges, an overview

In this work, we provide a comprehensive treatment of security for a number of bridge architectures. We begin by describing two simple approaches for building bridges—direct observation and fixed committees—before discussing rotating-committee and ZK bridges, as well as bridges based on advanced threshold signatures (ATS), which is our main focus.

Two simple bridge architectures. In the *direct observation* (DO) approach, to prove that some statement holds for the sending ledger, the nodes of the receiving ledger directly observe the sending ledger and verify the statement. A slight refinement of this idea (which is used in other bridges below) requires the receiving ledger parties to only store headers that serve as (correctly observed) fingerprints of the sending ledger. Later, Certify (run by the nodes of the sending ledger) generates a proof of the statement with respect to a header value, which can be transferred off-chain to the nodes of the receiving chain, and verified against the recorded header. The main drawback of DO bridges is their high verification and storage overhead: receiving-ledger nodes must continuously track the sending ledger and validate its consensus rules. This significantly increases complexity and limits its scalability.

An alternative design for bridges relies on *fixed committees*, where a predefined set of parties attests to the validity of statements about the sending chain to a receiving ledger. While fixed-committee (FC) bridges are more efficient—yielding, for example, compact consensus certificates and minimal on-chain verification costs—they introduce a strong trust assumption: security now depends on the honesty and availability of a fixed subset of participants, making such bridges fragile and vulnerable to corruption, cartels, or long-term key compromise. We explore the security of the DO and FC bridges in Appendices H-A and H-C, respectively.

The two simple bridge architectures have complementary strengths. The Direct-Observation (DO) bridge is *trustless*, while the Fixed-Committee (FC) bridge has *compact consensus certificates* that are validated by the receiving ledger. We stress that consensus certificates allow validating the headers of sending chains on the receiving ledger, while property certificates validate *statements* against already validated headers.

Architectures using rotating committees. To overcome the limitations of these approaches, we consider an architecture that enables trade-offs between the efficiency of committee-based designs and the stronger security guarantees enjoyed by the direct-observation approach.

The core insight to avoid the downsides of fixed committees is to employ a *stake-based committee-selection* mechanism, which is often implicit and readily available in proof-of-stake consensus protocols. One then tasks (a threshold number of) the members of the committee to sign sufficient information to

establish the bridge. In particular, one needs to sign the header of the sending chain and enough information to determine the next committee to be used. Due to the ad-hoc nature of the committees and the desire to keep certificates small, Gaži, Kiayias and Zindros (GKZ) [27] designed a dedicated signature scheme for building bridges, namely a threshold signature with signers picked from an ad-hoc committee. They refer to such a scheme as an *Ad-hoc Threshold Multi-Signature*. Due to their utility for building blockchain bridges and oracles, such signatures have recently received increased research attention under the guise of *Multiverse Threshold Signatures* (MTS) [4] and *Silent-Setup Threshold Signatures* (STS) [25]. We prefer the more general term *Advanced Threshold Signature* (ATS) to emphasize that our results are also applicable to these alternative formulations. In this work, we will formally specify, model-check, and prove a trustless bridge architecture based on ATS. We provide more details and an overview of our technical contribution in Section I-C below.

Architectures using traditional signatures and ZK. In blockchains where the consensus mechanism relies on iterated Byzantine Fault-Tolerant (BFT) protocols [18], such as Tendermint [46] or HotStuff [52], quorums of validators create native *consensus certificates* that can be directly verified. For example, Cosmos uses Tendermint, and Ethereum supports both heavy-weight consensus certificates through its beacon chain and a more light-weight variant through its light-client architecture. Such chains do not require a full external bridge protocol. We briefly explain what bridges following such a design look like, and how they can be captured in our definitional framework.

Consensus-certificate bridge. These bridges essentially transfer *native* consensus certificates on the state of the sending chain for verification on a receiving ledger. Thus, by relying on this built-in consensus mechanism, such bridges minimize trust assumptions and reduce attack surface. A number of deployed bridges (e.g., Rainbow [39] and Polygon PoS [41]) follow this blueprint. To give a complete landscape for bridges, we cover the formalization of this bridge in Appendix H-B. We formulate a novel notion of secure and live consensus certificate as a (chain of) artifacts that can be extracted from a given chain. We then use this definition to define a simple native consensus-certificate bridge as outlined above.

Compressing consensus certificates using SNARKs. Native consensus certificates can be large. Rather than re-implementing the consensus logic of the sending ledger in the receiving ledger (which would be complex and expensive), the zkBridge construction of Xie et al. [51] generates a succinct proof of *light-client execution*: the receiving ledger maintains a compact representation of the sending chain’s state, typically a finalized block header. To advance this state, the bridge must verify on the receiving side that the sending chain has correctly progressed according to the light-client protocol—this corresponds to succinctly proving and verifying (and thus compressing) a native consensus certificate.

We formalize Xie et al.’s work in our framework. Towards this, we formalize light-client protocols by restricting liveness and soundness of bridge definitions to an isolated “client-local” receiving ledger. One then establishes a bridge by running the client-local ledger in a smart contract on the receiving ledger. We develop these ideas in full detail in Appendix I-D.

A key limitation of using consensus certificates is that many blockchains do not natively support them. Furthermore, interoperability poses other challenges: the receiving ledger must be capable of interpreting and verifying the certificate format and validation logic of the sending ledger, which often requires custom integration. This is an issue when bridging chains with different cryptographic primitives in their design, e.g., chains supporting different elliptic curve arithmetic or different signature schemes. Compression via SNARKs can help with interoperability but requires an expensive prover on the sending side. For example, Xie et al. use a two-layered deVirgo+Groth16 prover for balancing proof generation and verification efficiency.

C. An architecture based on Advanced Threshold Signatures

Our ATS bridge considers bridge committees that are regularly rotated, analogous to block-production committees in ledger constructions (e.g., beacon committees in Ethereum, or slot leaders for Cardano). We follow GKZ [27] and consider bridges where a secure committee is selected from the sending ledger, and then required to participate in the signing of header values via an Advanced Threshold Signature (ATS) scheme. An ATS is similar to threshold signature schemes in that only a threshold number of valid signatures need to be present, thus avoiding the need for all signers to be online. It differs from them in that it does not require a key-generation protocol for distributing key shares; instead each potential signer generates their own signing key. (Recently this has also been referred to as a Silent-Setup [25].)

Our ATS bridge follows GKZ, but is streamlined in several aspects, and presented within the UC execution framework. Specifically, we assume that the ledger offers (or can be extended with) a committee-selection procedure that at the beginning of each epoch returns a sufficiently large committee satisfying a threshold honesty property. Each committee member signs the block header of an agreed prefix of the sending chain. An aggregator collects and aggregates the signatures into an aggregate threshold signature that it submits to a contract on the receiving ledger for verification.

Our protocol, being expressed in the UC framework relative to idealized building blocks, accurately accounts for network delays (more precisely we operate in the Δ -delayed synchronous setting), which influence the propagation of protocol messages, including updated aggregate keys after committee rotation. Bridge soundness and liveness are summarized informally in Theorem 1, and formally established in Theorems 10 and 11, Appendix I-C.

The ATS bridge abstracts features of rotating committee constructions such as Chainflip [19] and Thorchain [48], where the committee signs headers using a threshold signature scheme.

D. Security analyses

Proofs. In the appendices we carry out extensive pen-and-paper proofs of security for the five bridges above (advanced threshold signature (ATS), direct observation, fixed committee, consensus certificate, and ZK bridge). For each bridge, our theorems explicitly state the trust assumptions that are needed for liveness and soundness. We formulate our theorems with respect to ledger properties (as opposed to an ideal ledger functionality), as this results in *stronger* overall statements. Indeed, ideal ledgers can be shown—and have been model checked—to satisfy the ledger properties that we rely on. Our assumptions, by virtue of being weaker, allow our theorems to be applied to a potentially larger class of blockchains.

The proofs of our theorems are structured as sequences of games, similarly to game-based security analyses. This clearly isolates the assumptions (cryptographic, ledger, and trust) that the proofs rely on, and results in proofs that are easier to verify. It also allows us to derive concrete overall advantage bounds.

Model checking. We complement our theoretical analysis by presenting experimental evidence towards the security and correctness of our protocols and definitions. To this end, we carry out a systematic application of (bounded) model checking, implemented in the Quint specification language [44], to the ATS bridge and all its underlying building blocks. These include, among others, checking network diffusion, zero-knowledge, ledgers, and the ATS primitive.

This effort is motivated in part by UC’s subtle operational semantics, which often leads to complex ideal functionalities and protocols. Specifically, in UC, when simulators are called they do not need to respond immediately and can yield control to the environment. The latter can, in turn, choose to re-enter a functionality at a different point. Moreover, simulators can resume execution at *any* point where they were called. (This is how UC models concurrency in a single-threaded manner.) This nonlinear flow often hides concurrency bugs that are easily missed by manual inspection. A major effort in Quint consists of creating scaffolding to support UC’s control flow.

Indeed, we found bugs/flaws in almost all functionalities that we model checked, even seemingly simple ones. For example, we identify a “flip-flop” bug in a seemingly innocuous definition of signature ideal functionality (see $\mathcal{F}\text{-Sig}$ in Appendix C and discussions in Section V), where a specific sequence of yields to and resumes by the simulator result in the system returning inconsistent verification results. As a second example, we designed our ATS ideal functionality *in tandem* with model checking. In this process, multiple correctness and security bugs were discovered (sometimes even when we were convinced that the functionality was “obviously correct”). The result is functionality $\mathcal{F}\text{-ATS}$ (Appendix I-A), which despite its complexity has been thoroughly checked for a variety of trace and internal state properties. We note that precise code-based definitions were critical in this process.

Model checking relies on building blocks that are *ideally* secure, and thus matches UC’s approach to security well. Indeed, model checking is typically non-probabilistic, meaning

that Quint tries to analyze all possible inputs and responses from pre-defined bounded spaces; even a single set of inputs and responses that violate a given property is not tolerated regardless of probabilities with which this case would arise during the execution.

The full Quint code, which comprises approximately 10K loc and is attached as auxiliary material, and in Section V we provide an overview. Our work demonstrates that model checking can be a viable method to complement pen-and-paper proofs, even for fairly complex protocols in UC.

II. PRELIMINARIES AND DEFINITIONAL FRAMEWORK

A. Basic notation

Given a tuple $t = (a, b, \dots)$, we use $t.a$ to denote the a -coordinate of t . For a list L we denote its i -th entry by $L[i]$. We assume throughout that $\perp$ is the “none” option type and that $\square$ denotes a value that is not set.

We let $\text{prefix}(L, \ell)$ denote the function that truncates a list L to length ℓ . We define $\text{isPrefix}(L_1, L_2) := (L_1 = \text{prefix}(L_2, |L_1|))$. Since isPrefix forms a partial order (it is reflexive, antisymmetric, and transitive), we denote it by $\preceq$ for succinctness.

We use the following font faces throughout: inputs and **vectors** for data, *Algorithms* for cryptographic schemes, *methods* of contracts, *Interfaces* of protocols and functionalities, *Oracles* of security games, and *FINALIZATION* procedures of games and property-based notions. All algorithms are randomized unless stated otherwise. We denote the (internal) state of an algorithm by st , written as $\text{Alg}(\text{input}; \text{st})$. We denote the empty string by ε . Unless stated otherwise, lists and sets are initialized to empty, integers to 0, and boolean variables to false.

For stateful algorithms, we need to store various data and compute on them as an execution unfolds. We adopt the following notation: if the state of an algorithm is st and we need to store some value under field x in st , we write $\text{st}.x \leftarrow x$. We also apply this convention to smart contracts and variables in their memories. When an input or an output of an algorithm is not used we replace it with “ $_$ ”.

B. The UC framework

We rely on the Universal Composability (UC) framework [13], [14] as an execution model for our protocols. In this framework, distributed cryptographic protocols are modeled as sets of interactive Turing machines. Instead of an absolute statement of security (e.g., that an adversary cannot guess a bit), security is formulated in *relative* terms: for any adversary against a real protocol π there exists an ideal adversary (aka. a simulator) against an ideal counterpart ϕ so that no interactive distinguisher (aka. the environment) can distinguish the two. If this is the case, we say π UC-realizes ϕ .

This approach to security enables modular design: a (higher-level) protocol ρ can be designed with respect to a simpler ideal process ϕ , and then ϕ is replaced by any real process π that UC-realizes ϕ . This is due to the fact that the effects of the higher-level protocol ρ can be absorbed into the environment. (This

is essentially the UC composition theorem.) This replacement process, in turn, can be extended to multiple ideal building blocks, and enables a modular treatment of security for complex protocols. More details about the UC framework can be found in Appendix B.

We specify protocols and ideal functionalities in a precise code-based language. This results in specifications that are ready to be model-checked in Quint. In more detail, we write protocols as collections of interfaces $P.\text{Interface}(x)$ for each party P in some set $\mathbf{P}$, which is a parameter of the protocol. In our specifications we use a simple programming language with basic data structures and operations (integers, lists, sets, etc.). We also allow *passing the execution token* to adversaries/simulators; this allows for (single-threaded) concurrency. For compactness, we adapt the convention that upon the first call to a functionality, a special $\text{Init}()$ procedure is executed once.

Adversaries can *corrupt* a machine when instructed to do so by the environment. As a result, the adversary fully takes over the machine: (i) all inputs and outputs to the machine are passed to the adversary, and (ii) the adversary can replace the outputs before they are returned. With *static* corruptions, the adversary corrupts machines at the onset before any other interaction. As usual, we assume the real-world adversary is dummy (i.e., absorbed into the environment).

Our protocols operate in a Δ -delayed synchronous model, following standard approaches [2], [5], [32]. Specifically, we use a shared *clock functionality* $\mathcal{G}\text{-Clock}$, which initializes a global counter, shared across all parties, to zero. $\mathcal{G}\text{-Clock}$ increments the counter as soon as all machines registered with the $\mathcal{G}\text{-Clock}$ have expressed their willingness to proceed to the next round via $P.\text{OK}()$. A $P.\text{Round}()$ interface provides awareness of the current round r . The pseudo-code and further discussions can be found in Appendix C.

We let $\text{view}[r]$ denote the set of inputs and outputs in round r . In more detail, when interface $P.\text{Interface}(x)$ is invoked in round r , an entry $(\text{handle}, P.\text{Interface}(x))$ is added to $\text{view}[r]$ where handle is a unique identifier for the call. (This is formally needed as there could be multiple calls to the same interface with the same input(s).) When this interface returns an output y in round r' entry (handle, y) is added to $\text{view}[r']$. The set of corrupted parties in round r is denoted by $\mathbf{C}[r]$.

C. Property-based UC

We formulate security of ledgers and bridges by requiring that certain *properties* of the execution hold with overwhelming probability. Properties are predicates over the full execution trace view and party corruption status $\mathbf{C}$ (as recorded for all rounds). We define a security property as a finalization procedure SECURITYPROPERTY that outputs a bit indicating an adversarial winning condition is met. It is called exactly once by the environment at the end of the execution. We write winning conditions by negating the property that is expected to hold for a secure construction. For example, collision resistance $\text{CR}(x, x')$ is written as **check** $x \neq x'$; **return** $\neg(\text{Hash}(x) \neq \text{Hash}(x'))$.

Compared to standard game-based formulations, there are three noteworthy benefits of this approach: *First*, ideal building blocks that underlie a security game can be replaced by protocols that UC-realize them (as per UC composition theorem). *Second*, property-based UC allows one to make explicit various assumptions that are tacitly in place in traditional game-based modeling. These include underlying network assumptions, setup assumptions (such as a CRS or a PKI), and access to a shared clock. In game-based definitions these are typically handled by the challenger, often in a less principled way. *Third*, idealizing all cryptographic building blocks of a protocol makes it amenable to model checking.

D. Idealized building blocks

Besides $\mathcal{G}\text{-Clock}$, we also use a network *diffusion functionality* $\mathcal{F}\text{-Diffuse}$ that allows parties to diffuse a message among all parties connected to it with some delay [3], [22]. When a party fetches messages from the diffusion network, all messages that were sent at least Δ rounds in the past (together with some possibly more recent messages) are delivered to the party. During this process the adversary can *inject* messages of its choice. $\mathcal{F}\text{-Diffuse}$ formalizes this resource and its code is listed under Functionality 6 in Appendix C.

We also use a standard $\mathcal{F}\text{-CRS}$ functionality (see Functionality 7 in Appendix C) to allow access to a common reference string generated via a string generation algorithm Setup , as well as $\mathcal{F}\text{-Beacon}$ (Functionality 8 in Appendix C) that allows access to a common fresh randomness in each round. ($\mathcal{F}\text{-Beacon}$ is used to generate some initial random seed in a genesis block.) Finally, functionality $\mathcal{F}\text{-PKI}$ idealizes a PKI and allows parties to register their public keys, so that later any party can authentically retrieve them; see Functionality 9 in Appendix C.

We rely on (succinct) non-interactive proofs of arguments $\text{SNARK} := (\text{Setup}, \text{Prove}, \text{Verify})$ and collision resistant hash functions Hash . The syntax and security of these primitives are recalled in Appendix D.

III. BRIDGE SECURITY

In this section, we formalize the notion of a bridge protocol and establish two fundamental security properties for them: *bridge soundness* and *bridge liveness*. We subsequently rely on these properties to formally analyze the security of a number of existing bridges, via manual pen-and-paper proofs and automatically using a model checker.

A. Ledgers

Bridges operate alongside (two or more) ledgers. We thus begin our treatment with the security of ledgers.

Rather than modeling a ledger as a general protocol in the UC framework (for example, one consisting of arbitrary building blocks), we consider ledger protocols that follow a specific template: they are built out of a given number of idealized building blocks (diffusion network, CRS, PKI, and beacon) as well as a collection of *algorithms* for ledger maintenance, which we call a *distributed ledger scheme*. This distinction between

Protocol 1: Π -Ledger $[P, \text{sid}^*, r_0, w, \text{DLS}]$

Building Blocks:

- 1: $\mathcal{G}\text{-Clock}[P']$ with $P \subset P'$
- 2: $\mathcal{F}\text{-Diffuse}[P, \dots]; \mathcal{F}\text{-CRS}[P, \text{DLS.Setup}]$
- 3: $\mathcal{F}\text{-PKI}[P]; \mathcal{F}\text{-Beacon}[P]$

$P_{\text{sid}}.\text{Init}()$:

- 4: $\mathcal{G}\text{-Clock}.P_{\text{sid}^*}.\text{Reg}()$
- 5: $\text{st}[P_{\text{sid}^*}] \leftarrow \perp$ // P 's in session sid^* run the ledger

$P_{\text{sid}}.\text{KGen}()$ // any sid

- 6: **check** once
- 7: $r \leftarrow \mathcal{G}\text{-Clock}.P_{\text{sid}^*}.\text{Round}()$
- 8: **if** $r \neq r_0$ **then return** $\perp$
- 9: $\text{pp} \leftarrow \mathcal{F}\text{-CRS}.P_{\text{sid}^*}.\text{Get}()$
- 10: $\text{st}[P_{\text{sid}^*}].(\text{sk}, \text{vk}) \leftarrow \text{DLS.KGen}(\text{pp})$
- 11: $\mathcal{F}\text{-PKI}.P_{\text{sid}^*}.\text{Reg}(\text{vk}, w[P])$
- 12: $\mathcal{G}\text{-Clock}.P_{\text{sid}^*}.\text{OK}()$
- 13: **return** ok

$P_{\text{sid}}.\text{Genesis}()$ // any sid

- 14: $r \leftarrow \mathcal{G}\text{-Clock}.P_{\text{sid}^*}.\text{Round}()$
- 15: **if** $r \neq r_0 + 1$ **then return** $\text{st}[P_{\text{sid}^*}].G$
- 16: $\text{pp} \leftarrow \mathcal{F}\text{-CRS}.P_{\text{sid}^*}.\text{Get}()$
- 17: $(\text{vk}, w) \leftarrow \mathcal{F}\text{-PKI}.P_{\text{sid}^*}.\text{Get}()$
- 18: $\text{nonce} \leftarrow \mathcal{F}\text{-Beacon}.P_{\text{sid}^*}.\text{Get}()$

- 19: $G \leftarrow \text{DLS.Genesis}(\text{pp}, \text{vk}, \text{nonce})$
- 20: $\text{st}[P_{\text{sid}^*}].G \leftarrow G$
- 21: $\mathcal{G}\text{-Clock}.P_{\text{sid}^*}.\text{OK}()$
- 22: **return** G

$P_{\text{sid}}.\text{Submit}(\text{tx})$ // any sid

- 23: $r \leftarrow \mathcal{G}\text{-Clock}.P_{\text{sid}^*}.\text{Round}()$
- 24: **if** $r \leq r_0 + 1$ **then return** $\perp$
- 25: $m \leftarrow \mathcal{F}\text{-Diffuse}.P_{\text{sid}^*}.\text{Fetch}()$
- 26: $(m; \text{st}[P_{\text{sid}^*}]) \leftarrow \text{DLS.BC}^{\text{DLS.Val}}(r, m, \text{tx}; \text{st}[P_{\text{sid}^*}])$
- 27: $\mathcal{F}\text{-Diffuse}.P_{\text{sid}^*}.\text{Diffuse}(m)$
- 28: $\mathcal{G}\text{-Clock}.P_{\text{sid}^*}.\text{OK}()$
- 29: **return** ok

$P_{\text{sid}}.\text{Chain}()$ // any sid

- 30: **return** $\text{DLS.Chain}(\text{st}[P_{\text{sid}^*}])$

$P_{\text{sid}}.\text{Ledger}()$ // any sid

- 31: **return** $\text{DLS.Ledger}(\text{DLS.Chain}(\text{st}[P_{\text{sid}^*}]))$

$P_{\text{sid}}.\text{Header}()$ // any sid

- 32: **return** $\text{DLS.Header}(\text{DLS.Chain}(\text{st}[P_{\text{sid}^*}]))$

$P_{\text{sid}}.\text{AChain}()$ // any sid

- 33: **return** $\text{DLS.AChain}(\text{DLS.Chain}(\text{st}[P_{\text{sid}^*}]))$

schemes and protocols (which we later also apply to bridges) facilitates the concrete instantiation of ledgers and bridges more compactly by describing only their underlying schemes, while reusing the protocol “boilerplate” for interactions with the infrastructure modeled by the hybrid functionalities.

Distributed ledger scheme. A distributed ledger scheme (DLS) DLS consists of the following PPT algorithms.

$\text{Setup}(\lambda) \rightarrow \text{pp}$: The setup algorithm, on input the security parameter λ , outputs the common parameters pp . This algorithm is subsequently used to parametrize $\mathcal{F}\text{-CRS}$.

$\text{KGen}(\text{pp}) \rightarrow (\text{sk}, \text{vk})$: The maintenance key-generation algorithm, on input the public parameters pp , outputs a key pair used for ledger maintenance (e.g., maintaining a blockchain containing the ledger). This algorithm is typically used in stake-based protocols; to model proof-of-work-type ledgers, we allow KGen to be empty.

$\text{Genesis}(\text{pp}, \text{vk}, w, \text{nonce}) \rightarrow G$: The deterministic genesis block generation algorithm, on input the public parameters pp , a list of verification keys vk (for parties involved in maintaining a chain), corresponding stakes w , and a nonce (generated via a randomness beacon), outputs a genesis block G . The nonce is used to model probabilistic Genesis algorithms that, for example, thwart grinding attacks. For proof-of-work protocols w is not needed.

$\text{Val}(G, L, \text{tx}) \rightarrow b$: The transaction validity predicate decides the validity of a given transaction tx wrt. a given ledger L , and genesis block G .

$\text{BC}^{\text{Val}}(r, m, \text{tx}; \text{st}) \rightarrow (m; \text{st})$: The main maintenance algorithm is a stateful algorithm having subroutine access to Val that, on input a round number r , a list of incoming

network messages m , a transaction tx , and implicit state st , outputs a message m (to be sent out on the network) and an updated state st . We model parties that are silent in a certain round via message $\perp$.

$\text{Chain}(\text{st}) \rightarrow C$: The chain extraction algorithm, on input a state st , outputs a (public) chain C . A chain C is a list of blocks (where a block is a bit string of some given maximal length). Blocks, in addition to transactions, may also contain additional data such as headers, proofs of work, and consensus certificates.

$\text{Ledger}(C) \rightarrow L$: The ledger extraction algorithm, on input a chain C , outputs a list of transactions as the ledger L . Typically, this entails extracting the list of transactions in each block of C , and then “flattening” the result.

$\text{getGenesis}(C)$: Given a chain, this algorithm returns the genesis block of C as output by the Genesis .

$\text{Header}(C) \rightarrow h$: The deterministic header extraction algorithm. On input chain C , it returns some header information h , which may be $\perp$. Overloading, we define $\text{Header}(C, \ell) := \text{Header}(\text{prefix}(C, \ell))$, where the output is referred to as the header information at block height ℓ . Valid headers are required to be collision resistant, a property that is formalized in Appendix E-E.

$\text{AChain}(C) \rightarrow C'$: The deterministic agreed-prefix computation algorithm. On input a chain C , it outputs a prefix $C' \preceq C$. The interpretation is that all honest parties share C' as a prefix (and are aware of it). We also define $\text{apl}(C) := |\text{AChain}(C)|$.

Ledger protocol. A DLS, as defined above, can be combined with a number of standard hybrid functionalities to define a full-

fledged ledger protocol. In more detail, a ledger protocol for parties $\mathbf{P}$, stake distribution $\mathbf{w}$, and delay parameter δ associated with a DLS DLS comprises interfaces $\mathbf{P.KGen}$, $\mathbf{P.Genesis}$, $\mathbf{P.Submit}$, $\mathbf{P.Ledger}$, $\mathbf{P.Chain}$, $\mathbf{P.Header}$, and $\mathbf{P.AChain}$. In essence, these interfaces invoke the corresponding scheme algorithms, but also specify how their inputs are collected, and how outputs are sent to other parties. See Protocol 1 for the details.

Client-server model. For simplicity, we do not distinguish between clients, who would only submit transactions to the ledger, and servers, who maintain the ledger. This distinction can be made by having $\mathbf{BC}$ perform a different task for clients, e.g., only sign transactions and diffuse them (and not register with the clock).

Trust assumptions. Following [28], we formulate our security properties with respect to assumptions explicitly checked by the finalization procedure (as opposed to requiring that adversaries respect them in theorem statements). We formalize assumptions as predicates over a set of parties $\mathbf{P}$, a collection $\mathbf{C}[s]$ of subsets of $\mathbf{P}$ for $s \in [r]$ indicating corrupt parties in round s , and a collection of *ledger views* $\mathbf{L}(\mathbf{P}, s)$ for each party $\mathbf{P} \in \mathbf{P}$, and $s \in [r]$. That is, assumptions take the form

$$\mathbb{A}(r; \mathbf{P}, \mathbf{G}, (\mathbf{C}[s], \mathbf{L}(\mathbf{P}, s))_{\mathbf{P} \in \mathbf{P}, s \in [r]}),$$

where $\mathbb{A}$ is an algorithm that returns a boolean. To ease notation, we will simply write “**check** $\mathbb{A}(r)$ ” in our finalization procedures, meaning that the procedures return false if the assumption is not met. We say that some assumption $\mathbb{A}$ implies assumption $\mathbb{A}'$ if this implication holds for all rounds. We further assume that assumptions are monotonic in that $\forall r. \mathbb{A}(r+1) \Rightarrow \mathbb{A}(r)$ (e.g., if a party is honest in some round $r+1$, it was also honest in the preceding round r).

Ledger security. Employing property-based formulations, we follow the seminal works of Garay, Kiayias, and Leonardos (GKL) [23], Pass, Seeman, and Shelat (PSS) [40], as well as Tzinas, Sridhar, and Zindros (TSZ) [49].

We define ledger security as properties on the execution trace of a (real) UC protocol execution. Our definitions refer to the view of different parties in different rounds to define what an adversarial environment (with a dummy adversary) can and cannot do.

We start with two simple correctness requirements. *Genesis correctness* states that in every round, the genesis block output by $\mathbf{P.Genesis}$ matches that returned by $\mathbf{getGenesis}(\mathbf{C})$, where $\mathbf{C}$ is $\mathbf{P}$ ’s view of the chain. We also require *ledger validity*, which states that in every round, all transactions included in any honest party’s view of the ledger, as computed via $\mathbf{Ledger}(\mathbf{C})$, are valid.

Ledger safety. We follow [49] in defining the safety property, which intuitively requires that the ledgers are *append-only*, and that the ledgers of any two honest parties are consistent in the sense that one is always a prefix of the other.

Ledger δ -liveness. Liveness requires that if an honest party $\mathbf{P}$ submits a transaction at some round r and this transaction *remains valid* for at least δ rounds in the view of all honest

parties that have not already included it in their ledger, then at most δ rounds after r any other honest party $\mathbf{Q}$ will report the transaction in its local view of the ledger.

Agreed prefix. Our bridge constructions will need to report $\mathbf{AChain}$ headers of a sending ledger protocol’s chain on a receiving ledger. To make this formal, we define an *agreed prefix* property which requires that in every round all parties can output an agreed upon prefix of everyone’s chain by applying the $\mathbf{AChain}$ function to their chain. Moreover, the agreed prefix of the ledger at any round is required to extend the agreed prefix of any sufficiently old ledger. This is reminiscent of the “super-safety” property of [49, Definition 13] stated for chains. A second security property is $\mathbf{APGROWTH}$ requiring that the agreed prefix grows with the ledger; that is, any party’s local chain (which grows due to liveness) is contained in the agreed prefix after some delay.

The formal definitions for the liveness, and (super) safety properties above are provided in Finalizations 16 and 17 in Appendix E.

B. Bridges: syntax and security

Following the above approach for distributed ledger security, we define security of blockchain bridges via bridge schemes and bridge protocols. We provide intuition towards the algorithms and interfaces inline. Throughout, we use index i for the *sending ledger* and index j for the *receiving ledger*. Accordingly, we use these indices to indicate their associated algorithms, interfaces, inputs, and outputs. For simplicity, and without loss of generality, we assume that every bridge party is a maintainer on both ledgers (generality arises from the choice of parties to refrain from using the read chain and submit transaction features).

Bridge schemes. A *blockchain bridge scheme* $\mathbf{BBS}$ for a *bridge relation* $\mathcal{R}(\mathbf{x}, \mathbf{C})$, which specifies the properties of a sending chain that a receiving ledger should be able to verify, consists of the following PPT algorithms.

$\mathbf{Setup}(\lambda) \rightarrow \mathbf{pp}$: This is the setup procedure of the bridge, which generates a set of common parameters $\mathbf{pp}$.

$\mathbf{BKGen}(\mathbf{pp}) \rightarrow (\mathbf{bsk}, \mathbf{bvk})$: This is the bridge-maintenance key-generation algorithm.

$\mathbf{RoT}(\mathbf{pp}, \mathbf{bvk}, \mathbf{C}_i, \mathbf{C}_j) \rightarrow (\mathbf{rot}, \mathbf{aux})$: This is the *deterministic root-of-trust* algorithm. Given the public parameters $\mathbf{pp}$, a list of bridge maintenance keys $\mathbf{bvk}$, and the public chain data of the sending and receiving ledgers (one or both of which may be $\perp$ if they are not needed), it outputs the root-of-trust $\mathbf{rot}$ and some auxiliary data $\mathbf{aux}$, to be stored locally. The $\mathbf{rot}$ string is used to bootstrap trust in bridges (e.g., to identify the bridge contract). Informally, it can be seen as a sort of “genesis block” for a bridge but it will typically be more succinct.

$\mathbf{Br}(r, \mathbf{m}, \mathbf{C}_i, \mathbf{C}_j; \mathbf{bst}) \rightarrow (\mathbf{bt}_{x_i}, \mathbf{bt}_{x_j}, \mathbf{m}; \mathbf{bst})$: This is the *stateful bridge-maintenance* algorithm. Given a round number r , incoming messages $\mathbf{m}$, the chains $\mathbf{C}_i$ and $\mathbf{C}_j$, it outputs two sets of transactions $\mathbf{bt}_{x_i}$ and $\mathbf{bt}_{x_j}$ for the two ledgers respectively, as well as a protocol message $\mathbf{m}$ to be sent

Protocol 2: Π -Bridge $[\mathbf{P}, \text{sid}^*, r_0, \text{BBS}]$

Building Blocks: $\parallel i, j$ fixed with $i \neq j$

- 1: $\Pi\text{-Ledger}_i[\mathbf{P}_i, \dots]$ with $\mathbf{P} \subseteq \mathbf{P}_i$ inhering $\mathcal{G}\text{-Clock}[\mathbf{P}']$
- 2: $\Pi\text{-Ledger}_j[\mathbf{P}_j, \dots]$ with $\mathbf{P} \subseteq \mathbf{P}_j$ & same $\mathcal{G}\text{-Clock}[\mathbf{P}']$
- 3: $\mathcal{F}\text{-Diffuse}[\mathbf{P}, \dots]; \mathcal{F}\text{-CRS}[\mathbf{P}, \text{BBS.Setup}]; \mathcal{F}\text{-PKI}[\mathbf{P}]$

P.Init():

- 4: $\mathcal{G}\text{-Clock.P}_{\text{sid}^*}.\text{Reg}()$
- 5: $\text{bst}[\mathbf{P}] \leftarrow \perp$ $\parallel$ party-specific state

P.BKGen() once

- 6: $r \leftarrow \mathcal{G}\text{-Clock.P}_{\text{sid}^*}.\text{Round}()$
- 7: **if** $r \neq r_0$ **then** $\mathcal{G}\text{-Clock.P}_{\text{sid}^*}.\text{OK}()$
- 8: $\text{pp} \leftarrow \mathcal{F}\text{-CRS.P.Get}()$
- 9: $(\text{bsk}, \text{bvk}) \leftarrow \text{BBS.BKGen}(\text{pp})$
- 10: $\text{bst}[\mathbf{P}].(\text{P}, \text{bsk}, \text{bvk}) \leftarrow (\text{P}, \text{bsk}, \text{bvk})$
- 11: $\mathcal{F}\text{-PKI.P.Reg}(\text{bvk})$
- 12: $\mathcal{G}\text{-Clock.P}_{\text{sid}^*}.\text{OK}()$
- 13: **return** ok

P.RoT() once

- 14: $r \leftarrow \mathcal{G}\text{-Clock.P}_{\text{sid}^*}.\text{Round}()$
- 15: **if** $r \neq r_0 + 1$ **then return** $\text{bst}[\mathbf{P}].\text{rot}$
- 16: $\text{pp} \leftarrow \mathcal{F}\text{-CRS.P.Get}()$
- 17: $\text{bvk} \leftarrow \mathcal{F}\text{-PKI.P.Get}()$
- 18: $\mathbf{C}_i \leftarrow \Pi\text{-Ledger}_i.\text{P}_{\text{sid}^*}.\text{AChain}()$
- 19: $\mathbf{C}_j \leftarrow \Pi\text{-Ledger}_j.\text{P}_{\text{sid}^*}.\text{AChain}()$
- 20: $(\text{rot}, \text{aux}) \leftarrow \text{BBS.RoT}(\text{pp}, \text{bvk}, \mathbf{C}_i, \mathbf{C}_j)$
- 21: $\text{bst}[\mathbf{P}].(\text{rot}, \text{aux}) \leftarrow (\text{rot}, \text{aux})$
- 22: $\mathcal{G}\text{-Clock.P}_{\text{sid}^*}.\text{OK}()$
- 23: **return** rot

P.Bridge() once

- 24: $r \leftarrow \mathcal{G}\text{-Clock.P}_{\text{sid}^*}.\text{Round}()$
- 25: **if** $r \leq r_0 + 1$ **then** $\mathcal{G}\text{-Clock.P}_{\text{sid}^*}.\text{OK}()$
- 26: $\mathbf{C}_i \leftarrow \Pi\text{-Ledger}_i.\text{P}_{\text{sid}^*}.\text{Chain}()$
- 27: $\mathbf{C}_j \leftarrow \Pi\text{-Ledger}_j.\text{P}_{\text{sid}^*}.\text{Chain}()$
- 28: $\mathbf{m} \leftarrow \mathcal{F}\text{-Diffuse.P.Fetch}()$
- 29: $(\text{bt}_i, \text{bt}_j, \mathbf{m}; \text{bst}[\mathbf{P}]) \leftarrow \text{BBS.Br}(r, \mathbf{m}, \mathbf{C}_i, \mathbf{C}_j; \text{bst}[\mathbf{P}])$
- 30: $\mathcal{F}\text{-Diffuse.P.Diffuse}(\mathbf{m})$
- 31: $\Pi\text{-Ledger}_i.\text{P}_{\text{sid}^*}.\text{Submit}(\text{bt}_i)$
- 32: $\Pi\text{-Ledger}_j.\text{P}_{\text{sid}^*}.\text{Submit}(\text{bt}_j)$
- 33: $\mathcal{G}\text{-Clock.P}_{\text{sid}^*}.\text{OK}()$
- 34: **return** ok

P.Certify(rot, x)

- 35: **if** $\mathbf{P} \notin \mathbf{P}_i$ **then return** $\perp$
- 36: $\mathbf{C}_i \leftarrow \Pi\text{-Ledger}_i.\text{P}_{\text{sid}^*}.\text{Chain}()$
- 37: $\pi \leftarrow \text{BBS.Certify}(\text{rot}, \mathbf{x}, \mathbf{C}_i)$
- 38: **return** π

P.Verify(rot, x, π)

- 39: **if** $\mathbf{P} \notin \mathbf{P}_j$ **then return** $\perp$
- 40: $r \leftarrow \mathcal{G}\text{-Clock.P}_{\text{sid}^*}.\text{Round}()$
- 41: $\mathbf{L}_j \leftarrow \Pi\text{-Ledger}_j.\text{P}_{\text{sid}^*}.\text{Read}()$
- 42: $\text{mem}_j \leftarrow \text{BBS.getVerData}(\mathbf{G}_j, \text{rot}, \mathbf{L}_j)$
- 43: **return** $\text{BBS.Verify}(\text{mem}_j, \mathbf{x}, \pi)$

to other parties. The algorithm also updates its internal bridge state.

$\text{Certify}(\text{rot}, \mathbf{x}, \mathbf{C}) \rightarrow \pi$: This is the property certification algorithm. Given the root of trust, a statement $\mathbf{x}$, and sending chain data $\mathbf{C}$, it computes a proof π .

$\text{Verify}(\text{mem}, \mathbf{x}, \pi) \rightarrow b$: This is the property verification algorithm. Given contract memory mem (discussed below), a statement $\mathbf{x}$ and a proof π , it returns a bit b .

The primary purpose of a bridge is to provide smart contracts on the receiving ledger with the ability to verify statements about the sending chain. Thus our bridges (e.g., the ATS bridge constructed below in Section IV) deploy, on the receiving ledger, a contract and a specific method $\text{verify}(\mathbf{x}, \pi)$ that checks $\text{Verify}(\text{mem}, \mathbf{x}, \pi)$. Here mem is the contract memory, which can be recovered via the ledger.

Bridge protocols. Similar to ledger protocols, given a bridge scheme BBS, a set of bridge parties $\mathbf{P}$, a delay parameter Δ (which may be different from the delay parameters of ledgers), we define the associated bridge protocol $\Pi\text{-Bridge}$ wrt. a sending ledger protocol $\Pi\text{-Ledger}_i$ and receiving ledger protocol $\Pi\text{-Ledger}_j$ as shown in Protocol 2. This protocol consists of interfaces $\text{P.BKGen}()$, $\text{P.RoT}()$, $\text{P.Bridge}()$, $\text{P.Certify}(\text{rot}, \mathbf{x})$, and $\text{P.Verify}(\text{rot}, \mathbf{x}, \pi)$. Note that P.Certify and P.Verify directly take rot as input and thus the parties need access to an authentic copy of the root-of-trust string rot . The interpretation is that rot is distributed via an authentic

off-band mechanism, e.g., through a web service that securely distributes the bridge software. The P.Verify interface uses a verification-data-extraction algorithm getVerData that given the genesis block, the root of trust, and the receiving ledger, returns the bridge contract memory mem . (Formally, via the contractState function, which we define in Appendix E-B.) Further details are provided in Appendix G.

Bridge assumptions. Bridge trust assumptions subsume assumptions about the sending and receiving ledgers:

$$\mathbb{A}(r; \mathbf{P}, \mathbf{P}_i, \mathbf{P}_j, \mathbf{G}_i, \mathbf{G}_j, (\mathbf{C}[s], \mathbf{L}_i(\mathbf{P}, s), \mathbf{L}_j(\mathbf{Q}, s)))_{\mathbf{P} \in \mathbf{P}_i, \mathbf{Q} \in \mathbf{P}_j, s \in [r]}.$$

For brevity, we also refer to bridge assumptions by $\mathbb{A}(r)$. Whether an assumption is for ledgers or bridges will be clear from the context. For example, an assumption might require that at most a ρ fraction of sending ledger parties are corrupt (e.g., for soundness) whilst allowing at most a ρ' fraction on the receiving ledger (for liveness).

Bridge soundness. A sound bridge guarantees that, under some trust assumption $\mathbb{A}$, if some statement about the sending ledger is verified on the receiving ledger, then that property indeed holds for the sending ledger with respect to *all* honest parties' ledgers in a preceding round. More formally, suppose a statement $\mathbf{x}$ and an (adversarial) proof π for it verify on the receiving ledger wrt. $\mathbf{L}_j(\mathbf{Q}, r)$ and a rot that was output by some honest party. Then there exists a round $r' \leq r$ such that

Finalization 3: Bridge Soundness and Liveness

A-BRIDGE SOUNDNESS($P, Q, r, \text{rot}, x, \pi$)

```

1: check  $\mathbb{A}(r)$ 
2: check  $r \leq r_{\max}$ 
3: check  $P \notin C[r_0 + 1] \wedge Q \notin C[r]$ 
4: check  $(\_, P.\text{RoT}, \_, \text{rot}) \in \text{view}[r_0 + 1]$ 
5:  $\text{mem} \leftarrow \text{getVerData}(G_j, \text{rot}, L_j(Q, r))$ 
6: check  $\text{Verify}(\text{mem}, x, \pi)$ 
7: return  $\neg(\exists r' \leq r. \forall P' \in \mathbf{P}_i \setminus C[r']. \mathcal{R}(x, C_i(P', r')))$ 

```

($\mathbb{A}, \delta$)-BRIDGE LIVENESS($P, Q, r, r', \text{rot}, x, \pi$)

```

8: check  $\mathbb{A}(r')$ 
9: check  $r + \delta \leq r' \leq r_{\max}$ 
10: check  $P \notin C[r] \wedge Q \notin C[r']$ 
11: check  $(\_, P.\text{RoT}, \_, \text{rot}) \in \text{view}[r_0 + 1]$ 
12: check  $\mathcal{R}(x, C_i(P, r))$ 
13: check  $(\_, P.\text{Certify}, x, \pi) \in \text{view}[r]$  //  $\pi = \perp$  allowed
14:  $\text{mem} \leftarrow \text{getVerData}(G_j, \text{rot}, L_j(Q, r'))$ 
15: return  $\neg \text{Verify}(\text{mem}, x, \pi)$ 

```

statement x is in fact true with respect to the ledgers of *all* parties P' that are honest in r' . (See Finalization 3, left.)

Bridge liveness. A live bridge ensures successful verification of a true property of a sending ledger on a receiving ledger. More formally, suppose we are given a sending ledger $L_i(P, r)$ from an honest party P 's view in round r with respect to which a statement x holds according to the relation $\mathcal{R}$. Then if a proof π for x is obtained in this round, it will verify on the receiving ledger j from any honest party Q 's view in any round r' , as long as r' is at least δ rounds after r . This property is formalized in Finalization 3, right.

Note that, as part of these definitions, the adversary supplies the root of trust rot , and the properties are only required to hold if rot matches the value output $P.\text{RoT}$ of an honest party P . Our bridges also meet a simple “RoT agreement” property, which states that all honest parties output the same rot .

We finally note that since bridge state spaces are separated from both other bridges and ledgers, a simple hybrid argument shows that one can safely run multiple (independent) bridges across multiple ledgers, with some bridges possibly operating in opposite directions. For the same reason, running a bridge alongside a ledger does not affect the security of the ledgers.

IV. THE ATS BRIDGE

A. ATS: improved game-based definitions

The concept of *Ad-hoc / Advanced Threshold Signatures* (ATS) was originally introduced in [28] to address the problem of enabling side-chain nodes to succinctly authenticate to mainchain nodes without requiring the mainchain to directly observe the side-chain. In other words, the goal was to build a bridge between these ledgers.²

More formally, an ATS scheme consists of the following algorithms.

Setup(λ) $\rightarrow$ pp: The setup algorithm. It takes a security parameter λ and generates system parameters pp.

Gen(pp) $\rightarrow$ (sk, pk): The key generation algorithm. It takes the system parameters pp and generates a key pair (sk, vk).

Sig(sk, m) $\rightarrow$ σ : The signing algorithm that takes a secret key and a message, and outputs a signature.

Ver(vk, m, σ) $\rightarrow$ b: The deterministic verification algorithm that takes a verification key vk, a message m, and a signature σ , and returns a bit.

AKey(vk) $\rightarrow$ avk: The key aggregation algorithm. Given a list of possibly non-unique verification keys, $[vk_1, \dots, vk_n]$, it deterministically generates an aggregate verification key avk.

ACheck(avk, vk) $\rightarrow$ b: The aggregation check algorithm. Given a list of verification keys vk and an aggregate verification key avk, it deterministically checks whether avk is the aggregate key for vk.³

ASig(m, vk, $\{(\text{vk}_i, \sigma_i) : i \in [k]\}$) $\rightarrow$ σ : The signature aggregation algorithm that takes a message m, a set of verification key/signature pairs (vk_i, σ_i) , and aggregates the signatures into a threshold signature τ .

AVer(avk, m, τ) $\rightarrow$ b: The aggregate verification algorithm. Given a message m, an aggregate verification key avk, and an aggregate threshold signature τ , this algorithm deterministically returns a bit.

ATS correctness and security. We define the security of an ATS scheme through a (standard) game, shown in Game 1, between a challenger and an adversary. In this game, the adversary can request the ATS public parameters once at the onset, generate keys for each party, and corrupt parties to obtain their secret keys. The adversary also has access to a signing oracle that generates signatures on messages of its choice for a given party. The game tracks which parties have signed which messages in a list **Signers**, indexed by messages and containing verification keys. The game also keeps track of the signatures generated in a set **Sigs**, indexed by party-message pairs.

The t -UF-CMA property requires that no adversary is able to non-trivially compute a list of verification keys vk, containing honest, corrupt, or newly generated malicious keys, an associated avk that passes ACheck, and an aggregate message-signature pair (m, τ) that verifies. The winning condition requires that the number of signatures that the adversary can obtain trivially (be it via oracle calls, corruption or malicious keys) is bounded by the threshold t . If vk contains duplicated entries, we count them with multiplicity.

²Interestingly, their paper does not mention blockchain bridges once as it predates the popularity of this concept.

³ACheck is redundant due to AKey being deterministic, but we include it for compatibility with the GZK ATMS formulation.

Game 1: ATS t -Unforgeability

```

Initialize()    // once at the onset
1:  $\mathbf{C} \leftarrow \mathbf{H} \leftarrow \emptyset$ ;  $\text{VK}[*] \leftarrow \text{SK}[*] \leftarrow \square$ 
2:  $\text{Signers}[*] \leftarrow \text{Sigs}[*,*] \leftarrow \emptyset$ 
3:  $\text{pp} \leftarrow \text{ATS.Setup}()$ 
4: return  $\text{pp}$ 

Gen( $i$ )
5: if  $\text{VK}[i] = \perp$  then
6:    $(\text{SK}[i], \text{VK}[i]) \leftarrow \text{ATS.Gen}(\text{pp})$ 
7:    $\mathbf{H} \leftarrow \mathbf{H} \cup \{\text{VK}[i]\}$ 
8: return  $\text{VK}[i]$ 

Corrupt( $i$ )
9:  $\mathbf{C} \leftarrow \mathbf{C} \cup \{\text{VK}[i]\}$ ;  $\mathbf{H} \leftarrow \mathbf{H} \setminus \{\text{VK}[i]\}$ 
10: return  $\text{SK}[i]$ 

Sign( $i, m$ )
11: check  $\text{VK}[i] \in \mathbf{H}$ 
12:  $\sigma \leftarrow \text{ATS.Sig}(\text{SK}[i], m)$ 
13:  $\text{Signers}[m] \leftarrow \text{Signers}[m] \cup \{\text{VK}[i]\}$ 
14:  $\text{Sigs}[i, m] \leftarrow \text{Sigs}[i, m] \cup \{\sigma\}$ 
15: return  $\sigma$ 

 $t$ -UF-CMA( $\text{vk}, \text{avk}, m, \tau$ )
16:  $\text{vk}' \leftarrow \{\text{vk} \in \text{vk} : \text{vk} \in (\mathbf{C} \cup \text{Signers}[m]) \vee \text{vk} \notin \text{VK}\}$ 
17: check  $|\text{vk}'| \leq t$ 
18: check  $\text{ATS.ACHeck}(\text{avk}, \text{vk})$ 
19: return  $\text{ATS.AVer}(\text{avk}, m, \tau)$ 

```

In Appendix I, in Game 5 we extended this definition to cover correctness with respect to aggregation, signing and aggregate signing.

B. The ATS ideal functionality

To facilitate model checking, which relies on *ideally secure* building blocks, we define an ideal functionality for ATS schemes. To the best of our knowledge, this functionality is novel; despite its complexity, it has been thoroughly checked for flaws. We outline this functionality below and describe how its design prevents (ideal-world) adversaries from breaking its correctness and unforgeability properties. This functionality is potentially of interest in other contexts involving threshold signatures [4], [25].

Functionality $\mathcal{F}$ -ATS comprises interfaces that match the algorithms of an ATS scheme, with the exception of setup, which, consistent with UC approaches, is provided by independent idealized building blocks formulated via setup assumptions. The interfaces defer to the simulator in most cases, but they also enforce consistency, security, and correctness by maintaining state across interfaces, sanitizing simulator outputs, and enforcing the scheme's logic. We present its full specification in Functionality 24 of Appendix I-A.

C. Committee selection

The ATS bridge construction requires the selection of a *committee* to produce an aggregate ATS signature on the headers of the sending ledger. We abstract this committee selection process via an algorithm comSel that takes a chain state $\mathbf{C}$ and outputs a committee $\text{Com} \subseteq \mathbf{P}$. This committee is selected every R rounds, called an *epoch*, during which a fixed

Finalization 4: Committee Selection Security

```

 $(\mathbb{A}, c, \rho, \delta)$ -SECURECOMSEL( $\mathbf{P}, r, r'$ )
1: check  $\mathbb{A}(r')$ 
2: check  $r \leq r' \leq r + \delta$ 
3: check  $\mathbf{P} \notin \mathbf{C}[r]$ 
4:  $\text{Com} \leftarrow \text{comSel}[c, \rho](\text{AChain}(\mathbf{C}(\mathbf{P}, r)))$ 
5:  $\text{Com}_c \leftarrow [\mathbf{P}' \in \text{Com} : \mathbf{P}' \in \mathbf{C}[r']]$ 
6:  $\text{win}_{\text{cor}} \leftarrow \neg(|\text{Com}| = c \wedge \text{Com} \subseteq \mathbf{P})$ 
7:  $\text{win}_{\text{sec}} \leftarrow \neg(|\text{Com}_c| \leq \rho \cdot |\text{Com}|)$ 
8: return  $\text{win}_{\text{cor}} \vee \text{win}_{\text{sec}}$ 

```

stake distribution is used for sampling slot leaders (typically that of the beginning of the epoch). Formally, we introduce the following auxiliary ledger algorithm.

$\text{comSel}(\mathbf{C})$: The deterministic committee selection algorithm that on input a chain $\mathbf{C}$ outputs a committee $\text{Com} \subseteq \mathbf{P}$, where $\mathbf{P}$ is the set of ledger maintainers. Com may have repeated entries.

The committee Com is required to satisfy certain conditions. Concretely, in our setting, for given parameters c and ρ we require Com to be of size exactly c such that at most a ρ fraction of the parties in Com , counted with multiplicity, are corrupt. This property is required to hold conditionally on the ledger satisfying its trust assumption $\mathbb{A}$. We formalize this property in Finalization 4, where the underlying execution model is that of a ledger (see Protocol 1). If the trust assumptions on the ledger imply those for the committee selection (for example when they are the same assumption), a secure committee-selection process allows one to *transfer* the security of the ledger to that of the committee running the bridge, thereby avoiding the introduction of additional trust assumptions. We note that this formalizes a *public* selection process as $\mathbf{C}$ is public.

A number of committee-selection procedures (implicit in the ledgers) that produce small committees under certain mild assumptions on stake distribution (e.g., that there are parties holding approximately a $1/n$ fraction of stake for a committee of size n) are presented in [26].

D. The ATS bridge

We now explain the high-level intuition behind the ATS bridge scheme before presenting it in pseudo-code. Let $R \geq \Delta + 3$ denote the length of an epoch and let (c, ρ) be the parameters for the committee selection described above for the sending ledger; that is, comSel_i always returns a committee of size c such that at most $\rho \cdot c$ members of that committee are corrupt. We set the ATS parameter $t := \rho c$.

The bridge setup algorithm runs the setup algorithms of the ATS scheme and the proof system. Bridge maintenance keys consist of a pair of ATS keys. The root-of-trust algorithm selects a committee from the sending chain maintainers (thus, $\mathbf{P}_i = \mathbf{P}$) and computes the aggregate verification key for this committee using keys sampled at the key-generation phase. This aggregate key and the output of setup form the root of trust. Parties store the list of bridge maintenance verification

Bridge Scheme 1: ATS

```

Setup( $\lambda$ )
1:  $pp \leftarrow \text{ATS.Setup}(\lambda)$ 
2:  $crs \leftarrow \text{SNARK.Setup}(\lambda)$ 
3: return ( $pp, crs$ )

BKGen( $(pp, crs)$ ) when  $r = r_0$ 
4: return  $\text{ATS.KGen}(pp)$ 

RoT( $(pp, crs), \text{bvk}, C_i, C_j$ ) when  $r = r_0 + 1$ 
5:  $\text{Com} \leftarrow \text{comSel}_i(\text{AChain}_i(C_i))$ 
6:  $\text{avk} \leftarrow \text{ATS.AKey}([\text{bvk}[P] : P \in \text{Com}])$ 
7:  $\text{rot} \leftarrow (pp, crs, \text{avk}); \text{aux} \leftarrow (\text{bvk}, \text{Com})$ 
8: return ( $\text{rot}, \text{aux}$ )

Br( $r, \text{m}, C_i, C_j; \text{bst}$ ) when  $r = r_0 + 2$ 
9:  $\text{rot} \leftarrow \text{bst.rot}$ 
10:  $\text{code} \leftarrow \text{ATSBridge}[\text{rot}]$ 
11:  $\text{cID} \leftarrow \text{Hash}(\text{code}); \text{bst.cID} \leftarrow \text{cID}$ 
12:  $\text{btX}_j \leftarrow (\text{cID}, \text{install}, \text{code})$ 
13: return ( $\perp, \text{btX}_j, \perp; \text{bst}$ )

Br( $r, \text{m}, C_i, C_j; \text{bst}$ ) when  $r \in [r_0 + 3, R(\lfloor r_0/R \rfloor + 1) - 1]$ 
14: return ( $\perp, \perp, \perp; \text{bst}$ ) // wait for next epoch

Br( $r, \text{m}, C_i, C_j; \text{bst}$ ) when  $r \equiv 0 \pmod R$ 
15:  $P \leftarrow \text{bst.P}; \text{sk} \leftarrow \text{bst.bsk}; \text{aux} \leftarrow \text{bst.aux}$ 
16:  $(\text{bvk}, \text{Com}) \leftarrow \text{parse}(\text{aux})$ 
17:  $\text{Com}' \leftarrow \text{comSel}_i(\text{AChain}_i(C_i))$ 
18:  $\text{avk}' \leftarrow \text{ATS.AKey}([\text{bvk}[P] : P \in \text{Com}'])$ 
19:  $h \leftarrow \text{Header}_i(C_i, \text{apl}_i(C_i))$ 
20:  $\sigma \leftarrow \text{ATS.Sig}(\text{sk}, (h, \text{avk}'))$ 
21:  $\text{bst.aux}' \leftarrow (\text{bvk}, \text{Com}')$ 
22:  $(\text{bst.h}, \text{bst.avk}') \leftarrow (h, \text{avk}')$ 
23:  $\text{bst.m} \leftarrow \text{bst.m} : \text{m}$ 
24: if  $P \notin \text{Com}$  then return ( $\perp, \perp, \perp; \text{bst}$ )
25: return ( $\perp, \perp, (P, \text{bvk}[P], \sigma); \text{bst}$ )

Br( $r, \text{m}, C_i, C_j; \text{bst}$ ) when  $r \in [1, \Delta + 1] \pmod R$ 
26:  $\text{bst.m} \leftarrow \text{bst.m} : \text{m}$ 
27: return ( $\perp, \perp, \perp; \text{bst}$ )

Br( $r, \text{m}, C_i, C_j; \text{bst}$ ) when  $r \equiv \Delta + 2 \pmod R$  // aggregation
28:  $(\text{bvk}, \text{Com}) \leftarrow \text{bst.aux}; \text{cID} \leftarrow \text{bst.cID}$ 
29:  $(h, \text{avk}') \leftarrow (\text{bst.h}, \text{bst.avk}')$ 
30:  $\Sigma \leftarrow [(\text{bvk}[P], \sigma) : (P, \text{bvk}[P], \sigma) \in \text{bst.m} \wedge P \in \text{Com} \wedge \text{ATS.Ver}(\text{bvk}[P], (h, \text{avk}'), \sigma)]$ 
31:  $\tau \leftarrow \text{ATS.ASig}((h, \text{avk}'), [\text{bvk}[P] : P \in \text{Com}], \Sigma)$ 
32:  $\text{btX}_j \leftarrow (\text{cID}, \text{update}, (h, \text{avk}', \tau))$ 
33:  $\text{bst.m} \leftarrow []; \text{bst.aux} \leftarrow \text{bst.aux}'$ 
34: return ( $\perp, \text{btX}_j, \perp; \text{bst}$ )

Br( $r, \text{m}, C_i, C_j; \text{bst}$ ) when  $r \in [\Delta + 3, R - 1] \pmod R$ 
35: return ( $\perp, \perp, \perp; \text{bst}$ ) // wait for next epoch

Certify( $\text{rot}, x, C$ )
36:  $h \leftarrow \text{Header}_i(C, \text{apl}_i(C))$ 
37:  $\pi' \leftarrow \text{SNARK.Prove}(\text{rot.crs}, (x, h), C)$ 
38:  $\pi \leftarrow (h, \pi'); \text{return } \pi$ 

Verify( $\text{mem}, x, \pi$ )
39:  $\text{crs} \leftarrow \text{mem.crs}; (h, \pi') \leftarrow \pi$ 
40: check  $h \in \text{mem.hs}$ 
41: return  $\text{SNARK.Verify}(\text{crs}, (x, h), \pi')$ 

ATSBridge[rot].update( $h, \text{avk}', \tau$ )
42:  $(pp, crs, \text{avk}_0) \leftarrow \text{parse}(\text{rot})$ 
43:  $\text{avk} \leftarrow \text{mem.avk}$ 
44: if  $\text{avk} = \perp$  then  $\text{avk} \leftarrow \text{avk}_0$ 
45: check  $h \notin \text{mem.hs} \wedge \text{ATS.AVer}(\text{avk}, (h, \text{avk}'), \tau)$ 
46:  $\text{mem.hs} \leftarrow \text{mem.hs} : h; \text{mem.avk} \leftarrow \text{avk}'$ 
47: return ok

ATSBridge[rot].verify( $x, \pi$ )
48: return Verify( $\text{mem}, x, \pi$ )

```

keys and the initially selected committee locally as auxiliary data.

There are different cases for the main bridge algorithm Br depending on r and the initial round r_0 , as described next.

$r = r_0 + 2$: The bridge deploys the supporting smart contract ATSBridge on chain C_j .

r from $r_0 + 3$ to just before the next epoch: The bridge parties wait for the next epoch.

$r \equiv 0 \pmod R$: Parties rerun the committee-selection algorithm to determine the incoming committee for the *next* epoch. They then compute avk' , the aggregate key of that committee, as well as the header value h for the agreed prefix of the epoch. They also check the existing, outgoing committee for the *current* epoch to determine if they are members. If so, they sign (avk', h) and diffuse the signature.

Next $\Delta + 1$ rounds: The bridge protocol waits $\Delta + 1$ rounds for signatures to diffuse.

$r \equiv \Delta + 2 \pmod R$: All signatures have been diffused and collected, so that they can be aggregated by any party. The updated (h, avk') and its signature are submitted to the bridge contract on C_j .

$r \in [\Delta + 3, R - 1] \pmod R$: The bridge sleeps until the next epoch. (When $r \equiv 0 \pmod R$ the bridge proceeds as specified above.)

We present the details in Bridge Scheme 1. Here, the bridge contract ATSBridge has two methods. The `update` method receives arguments of the form (h, avk') supported by aggregate signatures wrt. an existing avk . If verification is successful, avk is updated to avk' and the header is appended to the contract memory. The `verify` method verifies statements using mem , the contract memory.

Whilst we prove security against static adversaries, our formulation of committee selection is compatible with adaptive ones, albeit, with an implied corruption delay. Effectively, we need the committee member to stay honest from the time of selection, until the bridge contract updates. If we also introduce *key rotation* and secure erasures, this delay can be shortened: keys will rotate when the committee signs, so members need to stay uncorrupted only until that point. Aggregation and updating the bridge contract can take place even after the committee is corrupted.

In Theorems 10 and 11 in Appendix I-C we show that the ATS bridge constructed above is sound and live as long as

the underlying ledgers, the ATS, the committee selection, and the proof system are secure and correct. We state an informal version below.

Theorem 1 (ATS Bridge, informal). *The ATS bridge (protocol Π -Bridge wrt. Bridge Scheme 1) from a sending ledger Π -Ledger_i to a receiving ledger Π -Ledger_j provides bridge soundness and liveness under the following conditions: Both ledgers satisfy ledger validity, Π -Ledger_i satisfies secure committee selection and has collision-resistant headers, Π -Ledger_j satisfies ledger liveness, ATS satisfies t-UF-CMA and correctness, and SNARK is complete and knowledge sound.*

V. MODEL CHECKING IN QUINT

Bounded model checking is a formal verification technique that systematically searches for traces which violate the specification. By encoding system behavior and security properties into logical formulas, model checking uses SAT/SMT solvers to find counterexamples (bugs) that violate those properties. This methodology is justified by the so-called “small scope hypothesis” [30], stating that in (natural) systems, bugs arise within traces that have relatively few objects.

We carry out a systematic model-checking of our UC protocols, which is a novel aspect of our work, and potentially of wider interest. Model-checking such protocols is motivated in part by subtleties in UC’s operational semantics. Indeed, the specification of UC functionalities that we use appears deceptively simple: they are often interpreted as simple programming with procedure calls. However, the underlying operational semantics allows *resumptions*. Intuitively, this means that any call to the simulator pauses the execution of the functionality and allows the simulator to hand control back to the environment. The latter can then call the functionality again. Moreover, whenever the simulator is activated again, it has the option of resuming any pending calls from the functionality.

We model check the ATS bridge, including all its underlying building blocks, in Quint [44]. Quint is a specification language and a model checker with a focus on distributed and concurrent systems. Quint’s theoretical foundations are closely related to TLA⁺ [35], [36] and support first-order logic, state-transition systems, and linear temporal logic with actions. However, Quint has a more user-friendly syntax, supports various modes of verification and temporal properties, and is statically typed and effect aware.

By implementing our system in Quint we obtain executable specifications, which can be analyzed in a variety of ways. For example, we can: (i) execute the system for specific adversaries and environments (i.e., perform tests); this is a basic but limited form of guarantee that we can easily obtain; (ii) use Quint simulations to produce a large set of traces; (iii) convert Quint code to TLA⁺ to use its toolset for further analysis or to the Apache SMT solver [33], [34] for formal verification. We use Quint 0.29.1 (with Rust backend) and Apache 0.51.1.

We checked each functionality used in the ATS bridge (see Table I) against a variety of internal state and trace properties. For example, we checked:

Target	Properties checked	Sim (samp/s)	Ver (time)	Size (loc)
$\mathcal{G}$ -Clock	Agreement, Liveness, Soundness	100	40s	536
$\mathcal{F}$ -PKI	PKI State Consistency	150	120s	700
$\mathcal{F}$ -Diffuse	Δ -Liveness, Consistency	100	6m	1125
$\mathcal{F}$ -ATS	Functional, Injective, Deterministic, Correctness, Unforgeability	20	7m	2145
$\mathcal{F}$ -NIZK	Soundness, Completeness, Consistency	100	4m	1384
$\mathcal{G}$ -Ledger	Liveness, Persistence, ComSel Security, Prefix Agreement	15	8m	2344
ATS Bridge	Liveness, Soundness, Output Consistency	5	–	3458

Table I

- The lists used to define each functionality are *functional* in that there are no inconsistencies in their entries. Moreover, those that are intended to be *injective* (e.g., VK in $\mathcal{F}$ -ATS) remain injective throughout the execution.
- Certain interfaces (such as Gen, Ver, AVer in $\mathcal{F}$ -ATS) are deterministic and always provide the same output on the same inputs. Other interfaces are *available* in that they always provide a non- $\perp$ output (as typically ensured by sanitizing simulator outputs).
- The ideal functionalities meet their expected *correctness* properties. For example, in $\mathcal{F}$ -ATS honestly generated plain (resp. aggregated) signatures always verify. Similarly, in $\mathcal{F}$ -NIZK, honestly generated proofs always verify. In $\mathcal{F}$ -Diffuse, messages that were sent Δ rounds ago are successfully fetched. For $\mathcal{G}$ -Ledger and Π -Bridge[ATS] we check ledger liveness and bridge liveness, that is, proved statements eventually verify.
- The ideal functionalities meet their security properties. For example, $\mathcal{F}$ -ATS signatures are unforgeable under threshold conditions, and parties in $\mathcal{G}$ -Ledger have approximately consistent views of the ledger. We check the ATS bridge (Π -Bridge with respect to Bridge Scheme 1) for soundness.

Table I summarizes our experimental results. Here, **Sim** denotes a “random walk” simulation strategy, while **Ver** represents an exhaustive exploration of a finite model. For **Sim**, we employ long executions (up to 1000 steps) on larger models. For **Ver**, we rely on shorter traces (up to length 10) and smaller universes (up to 5 objects per entity), as the tool terminates only when the entire state space is exhausted. We report samples per second (samp/s) and total verification time in seconds (s) and minutes (m). Due to system complexity, the toolchain cannot currently support exhaustive verification for larger instances; however, we note that simulation is often the more widely practiced method. Experiments were run on an Intel® Core™ Ultra 9 with 24 cores and 32GB of RAM. The full code is attached as supplementary material.

To avoid state space explosion, each functionality is parameterized by small finite sets from which we draw inputs and out-

puts. (For example, a finite set of all possible verification keys.) We control these sizes via a parameter `Universe_Size`.

In many cases, we uncovered bugs, some arguably subtle, which lead to robust definitions of various standard ideal functionalities. It is worth noting that, while writing the Quint code, many bugs were caught simply by static type-checking.

For a more subtle bug, consider the example of the well-known $\mathcal{F}$ -Sig functionality, included in Appendix C (cf. Ver. Bad and Ver. Good). This definition is only about 20 loc. It is fairly simple to convince oneself that Ver. B is indeed a good and fairly detailed idealization of digital signatures. However, when we implemented it in Quint, we immediately ran into an issue due to the resumption semantics mentioned above. Specifically, we discovered that this specification suffers from a “flip-flop” bug that allows the functionality to respond to verify requests inconsistently (i.e., the same request produced different responses). Quint reports a trace for the following attack: 1) the environment calls $P.\text{Verify}(vk, m, \sigma)$; 2) the simulator yields the execution to the environment; 3) the environment calls $P.\text{Gen}$ and the simulator allows it to run until the end; 4) the environment calls $P.\text{Sign}(m)$, which the simulator allows to run until the end; 5) the environment calls $P.\text{Verify}(vk, m, \sigma)$, where the simulator responds with 1; 6) the environment activates the simulator which in turn resumes the initially suspended execution of $P.\text{Verify}(vk, m, \sigma)$ producing output 0. We found similar deficiencies in nearly all initial UC specifications that we had.

Our model checking relies on *idealized* building blocks, and thus matches UC’s approach to security well. Indeed, model checking is typically non-probabilistic, meaning that Quint enumerates all possible inputs and responses from pre-defined bounded spaces. Even a *single* set of inputs and responses that violate a given property is not tolerated. Accordingly, we formulate idealized versions of the ATS scheme (Appendix I-A), and the ledger including committee selection and agreed-prefix length (Appendix F).

Model checking complements pen-and-paper proofs and substantially increases our confidence in the specifications of the overall bridge design and its building blocks. It also serves as a first (relatively low-cost) step towards the full formal verification of proofs and properties. This effort was made possible, in part, through unambiguous and falsifiable code-based specifications of various system components. Our work demonstrates the practical applicability of model checking to other complex UC protocols, whose specifications may rely on multiple building blocks. We hope this methodology helps create a robust library of precisely specified and well-tested ideal functionalities and (realistic) protocols.

REFERENCES

- [1] Christian Badertscher, Ran Canetti, Julia Hesse, Björn Tackmann, and Vassilis Zikas. Universal composition with global subroutines: Capturing global setup within plain UC. In Rafael Pass and Krzysztof Pietrzak, editors, *TCC 2020, Part III*, volume 12552 of *LNCS*, pages 1–30. Springer, Cham, November 2020.
- [2] Christian Badertscher, Peter Gazi, Aggelos Kiayias, Alexander Russell, and Vassilis Zikas. Ouroboros chronos: Permissionless clock synchronization via proof-of-stake. *Cryptology ePrint Archive*, Report 2019/838, 2019.
- [3] Christian Badertscher, Ueli Maurer, Daniel Tschudi, and Vassilis Zikas. Bitcoin as a transaction ledger: A composable treatment. In Jonathan Katz and Hovav Shacham, editors, *CRYPTO 2017, Part I*, volume 10401 of *LNCS*, pages 324–356. Springer, Cham, August 2017.
- [4] Leemon Baird, Sanjam Garg, Abhishek Jain, Pratyay Mukherjee, Rohit Sinha, Mingyuan Wang, and Yinuo Zhang. Threshold signatures in the multiverse. In *2023 IEEE Symposium on Security and Privacy*, pages 1454–1470. IEEE Computer Society Press, May 2023.
- [5] Carsten Baum, Bernardo David, Rafael Dowsley, Jesper Buus Nielsen, and Sabine Oechsner. TARDIS: A foundation of time-lock puzzles in UC. In Anne Canteaut and François-Xavier Standaert, editors, *EUROCRYPT 2021, Part III*, volume 12698 of *LNCS*, pages 429–459. Springer, Cham, October 2021.
- [6] Mihir Bellare and Phillip Rogaway. The security of triple encryption and a framework for code-based game-playing proofs. In Serge Vaudenay, editor, *EUROCRYPT 2006*, volume 4004 of *LNCS*, pages 409–426. Springer, Berlin, Heidelberg, May / June 2006.
- [7] M. Bishnoi, R. Bhatia, A. Jain, and B. Singh. Hash time locked contract based asset exchange solution for probabilistic public blockchains. *Cluster Computing*, 25(6):4189–4201, 2022. Published 29 June 2022.
- [8] Alexandra Boldyreva. Threshold signatures, multisignatures and blind signatures based on the gap-Diffie-Hellman-group signature scheme. In Yvo Desmedt, editor, *PKC 2003*, volume 2567 of *LNCS*, pages 31–46. Springer, Berlin, Heidelberg, January 2003.
- [9] Benedikt Bünz, Lucianna Kiffer, Loi Luu, and Mahdi Zamani. FlyClient: Super-light clients for cryptocurrencies. In *2020 IEEE Symposium on Security and Privacy*, pages 928–946. IEEE Computer Society Press, May 2020.
- [10] Jan Camenisch, Manu Drijvers, and Björn Tackmann. Multi-protocol UC and its use for building modular and efficient protocols. *Cryptology ePrint Archive*, Report 2019/065, 2019.
- [11] Jan Camenisch, Robert R. Enderlein, Stephan Krenn, Ralf Küsters, and Daniel Rausch. Universal composition with responsive environments. In Jung Hee Cheon and Tsuyoshi Takagi, editors, *ASIACRYPT 2016, Part II*, volume 10032 of *LNCS*, pages 807–840. Springer, Berlin, Heidelberg, December 2016.
- [12] Jan Camenisch, Stephan Krenn, Ralf Küsters, and Daniel Rausch. iUC: Flexible universal composable security made simple. In Steven D. Galbraith and Shihō Moriai, editors, *ASIACRYPT 2019, Part III*, volume 11923 of *LNCS*, pages 191–221. Springer, Cham, December 2019.
- [13] Ran Canetti. Universally composable security: A new paradigm for cryptographic protocols. In *42nd FOCS*, pages 136–145. IEEE Computer Society Press, October 2001.
- [14] Ran Canetti. Universally composable security: A new paradigm for cryptographic protocols. *Cryptology ePrint Archive*, Paper 2000/067, 2020. <https://eprint.iacr.org/2000/067>.
- [15] Ran Canetti, Yevgeniy Dodis, Rafael Pass, and Shabsi Walfish. Universally composable security with global setup. In Salil P. Vadhan, editor, *TCC 2007*, volume 4392 of *LNCS*, pages 61–85. Springer, Berlin, Heidelberg, February 2007.
- [16] Ran Canetti and Tal Rabin. Universal composition with joint state. In Dan Boneh, editor, *CRYPTO 2003*, volume 2729 of *LNCS*, pages 265–281. Springer, Berlin, Heidelberg, August 2003.
- [17] Ran Canetti, Daniel Shahaf, and Margarita Vald. Universally composable authentication and key-exchange with global PKI. In Chen-Mou Cheng, Kai-Min Chung, Giuseppe Persiano, and Bo-Yin Yang, editors, *PKC 2016, Part II*, volume 9615 of *LNCS*, pages 265–296. Springer, Berlin, Heidelberg, March 2016.
- [18] Miguel Castro and Barbara Liskov. Practical byzantine fault tolerance. In *Proceedings of the Third Symposium on Operating Systems Design and Implementation (OSDI)*, pages 173–186. USENIX Association, 1999.
- [19] Chainflip Protocol Development Team. Chainflip protocol whitepaper. <https://assets.chainflip.io/whitepaper.pdf>, 2020.
- [20] Panagiotis Chatzigiannis, Foteini Baldimtsi, and Konstantinos Chalkias. SoK: Blockchain light clients. In Ittay Eyal and Juan A. Garay, editors, *FC 2022*, volume 13411 of *LNCS*, pages 615–641. Springer, Cham, May 2022.
- [21] Ana Lúcia de Moura and Roberto Ierusalimsky. Revisiting coroutines. *ACM Trans. Program. Lang. Syst.*, 31(2):6:1–6:31, 2009.

- [22] Matthias Fitzi, Peter Gazi, Aggelos Kiayias, and Alexander Russell. Parallel chains: Improving throughput and latency of blockchain protocols via parallel composition. *Cryptology ePrint Archive*, Report 2018/1119, 2018.
- [23] Juan A. Garay, Aggelos Kiayias, and Nikos Leonardos. The bitcoin backbone protocol: Analysis and applications. In Elisabeth Oswald and Marc Fischlin, editors, *EUROCRYPT 2015, Part II*, volume 9057 of *LNCS*, pages 281–310. Springer, Berlin, Heidelberg, April 2015.
- [24] Juan A. Garay, Aggelos Kiayias, and Hong-Sheng Zhou. A framework for the sound specification of cryptographic tasks. In Andrew Myers and Michael Backes, editors, *CSF 2010 Computer Security Foundations Symposium*, pages 277–289. IEEE Computer Society Press, 2010.
- [25] Sanjam Garg, Abhishek Jain, Pratyay Mukherjee, Rohit Sinha, Mingyuan Wang, and Yinuo Zhang. hintS: Threshold signatures with silent setup. In *2024 IEEE Symposium on Security and Privacy*, pages 3034–3052. IEEE Computer Society Press, May 2024.
- [26] Peter Gazi, Aggelos Kiayias, and Alexander Russell. Fait accompli committee selection: Improving the size-security tradeoff of stake-based committees. In Weizhi Meng, Christian Damsgaard Jensen, Cas Cremers, and Engin Kirda, editors, *ACM CCS 2023*, pages 845–858. ACM Press, November 2023.
- [27] Peter Gazi, Aggelos Kiayias, and Dionysis Zindros. Proof-of-stake sidechains. In *2019 IEEE Symposium on Security and Privacy, SP 2019, San Francisco, CA, USA, May 19-23, 2019*, pages 139–156. IEEE, 2019.
- [28] Peter Gazi, Aggelos Kiayias, and Dionysis Zindros. Proof-of-stake sidechains. In *2019 IEEE Symposium on Security and Privacy*, pages 139–156. IEEE Computer Society Press, May 2019.
- [29] Dennis Hofheinz, Dominique Unruh, and Jörn Müller-Quade. Polynomial runtime and composability. *Journal of Cryptology*, 26(3):375–441, July 2013.
- [30] Daniel Jackson. *Software Abstractions - Logic, Language, and Analysis*. MIT Press, 2006.
- [31] Dimitris Karakostas and Aggelos Kiayias. Securing proof-of-work ledgers via checkpointing. *Cryptology ePrint Archive*, Report 2020/173, 2020.
- [32] Jonathan Katz, Ueli Maurer, Björn Tackmann, and Vassilis Zikas. Universally composable synchronous computation. In Amit Sahai, editor, *TCC 2013*, volume 7785 of *LNCS*, pages 477–498. Springer, Berlin, Heidelberg, March 2013.
- [33] Igor Konnov. Apalache: The symbolic model checker for TLA+. <https://apalache-mc.org/>.
- [34] Igor Konnov, Jure Kukovec, and Thanh-Hai Tran. Tla+ model checking made symbolic. *Proceedings of the ACM on Programming Languages*, 3(OOPSLA):123:1–123:30, oct 2019.
- [35] Leslie Lamport. TLA—Temporal Logic of Actions. <http://lamport.org>. Web page.
- [36] Leslie Lamport. *Specifying Systems*. Addison-Wesley, Boston, 2003. Electronic copy available at <http://lamport.org>.
- [37] Alfred Menezes and Nigel Smart. Security of signature schemes in a multi-user setting. *Des. Codes Cryptography*, 33(3):261–274, November 2004.
- [38] Satoshi Nakamoto. Bitcoin: A peer-to-peer electronic cash system. <https://bitcoin.org/bitcoin.pdf>, 2008. Accessed July 7, 2025.
- [39] NEAR. Eth-near rainbow bridge. <https://www.near.org/blog/eth-near-rainbow-bridge> (accessed June 2025), 2025.
- [40] Rafael Pass, Lior Seeman, and abhi shelat. Analysis of the blockchain protocol in asynchronous networks. In Jean-Sébastien Coron and Jesper Buus Nielsen, editors, *EUROCRYPT 2017, Part II*, volume 10211 of *LNCS*, pages 643–673. Springer, Cham, April / May 2017.
- [41] Polygon Labs. Polygon pos architecture overview. <https://docs.polygon.technology/pos/architecture/overview/>, 2025.
- [42] Phillip Rogaway. Formalizing human ignorance. In Phong Q. Nguyen, editor, *Progress in Cryptology - VIETCRYPT 06*, volume 4341 of *LNCS*, pages 211–228. Springer, Berlin, Heidelberg, September 2006.
- [43] Merkle Science. Hack track: Analysis of ronin network exploit. <https://www.merklescience.com/blog/hack-track-analysis-of-ronin-network-exploit> (accessed July 2025).
- [44] Informal Systems. Quint: An executable specification language. <https://quint-lang.org/>.
- [45] Roberto Tamassia. Authenticated data structures. In Giuseppe Di Battista and Uri Zwick, editors, *Algorithms - ESA 2003, 11th Annual European Symposium, Budapest, Hungary, September 16-19, 2003, Proceedings*, volume 2832 of *Lecture Notes in Computer Science*, pages 2–5. Springer, 2003.
- [46] Tendermint Core Development Team. Tendermint core: Byzantine fault tolerant (bft) middleware. <https://github.com/tendermint/tendermint>, 2025. Accessed: 2025-06-13; commit hash corresponding to latest version at access.
- [47] The Investopedia Team. Hashed Timelock Contract (HTLC): Overview and Examples in Crypto. <https://www.investopedia.com/terms/h/hashed-timelock-contract.asp>, 2023.
- [48] THORChain Protocol Development Team. Thorchain whitepaper. <https://github.com/thorchain/Resourses/blob/master/Whitepapers/whitepaper-en.md>, 2020.
- [49] Apostolos Tzinas, Srivatsan Sridhar, and Dionysis Zindros. On-chain timestamps are accurate. In Jeremy Clark and Elaine Shi, editors, *FC 2024, Part I*, volume 14744 of *LNCS*, pages 110–127. Springer, Cham, March 2024.
- [50] Wormhole Foundation. Token bridge overview. <https://wormhole.com/docs/products/token-bridge/overview/>, 2025.
- [51] Tiancheng Xie, Jiaheng Zhang, Zerui Cheng, Fan Zhang, Yupeng Zhang, Yongzheng Jia, Dan Boneh, and Dawn Song. zkBridge: Trustless cross-chain bridges made practical. In Heng Yin, Angelos Stavrou, Cas Cremers, and Elaine Shi, editors, *ACM CCS 2022*, pages 3003–3017. ACM Press, November 2022.
- [52] Maofan Yin, Dahlia Malkhi, Michael K. Reiter, Guy Golan Gueta, and Ittay Abraham. Hotstuff: Bft consensus with linearity and responsiveness. In *Proceedings of the 38th ACM Symposium on Principles of Distributed Computing (PODC)*, pages 347–356. ACM, 2019.
- [53] Alexei Zamyatin, Mustafa Al-Bassam, Dionysis Zindros, Eleftherios Kokoris-Kogias, Pedro Moreno-Sanchez, Aggelos Kiayias, and William J. Knottenbelt. SoK: Communication across distributed ledgers. In Nikita Borisov and Claudia Díaz, editors, *FC 2021, Part II*, volume 12675 of *LNCS*, pages 3–36. Springer, Berlin, Heidelberg, March 2021.

APPENDIX A BRIDGE-LIKE CONSTRUCTIONS

The terms blockchain and cross-chain bridge are used inconsistently across the literature, often conflated with a broad class of cross-chain applications that rely on some form of cross-chain communication (CCC) [53].

We distinguish general-purpose bridges from such notions. In particular, we treat bridges as protocols that enable *general-purpose* state verification or transaction synchronization between independent ledgers, whereas many cross-chain applications such as wrapped assets or shared liquidity pools merely operate atop such mechanisms, or implement them in a restricted way.

HTLC. A typical example of a cross-chain application that we do not consider a cross-chain bridge is hashed timelock contracts (HTLC) or adaptor signature-based atomic asset swaps [47], [7]. Such swaps, for instance, do not guarantee liveness, as the counter party can simply ignore the lock. Thus the statement that holds at the end of the protocol execution is of the form: either one can get hold of an asset on the sending ledger, or one gets her own asset back on the receiving ledger (and vice versa). We consider support of such limited application specific statements to be too limited for a general purpose bridge.

Simple payment verification. In the context of cross-chain communication and lightweight verification, it is important to consider simplified payment verification (SPV) clients, particularly as used in Bitcoin. SPV clients verify transaction inclusion based only on block headers, without downloading full blocks; as such, they are not DO (Direct Observation) clients in the strict sense. While SPV is not itself a bridge mechanism, it can be used as a building block for bridge construction.

To adapt SPV-style verification into a DO-like bridge framework, several modifications are necessary: (1) the update function must verify that incoming headers extend the existing chain consistently; (2) it must handle chain reorganization or only retain headers buried at least k blocks deep for some k ; and (3) *Certify* and *Verify* must be specialized to transaction inclusion proofs. While the connection between SPV and DO is not immediately obvious, it becomes clearer when abstracting their behaviors from the perspective of the receiving contract: both systems process externally provided updates and perform lightweight validation. The key difference lies in the validation predicate `xcVal`—SPV assumes minimal checks (e.g., Proof-of-Work validity), whereas DO requires alignment with the full view of the sending chain. It remains an open question whether relaxing `xcVal` and other DO bridge components can yield a secure and meaningful SPV-style bridge in our formal model.

Light clients. Light clients play a foundational role in several bridge designs, especially in scenarios where the receiving ledger must validate consensus or transaction inclusion without full state replication. The SoK on blockchain light clients [20]

provides a taxonomy of client types, including SPV clients, which verify inclusion proofs using only block headers. Fly-Client [9] improves upon this model by allowing superlight clients to securely verify transactions with logarithmic communication. While SPV is not a bridge per se, it is a critical primitive in the sense that several bridge protocols (especially those compatible with Bitcoin) can be constructed using SPV-style verification, augmented with chain continuity checks and rollback-aware update functions [38].

Both bridge protocols and light clients benefit significantly from ledger finality properties, which reduce ambiguity around reorgs and improve the security and efficiency of cross-chain verification. Finality ensures that once a block is confirmed beyond a certain depth, it is unlikely (or impossible, in the case of strong finality) to be reverted—thus making proofs of inclusion or consensus status more reliable and simpler to verify.

APPENDIX B UC, PSEUDO-CODE, AND PROPERTIES

A. The UC framework

We formalize our security statements in the Universal Composability (UC) framework of Canetti [13], [14]. The UC framework provides strong security guarantees: protocols that are UC secure can be used securely and in a modular way in a wide variety of contexts. In a nutshell, UC compares the execution of a real protocol in the presence of an adversary to that of another ideal protocol in presence of another adversary (aka. a simulator). If for any adversary against the real protocol there exists an ideal adversary against the ideal protocol so that no interactive distinguisher (aka. the environment) can distinguish the two, the real protocol is said to UC-realize the ideal protocol.

This definition allows designing a higher-level protocol with respect to a simpler ideal protocol, and then replacing the ideal protocol with a real protocol that UC-realizes it. This is essentially due to the UC composition theorem, the proof of which relies on the fact that any higher-level protocol can be seen as being part of the environment. This replacement can be extended to multiple ideal protocols, and enables a modular treatment of security for complex protocols. It is thus particularly suitable for our goals.

While natural, defining a formal execution model that is sufficiently expressive to capture real protocols of interest turns out to be technical. For instance, one needs to delineate boundaries of protocol instances, appropriately model shared resources, consider corrupted parties, and constrain the environment so that the security is both feasible and meaningful.

We give a brief overview of the simpler UC framework for static systems. (Our protocols do not need to dynamically generate functionalities during runtime, and thus we do not need the full power of dynamic UC.) We refer to [14, Section 2] and [1, Section 2] for further details on UC for static systems (henceforth, called UC).

Interactive Turing machines. The UC framework models protocols as collections of *interactive Turing machines* (machines)

that interact through various tapes. An ITM resembles a Turing machine, having internal work tapes and a state transition function. However, an ITM (in UC) comes with three external tapes: input, subroutine-output and backdoor. The input tape holds inputs from calling machines and the subroutine-output tape contains return values for caller machines. The backdoor tape is used for communication with a special machine called the adversary (see below). To ease readability, we will simply use ‘machine’ instead of ITM.

Identities. UC machines possess unique identities of the form $(P, \text{sid} = (\text{pid}, i))$. P is a party identifier (PID) and is interpreted as a ‘local cluster’ of machines. pid is a ‘process identifier’ and is different for machines with different codes. For example, it can be set to the (hash of) code of the machine. i is an instance number and is used to indicate independent instances of the same process. We call $\text{sid} = (\text{pid}, i)$ the session-identifier (SID), which are unique across all machines active in an execution.

Machines specify the destination identities of other machines that they provide inputs or subroutine outputs to. This is done by including the identities on tapes. Machines also include their own identities on tapes so that the destination machine knows the source. Every machine comes with a *communication set* which defines which other machines and in what role it can talk to.

Protocols. A protocol π is a *set of well-connected* machines in the following sense: (1) a machine in π can pass inputs to another machine in π if and only if the latter can pass outputs back to the former; (2) a machine in π can pass inputs to some destination identity only if there is a machine in π with that identity. The *external identities* of a protocol are the set of identities outside π where some machine in π provides outputs to. The *main* machines of π are those that have at least one external identity. As sets, we use $\cup$ to combine protocols into larger protocols, or remove components using $\setminus$.

PID-wise connectivity. In UC one explicitly models network resources. Accordingly, we require that machines can only pass input to or receive outputs from other machines if they have the same PID. This means that the machines of a protocol communicate only ‘locally’ and use ideal functionalities to send network messages. We make exceptions to this rule when connecting to ideal functionalities, the environment, or the adversary (see below).

The environment and the adversary. The environment $\mathcal{Z}$ and the adversary $\mathcal{A}$ (which is called the *simulator* when run wrt. an ideal protocol to match attacks on a real protocol) are machines which have special PID and SID (e.g., 0 and 1 as their PIDs respectively, and $\perp$ for their SIDs). The environment models honest parties in higher-level protocols, including protocols in other sessions; the adversary, on the other hand, models adversarial influence on the protocol, such as generation of cryptographic objects in ideal functionalities or actions of the corrupted parties (see below). The environment and the adversary can freely interact. The adversary can talk to all machines on the backdoor tape. Below we define how $\mathcal{Z}$ interacts with machines in a given protocol.

PID-wise corruptions. The environment may instruct the adversary to *corrupt* a machine with PID P different from those of the adversary, the environment, and any ideal functionality. Consequently, the contents of the internal work tapes of all machines with PID P are sent to the adversary. Moreover, these machines then act as dummies: they forward all their incoming communication to $\mathcal{A}$, and receive instructions from $\mathcal{A}$ as to how to respond. This models PID-wise full Byzantine corruptions. The fact that the environment initiates corruptions ensures that in the real and ideal worlds corruptions are ‘commensurate’. Throughout, we let $\mathbf{C}$ denote an execution-wide set of corrupt parties in the system.

Establishing security in the fully Byzantine and adaptive model of corruptions is oftentimes challenging. One often resorts to weaker models, e.g., of *static corruptions* whereby the environment instructs the adversary to corrupt machines of its choice at the onset before any other interaction. Another weaker notion of relevance is that of δ -delayed corruptions [28] for some delay parameter δ . Here a machine that is instructed to be corrupted by the adversary becomes corrupt (its internal state is leaked and its control is handed over to the adversary) only after δ rounds.

Ideal functionalities. An *ideal functionality* models a ‘trusted service’ that collects inputs and carries out a specific task in a trusted manner. Given an ideal functionality $\mathcal{F}$, the ideal protocol is obtained by attaching a number of dummy parties to $\mathcal{F}$ who act as forwarders. This gives the higher-level protocols (or environment) the impression of interacting with a distributed protocol (rather than a single centralized service). By slight abuse of notation, we will use $\mathcal{F}$ for the ideal protocol for $\mathcal{F}$. Ideal functionalities are *incorruptible*, though their guarantees can deteriorate depending on the corruption status of their dummy parties. Formally, we assume the PIDs of ideal functionalities take special forms, which the environment is prohibited from corrupting. Note, however, that the adversary can still interact with ideal functionalities both through the backdoor tape and on behalf of the corrupted dummy parties.

Note that through ideal functionalities, protocols get access to *joint state*. ‘Local’ functionalities typically have caller-session-specific states, while ‘global’ (or shared) functionalities can share state across all caller sessions. (No state is shared across the sessions of $\mathcal{F}$, be it local or global.)

Remark. When specifying ideal functionalities, a core conceptual insight is that the simulator can influence the generation of cryptographic objects whose specific forms do not affect the security guarantees. For example, in a secure signature scheme, it does not matter what particular form the verification keys take, as long as no new verifying signature beyond those legitimately obtained through the ideal functionality, can be generated. This power, in turn, allows ideal functionalities to be realizable by concrete protocols. In UC, one typically abstracts away from secret keys (though on corruption they need to be simulated).

Environmental restrictions. The environment can send messages to machines in a protocol π *on behalf of other machines* as

well as to the adversary. This is required so that $\mathcal{Z}$ can indeed model higher-level protocols accessing π (and thus enable composition). The environment, however, can only pass inputs to the *main machines* of a protocol π on behalf of the *honest external identities*. (PID-wise connectivity is also required to be respected.) In this case we say that the communication set of $\mathcal{Z}$ is compatible with π .

Executions. The execution of a protocol π with an adversary $\mathcal{A}$ and an environment $\mathcal{Z}$ starts by running $\mathcal{Z}$, which then sends messages to other machines as specified above. The execution terminates returning a bit, which we denote by $\mathcal{Z}^{\pi, \mathcal{A}}$. (This notation follows [6] and is more intuitive and arguably closer to diagrammatic descriptions. In UC lingo this is written as $\text{EXEC}_{\pi, \mathcal{A}, \mathcal{Z}}$.) In this execution, we assume that a concrete value for the security parameter is hard-coded to all machines.

Efficiency. We assume that environments, adversaries, and protocols satisfy an appropriate notion of efficiency so that the entire execution can be efficiently emulated on a single machine. We give a concrete example. If each machine in the execution terminates—that is, halts and no longer sends any messages—after at most t steps over *all* its invocations, at least a single step is needed to send a message to another machine, and there are no chains of invocations of length more than c (any chain of executions of length at least $c + 1$ has a cycle), the maximum number of steps can be shown to be that corresponding to a depth- c tree of processes each invoking t processes. The total number of steps is t^c , which is polynomial (in the security parameter) as long as t is polynomial and c is a constant. As a special case, if the total number of machines in a protocol is m , execution is clearly upper bounded by $t \cdot m$. Other notions of efficiency exist [12], [14], [29].

B. UC emulation and composition

We restate UC emulation and composition theorems, despite not directly employing them in our development. We do, however, rely on global subroutines when stating our property-based definitions.

UC emulation. We say a protocol π UC-emulates another (typically more idealized) protocol ϕ if for any efficient adversary $\mathcal{A}$ there exists an efficient simulator $\mathcal{S}$ such that for all environments $\mathcal{Z}$ that are compatible with both π and ϕ , the distance $|\Pr[\mathcal{Z}^{\pi, \mathcal{A}} = 1] - \Pr[\mathcal{Z}^{\phi, \mathcal{S}} = 1]|$ is small.

Let ρ, ϕ , and π be protocols. Recall:

- ϕ is a *subroutine protocol* of ρ if $\phi \subseteq \rho$ as a set of ITMs.
- π is *compatible* with ϕ if there is an identity-preserving bijective correspondence between main machines of π and ϕ , and the entries corresponding to external identities in the communication sets of the main machines are equal.
- If ϕ is a subroutine protocol of ρ , then π is *identity-compatible* with ρ and ϕ if no machine in π has the same identity as a machine in $\rho \setminus \phi$.

We let $\rho^{\phi \rightarrow \pi} := (\rho \setminus \phi) \cup \pi$, i.e., a protocol that is similar to ρ except that protocol ϕ is replaced by protocol π . Under the above conditions one can show that $\rho^{\phi \rightarrow \pi}$ is indeed a protocol. Canetti [14] presents a composition theorem as follows.

Theorem 2 (Essentially [14, Theorem 3]). *Assume that π, ϕ , and ρ are protocols such that $\phi \subseteq \rho$, π is compatible with ϕ , π is identity-compatible with ρ and ϕ , and π UC-emulates ϕ . Then $\rho^{\phi \rightarrow \pi}$ UC-emulates ρ .*

Roughly speaking, the proof relies on the fact that the caller part of ρ , namely $\rho \setminus \pi$, can be absorbed into the environment, so that when this caller part is composed with π or ϕ , the results are indistinguishable as π UC-emulates ϕ . The fact that ρ and $\rho^{\phi \rightarrow \pi}$ are protocols ensures that any environment against that is compatible with ρ and $\rho^{\phi \rightarrow \pi}$ when converted to one against π or ρ with ρ absorbed, is also compatible with π and ρ .

C. Global functionalities

When designing systems, it is sometimes the case that certain components of the system should be modeled as global (or shared) functionalities. For instance, in our setting, the blockchain is typically a shared infrastructure that is used by different protocols. That is, one typically does *not* consider independent instances of blockchain for different applications. Examples of other shared functionalities include public-key infrastructure, or shared clocks that enable synchrony across different protocols. Thus, unlike local functionalities, global functionalities often have a single instance that may take input from, and provide outputs to multiple instances of (unknown) protocols. As such, the machines in a global functionality may be part of the main machines of a protocol, and thus the environment can directly call them. Thus, such functionalities can be in any state when they first get accessed by the protocol.

Complex frameworks exist to augment the UC framework to capture joint state or globals [15], [16]. The most recent is the UCGS framework of Badertscher et al. [1] that builds on the latest version of (dynamic) UC to model global functionalities as shared subroutines within the (plain) UC framework. This work, in particular, provides a simple and convincing treatment for the static fragment of UC [1, Section 2] that avoids complex frameworks. We give an overview next.

Given a protocol ρ , the UCGS framework (for static systems) treats its global sub-protocol γ together with other subroutines π as a sub-protocol $\hat{\pi} := \pi \cup \gamma$. To only replace π with ϕ , leaving γ untouched, one considers $\hat{\phi} := \phi \cup \gamma$ and invokes Theorem 2 with respect to $\hat{\pi}$, $\hat{\phi}$, and ρ . Note that the main machines of γ may form a subset of the main machines of $\pi \cup \gamma$, which allows γ to be accessed from higher-level protocols.

Theorem 3 (Composition with global subroutines). *Assume that π, ϕ, γ , and ρ are protocols. Suppose that $\phi, \gamma \subseteq \rho$, π is compatible with ϕ , π is identity-compatible with ρ and ϕ , and $(\pi \cup \gamma)$ UC-emulates $(\phi \cup \gamma)$. Then $\rho^{\phi \rightarrow \pi}$ UC-emulates ρ .*

Let $\hat{\pi} := \pi \cup \gamma$ and $\hat{\phi} := \phi \cup \gamma$. The proof uses the following observations: 1) if π and ϕ are protocols, then so are $\hat{\pi}$ and $\hat{\phi}$; 2) we have $\hat{\phi} \subseteq \rho$; 3) if π is compatible with ϕ , then $\hat{\pi}$ is compatible with $\hat{\phi}$, and 4) if π is identity-compatible with ρ and ϕ then $\hat{\pi}$ is identity-compatible with ρ and $\hat{\phi}$. This allows Theorem 2 to be applied to $\hat{\phi}$, $\hat{\pi}$, and ρ . We can conclude using the fact that $\rho^{\phi \rightarrow \hat{\pi}} = \rho^{\phi \rightarrow \pi}$.

D. Pseudo-code conventions

The language. The language of Turing machines is too low level for practical purposes. English specifications, on the other hand, can be ambiguous and error prone. We follow the code-based framework of [6, Appendix B] and use a programming language to express functionalities and protocols. This simple language supports basic data types (integers, sets, lists, maps, etc.) and common operations on them. We use $\perp$ for the option type indicating “none,” and $\square$ to indicate that some entry in a key-value store is “not set.” We use `ok` for the unit type and `_` for unused values. Reserved words with special meanings used for program structure and syntax (**if**, **then**, **for**, **return**, etc.) are typeset in bold. We do not use `while`, `loops`, or `goto` statements. To help readability, we use various syntactic sugar such as indentation, use of line break instead of “;”, subscripts and superscripts, Greek / Unicode characters, quantifiers over finite domains, and iteration “**for** $x \in S$ ” where x takes values in S in some natural order.

A major difference with conventional (stand-alone) games is that, besides subroutine calls, machines can *pass the execution* to another machine. This can be modeled via the language of *coroutines* [21]. Their use in this work is almost entirely confined to interactions with the simulator; calls to and between functionalities are expressed using subroutine calls/returns.

Interfaces and their connectivity. We express functionalities and protocols as collections of interfaces `P.Interface` for different parties P and session-ids `sid`, as well as special $\mathcal{Z}$ - and `Sim`-specific interfaces. An interface is a sequence of statements (program) in the programming language and corresponds to machines in UC. Parties P are required to always belong to a set of parties $\mathbf{P}$, which is a parameter of the functionality. For PID-wise connectivity, we require that the PID of the caller machine (which is another interface) always matches P , i.e., that of the called interface.

Session-ids are required to belong to a session-id set $\mathbf{SID}$, which is another functionality parameter. We will mainly consider $\mathbf{SID}$ sets of two types. First, for local functionalities, which are designed to respond to a single caller, this is a singleton set $\mathbf{SID} = \{\text{csid}\}$. Second, for global functionalities, which typically offer services to any caller process, $\mathbf{SID}$ is set to the entire session-id space.

For locals, these restrictions amount to `P.Interface` checking if the caller’s (P, sid) matches (P, csid) . For globals, only the PID is checked as there are no restrictions on $\mathbf{SID}$ s. (We will indicate this by an “any `sid`” annotation.)

Functionalities also have a session-id `sid*`, which enables independent instances to be created. (In other words, program variables in different instances are indexed by `sid*`.) Interfaces inherit this session-id and thus `P.Interface`’s identity is (P, sid^*) . This is used when calling other interfaces.

Corruptions. We model PID-wise corruptions via an environment-only `Z.Corrupt(P)` interface that adds P to an execution-wide set of corrupted parties $\mathbf{C}$. The interface notifies the adversary of corruption by providing it with the entire input-output history of all machines (interfaces) whose PID is P . We

provide the code below. Here `Leakage[P]` is a leakage list that is populated within the functionality as inputs and outputs are computed. For real protocols, this interface (being for $\mathcal{Z}$) is not realized; rather it is set to the contents of tapes, assuming possibly secure erasures.

`Z.Corrupt(P)`

```

1: if  $P \notin \mathbf{C}$  then
2:    $\mathbf{C} \leftarrow \mathbf{C} \cup \{P\}$ 
3:   Sim(P.Corrupt, Leakage[P])
4: return ok

```

We assume $\mathcal{Z}$ calls `Corrupt(P)` for all functionalities once it calls it for one. This is to ensure P -wise corruptions. Note that by the above structure, real-world leakage can be simulated towards the adversary by the simulator. For functionality with a party set $\mathbf{P}$, we let $\mathbf{H} := \mathbf{P} \setminus \mathbf{C}$ denote the honest parties.

When a party is corrupted it passes inputs to the adversary, and replaces outputs by those of the adversary, before they are returned. Below we present this corruption wrapper for `P.Interface` in a functionality $\mathcal{F}[\mathbf{P}, \text{sid}^*, \mathbf{SID}, \dots]$. Here caller is the caller’s identity. We have not assumed any checks on P , $\mathbf{P}$, $\mathbf{SID}$, etc. in the body of `P.Interface`; all checks have been written explicitly. Note that corruptions may arrive mid-execution. By removing the two **if** statements, we obtain the full interface code for honest parties. In our specification, however, we will only describe the main body, `P.Interface`.

`P.Full-Interface(in)`

```

5: if  $P \in \mathbf{C}$  then
6:   in  $\leftarrow \text{Sim}(\text{P.Interface-In}, \text{in})$ 
7: check  $P \in \mathbf{P}$ 
8: check caller.P =  $P \wedge \text{caller.sid} \in \mathbf{SID}$ 
9: out  $\leftarrow \text{P.Interface}(\text{in})$ 
10: if  $P \in \mathbf{C}$  then
11:   out  $\leftarrow \text{Sim}(\text{P.Interface-Out}, \text{out})$ 
12: return out

```

Interface access by $\mathcal{A}$ and $\mathcal{Z}$. The adversary, beside its own identity, can assume/spoof any identity whose PID component is in $\mathbf{C}$. The environment, on the other hand, beside its own identity, can assume any identity that is external in both the real and ideal worlds and whose PID component is not in $\mathbf{C}$.

Syntactic simplifications. For local functionalities, any caller’s $\mathbf{SID}$ must match `csid`. Moreover, different `csid`’s will result in different `sid*`, as the latter includes the pid of the functionality, which must be different for different `csid`’s. Thus different callers essentially talk to different state-separated instances. Therefore we will omit the caller `sid` and `csid`. We will make `sid*` explicit when this is used in calling other functionalities. Note that, since the interface’s and caller’s PID are required to match, the caller’s PID can be read from the interface’s name.

For global functionalities we will need to make explicit the caller’s identity as state is shared across different sessions. We will use `Psid.Interface` to denote that the caller’s id is `Psid`. (There is no ambiguity in overloading P as the interface’s and caller’s PID match.) Note that this notation can be also used for local functionalities, with the understanding that the `sid`

matches `csid`. If two functionalities have different `csid` then their `sid*` must be different as the latter contains (a fingerprint of) the functionality code. We emphasize that the identity at P_{sid} .Interface is (P, sid^*) , not (P, sid) .

Initialization. We specify global variables used in an ideal functionality under an (internal) initialization procedure `Init()`. This is executed once at the onset as soon as any interface is called.

For real protocols, upon first invocation, each party may need to perform basic actions such as registration with the clock. We model this under a party-specific initialization interface `P.Init()` which is executed once as soon as any interface for P is called. This choice separates state, and avoids including `P.Init()` as a preamble in every P -interface. We will also use this interface to declare party-wise global variables.⁴

Sanitization. We will often need to *sanitize* the outputs of simulators to ensure that attacks do not have an overly adverse effect on ideal functionalities. For instance, in honest-majority settings, we will typically sanitize a $\perp$ output by the simulator to a non- $\perp$ output so that liveness is preserved. We denote such a sanitization procedure by `San[Clean](v)` where `Clean(v)` is a predicate. We will then replace simulator output values v by a new $v' \leftarrow \text{San}[\text{Clean}](v)$, which unless stated otherwise, operates as follows.

```

San[Clean](v)
1: if Clean(v) then return v
2: v' ← {v' : Clean(v')}
3: return v'
```

In other words, the sanitize procedure `San[Clean](v)` is run when `Clean(v)` is not met. The result is a new value v' that is guaranteed to satisfy `Clean(v')`.⁵ Our sanitizers are written so that they only need *read-only* access to the functionality state. The specific way in which the sampling operation in line 2 is implemented is not critical; all that matters is that `Clean(v)` is met after sanitization so that the functionality when viewed as a hybrid is useful. This operation can be set to random sampling or taking the lexicographically first good value for v .

Real vs. ideal specifications. Ideal functionalities and real protocols differ in a number of aspects. First, ideal functionalities are aware of the corruption status of parties in their party sets. Second, they can pass the execution to the simulator and may offer simulator-specific interfaces (see, e.g., $\mathcal{F}$ -Diffuse). Finally, they may share state across parties.

Well-specified real protocols do not have any of these features (beyond those that are implicit in their underlying idealized building blocks). Moreover, real realization of ideal

functionalities do not specify how leakage is computed upon corruptions; rather this is set to contents of work tapes.⁶

In Appendix C we give several examples of idea functionalities.

E. Properties in UC

Views. We define the environment's view of the protocol view in an execution as follows. When an interface `Psid.Interface(x)` is called by the environment, an entry $(\text{handle}, P_{\text{sid}}.\text{Interface}, x, \square)$ is added to `view`, where `handle` is a unique handle/identifier for the call (needed as there could be multiple calls to the same interface with the same inputs). When this interface returns an output y to the environment entry (handle, y) is added to `view`.

To formalize some of our security properties, we need to talk about views in various rounds. This is a concept that only makes sense in the presence of a clock functionality $\mathcal{G}\text{-Clock}$. We formalize this by requiring that the environment calls the `Psid.Round()` interface of $\mathcal{G}\text{-Clock}$ immediately before each call it makes to other interfaces and immediately after it receives an output. The clock functionality is designed so that the `Psid.Round()` interface returns the current value of the clock's internal counter. Thus, inspecting this counter is an alternative way to define in which round a call was made or an output was received from. We note, however, that this is formally not a trace property.

Properties. An execution *property* is a predicate on the trace, `view`. Security requires specific properties hold for any environment/adversary. This is done via a special environment-specific *finalization interface* that outputs a bit based on the execution trace. We write these interfaces in small-caps as `SECPROP(..)`. Finalization interfaces can be called once by the environment at the end of the execution.

We express each `SECPROP` in code, with the interpretation that they are expected to hold for secure constructions. We thus adopt the convention that the return statement of a finalization procedure *negates* the target property. For instance, collision resistance is written as “**check** $x \neq x'$; **return** $\neg(\text{Hash}(x) \neq \text{Hash}(x'))$ ”. Here “**check** C ” is sugar for “**if** $\neg C$ **then return** 0”.

Advantage metrics. Property-based definitions operate with the dummy adversary. We thus omit the adversary and use environment and adversary interchangeably. For a protocol π and a finalization procedure `SECPROP` we denote the advantage of an adversary $\mathcal{Z}$ as

$$\text{Adv}_{\pi}^{\text{SECPROP}}(\mathcal{Z}) := |\Pr[\mathcal{Z}^{\pi} = 1] - t|.$$

where $t \in [0, 1]$. The probability above is taken over the entire randomness of the execution. Security requires that this

⁴Arguably, local and global variables can be modeled in a more disciplined way through storage functionalities. We, however, avoid this so that our programs have a more familiar form.

⁵We assume the underlying space for v are infinite of which $\neg \text{Clean}$ only rules finitely many out.

⁶Once again, a more disciplined approach would model storage as an ideal functionality, which may offer secure erasures. We avoid this extra layer of complexity to ease notation.

advantage is small for any efficient $\mathcal{Z}$.⁷ In our definitions, we always use $t = 0$ and thus drop the absolute sign.

There are two noteworthy benefits of this approach compared to standard game-based formulation. First, idealized building blocks that underlie a security game can be replaced by other less idealized building blocks that UC-realize them. This is a direct consequence of the UC composition theorem defined below. Second, property-based UC allows one to make explicit various assumptions that are tacitly in place in traditional game-based modeling, such as the underlying network assumption, setup assumptions such as a PKI, or access to shared software modules such as synchronous clocks.

We note that the work of [24] proposes a framework that, given properties expressed as admissible interactions between the environment, functionality, and adversary, derives a canonical ideal functionality satisfying these properties. That is, it allows formulating a property-based *composable* notion of security.

APPENDIX C SOME IDEAL FUNCTIONALITIES

As is well understood by experts working on UC, very few interesting cryptographic tasks can be achieved without any setup assumptions, modeled as an ideal functionality. For instance, NIZK requires a common reference string, and various consensus, key exchange, or multi-party computation goals require a PKI or authenticated channels; even information-theoretically secure protocols commonly assume secure channels or even secure broadcast. In that sense, as cryptographers we are living in some idealized world. By making the idealized setup functionalities explicit, we can aim to live in an ideal world that most closely resembles the real world of practitioners. Below we list idealized building blocks that we use in this work.

Global clocks. Formalizing synchronous, Δ -delayed networks requires a mechanism that allows parties to proceed synchronously in a round-by-round manner. Following standard approaches [2], [5], [32], we enable this via a *global clock functionality*, $\mathcal{G}\text{-Clock}$. This functionality initializes a global counter r (across all parties and sessions) to zero. It increments the counter once all registered honest parties express their willingness to proceed to the next round. This is done via the $P_{\text{sid}}.\text{OK}()$ interface that records when a party in a session is done for a round. (Philosophically, the clock reverses the causal advancement of time: instead of an independent clock that is always ticking, the clock only ticks when all parties have performed all their tasks for the current tick.) The $P_{\text{sid}}.\text{Round}()$ interface provides awareness of the current round.

In order to operate in a synchronous manner, parties in different sessions and processes need to register with the $\mathcal{G}\text{-Clock}$ at the onset. (Otherwise some parties might lag behind the execution.)

⁷The security parameter is fixed in $\mathcal{Z}$. For asymptotic security one can make it explicit and require the probability to be negligible. Moreover, one can adopt an appropriate notion of efficiency, as discussed in Section B-A.

We will sometimes annotate an interface with “once per round” to indicate that it is only executed once per round. This can be written formally via setting of per-party and round-specific flags. Note that different interfaces of the same functionality may need to be run in a single round. It is up to protocols and functionalities to keep track of all tasks performed (and OK the clock).

Functionality 5: $\mathcal{G}\text{-Clock}[\mathbf{P}]$

```

Init()
1:  $r \leftarrow 0$ ;  $\mathbf{Reg} \leftarrow \emptyset$ ;  $\text{Done}[*] \leftarrow 0$ 
 $P_{\text{sid}}.\text{Reg}()$  // any sid
2:  $\mathbf{Reg} \leftarrow \mathbf{Reg} \cup \{P_{\text{sid}}\}$ 
3: if  $P_{\text{sid}} \notin \mathbf{Reg}$  then
4:    $\text{Done}[P_{\text{sid}}] \leftarrow 0$ 
5: return ok
 $P_{\text{sid}}.\text{Round}()$  // any sid
6: return  $r$ 
Sim.Round()
7: return  $r$ 
 $P_{\text{sid}}.\text{OK}()$  // any sid
8: if  $P_{\text{sid}} \notin \mathbf{Reg}$  then return  $\perp$ 
9:  $\text{Done}[P_{\text{sid}}] \leftarrow 1$ 
10:  $\text{flag} \leftarrow 1$ 
11: for  $P'_{\text{sid}} \in \mathbf{Reg} \setminus \mathbf{C}$  do
12:   if  $\text{Done}[P'_{\text{sid}}] = 0$  then
13:      $\text{flag} \leftarrow 0$ 
14: if  $\text{flag} = 1$  then
15:    $r \leftarrow r + 1$ 
16:   for  $P'_{\text{sid}} \in \mathbf{Reg} \setminus \mathbf{C}$  do
17:      $\text{Done}[P'_{\text{sid}}] \leftarrow 0$ 
18: Sim( $P_{\text{sid}}.\text{OK}()$ )
19: return ok

```

Diffusion network. We formulate a network diffusion functionality $\mathcal{F}\text{-Diffuse}$ allowing a party to disperse messages among a set of parties with some maximal delay Δ (which is a parameter of the functionality) [3], [22]. When a message m is diffused, the current round is recorded, and the message is added to the “inbox” of all parties. When a party fetches messages from the diffusion network, all messages that were sent at least Δ rounds ago together with some more recent messages picked by the simulator are delivered to the party. The corresponding entries are removed from the party’s inbox. During this process the adversary can inject messages of its choice into the inbox of any party of its choice. Thus diffuse is a rather weak network functionality with no authentication, confidentiality, or agreement guarantees (e.g., it does not guarantee that the same set of messages are delivered to all parties). $\mathcal{F}\text{-Diffuse}$ is shown in Functionality 6.

Common reference string. $\mathcal{F}\text{-CRS}$ (Functionality 7) grants parties in $\mathbf{P}$ access to a common reference string generated using a string generation algorithm Setup . This functionality will be used to generate and agree on a set of public parameters,

Functionality 6: $\mathcal{F}$ -Diffuse $[\mathbf{P}, \text{sid}^*, \Delta]$ Building Blocks:1: $\mathcal{G}\text{-Clock}[\mathbf{P}']$ with $\mathbf{P} \subseteq \mathbf{P}'$ Init()2: $\text{ctr} \leftarrow 0; \mathbf{M}[\cdot, \cdot] \leftarrow \square$ P.Init()3: $\mathcal{G}\text{-Clock.P}_{\text{sid}^*}.\text{Reg}()$ P.Diffuse(m) once per round4: $r \leftarrow \mathcal{G}\text{-Clock.P}_{\text{sid}^*}.\text{Round}()$ 5: $\text{ctr} \leftarrow \text{ctr} + 1$ 6: **for** $\mathbf{P} \in \mathbf{P}$ **do**7: $\mathbf{M}[\mathbf{P}, \text{ctr}] \leftarrow (m, t)$ 8: $\text{Sim}(\mathbf{P}.\text{Diffuse}, m)$ 9: $\mathcal{G}\text{-Clock.P}_{\text{sid}^*}.\text{OK}()$ 10: **return** okSim.Diffuse(P, m)11: $r \leftarrow \mathcal{G}\text{-Clock.Sim.Round}()$ 12: $\text{ctr} \leftarrow \text{ctr} + 1$ 13: $\mathbf{M}[\mathbf{P}, \text{ctr}] \leftarrow (m, t)$ 14: **return** okP.Fetch()15: $r \leftarrow \mathcal{G}\text{-Clock.P}_{\text{sid}^*}.\text{Round}()$ 16: $i_{\text{late}} \leftarrow \{i \in [\text{ctr}] : \mathbf{M}[\mathbf{P}, i] \neq \square \wedge r - \mathbf{M}[\mathbf{P}, i].r \geq \Delta\}$ 17: $i_{\text{early}} \leftarrow \text{Sim}(\mathbf{P}.\text{Fetch})$ 18: $\mathbf{m} \leftarrow [\mathbf{M}[\mathbf{P}, i].m : i \in i_{\text{late}} \cup i_{\text{early}}]$ 19: **for** $i \in i_{\text{late}} \cup i_{\text{early}}$ **do**20: $\mathbf{M}[\mathbf{P}, i] \leftarrow \square$ 21: **return** m**Functionality 8: $\mathcal{F}$ -Beacon $[\mathbf{P}, \text{sid}^*, \ell]$** Building Blocks:1: $\mathcal{G}\text{-Clock}[\mathbf{P}']$ with $\mathbf{P} \subseteq \mathbf{P}'$ Init()2: $\text{nonce}[\cdot] \leftarrow \square$ P.Get()3: $r \leftarrow \mathcal{G}\text{-Clock.P}_{\text{sid}^*}.\text{Round}()$ 4: **if** $\text{nonce}[r] = \square$ **then**5: $\text{nonce}[r] \leftarrow \{0, 1\}^\ell$ 6: $\text{Sim}(\mathbf{P}.\text{Get}, \text{nonce}[r])$ 7: **return** $\text{nonce}[r]$ **Functionality 9: $\mathcal{F}$ -PKI $[\mathbf{P}, \text{sid}^*]$** Init()1: $\text{Ver}[\cdot, \cdot, \cdot] \leftarrow \square; \text{Reg}[\cdot] \leftarrow \square$ P.Reg(vk)2: **if** $\text{Reg}[\mathbf{P}] \neq \square$ **then return** $\perp$ 3: $\text{cert} \leftarrow \text{Sim}(\mathbf{P}.\text{Reg}, \text{vk})$ 4: $\text{cert} \leftarrow \text{San}[\text{Clean}](\text{cert})$ 5: $\text{Reg}[\mathbf{P}] \leftarrow \text{vk}$ 6: $\text{Ver}[\mathbf{P}, \text{vk}, \text{cert}] \leftarrow 1$ 7: **return** certP.Verify(Q, vk, cert)8: **return** $(\text{Ver}[\mathbf{Q}, \text{vk}, \text{cert}] = 1)$ P.Get()9: $\text{Sim}(\mathbf{P}.\text{Get})$ 10: **return** RegPred. Clean(cert)11: **return** $(\text{cert} \neq \perp)$

e.g., those in ATS or NIZKs. We assume the security parameter is hard-coded into Setup.

Functionality 7: $\mathcal{F}$ -CRS $[\mathbf{P}, \text{Setup}]$ Init()1: $\text{crs} \leftarrow \text{Setup}[\lambda]$ P.Get()2: $\text{Sim}(\mathbf{P}.\text{Get}, \text{crs})$ 3: **return** crs

Randomness beacon. $\mathcal{F}$ -Beacon (Functionality 8) provides common fresh randomness of length ℓ in each round.

Public-key infrastructure. $\mathcal{F}$ -PKI (Functionality 9) enables parties to register their public keys and obtain a certificate, which can be verified by any party. Specifically, Reg allows parties to register a key vk of their choice. As a result the party obtains a certificate cert. Verify allows any party to check if a certificate cert for a vk and party Q is valid. Get allows parties to obtain the keys of all registered parties. Our protocols use different sessions of $\mathcal{F}$ -PKI and we only define a local $\mathcal{F}$ -PKI. In particular, we avoid intricacies in using global PKIs [17].

Digital signatures. $\mathcal{F}$ -Sig (Functionality 10), which includes line G but excludes B, idealizes digital signatures. Gen allows parties to generate a key. The verification key is picked by

the simulator and sanitized so that it is different from existing vk's, including those for which a *valid* entry in Ver exists (i.e., “pre-forgeries” are avoided). Sign generates a signature on a given message. The signature is sanitized so that it is always available (due to locality) and has not been previously marked as invalid. Verify verifies signatures under any verification key. We do not assume that verification keys are authenticated; hence, a vk must be provided. This interface guarantees that no new signature under an honestly generated key verifies (unless the party has been corrupted). Therefore, signatures are strongly unforgeable.

The simulators for non-interactive, stateless signatures are often *responsive* [11] in that each simulator call is implemented by a separate algorithm that immediately returns (i.e., simulator calls are subroutine rather than coroutine calls).

Non-interactive zero-knowledge. Functionality 11 describes $\mathcal{F}$ -NIZK, modeling an idealized non-interactive zero-knowledge (NIZK) protocol. The relation $\mathcal{R}$ is fixed as a protocol parameter.

NIZK with external relations. Following Camenisch et al. [10], we introduce a NIZK functionality that *interacts* with other

Functionality 10: $\mathcal{F}\text{-Sig}[\mathbf{P}]$

```

Init()
1:  $\text{VK}[*] \leftarrow \square$ ;  $\text{Ver}[*,*,*] \leftarrow \square$ 
P.Gen()
2: if  $\text{VK}[\mathbf{P}] \neq \perp$  then return  $\text{VK}[\mathbf{P}]$ 
3:  $\text{vk} \leftarrow \text{Sim}(\text{P.Gen})$ 
4:  $\text{vk} \leftarrow \text{San}[\text{Clean}_{\text{vk}}](\text{vk}, \text{VK}, \text{Ver})$ 
5:  $\text{VK}[\mathbf{P}] \leftarrow \text{vk}$ 
6: return  $\text{vk}$ 
P.Sign(m)
7: check  $\text{VK}[\mathbf{P}] \neq \perp$ 
8:  $\text{vk} \leftarrow \text{VK}[\mathbf{P}]$ 
9:  $\sigma \leftarrow \text{Sim}(\text{P.Sign}, m)$ 
10:  $\sigma \leftarrow \text{San}[\text{Clean}_{\sigma}](m, \sigma, \text{Ver})$ 
11:  $\text{Ver}[\text{vk}, m, \sigma] \leftarrow 1$ 
12: return  $\sigma$ 
P.Verify(vk, m,  $\sigma$ )
13: G: if  $\text{Ver}[\text{vk}, m, \sigma] \neq \perp$  then return  $\text{Ver}[\text{vk}, m, \sigma]$ 
14:  $b \leftarrow \text{Sim}(\text{P.Verify}, \text{vk}, m, \sigma)$ 
15: if  $b \notin \{0, 1\}$  then  $b \leftarrow 0$ 
16: B: if  $\text{Ver}[\text{vk}, m, \sigma] \neq \perp$  then return  $\text{Ver}[\text{vk}, m, \sigma]$ 
17: if  $\exists \mathbf{P}' \neq \mathbf{C}. \text{vk} = \text{VK}[\mathbf{P}']$  then  $b \leftarrow 0$  // unique keys
18:  $\text{Ver}[\text{vk}, m, \sigma] \leftarrow b$ 
19: return  $b$ 
Pred. Cleanvk(vk, VK, Ver)
20: return  $(\text{vk} \neq \perp \wedge \text{vk} \notin \text{VK} \wedge \text{Ver}[\text{vk}, *, *] \neq 0)$ 
Pred. Clean $\sigma$ (m,  $\sigma$ , Ver)
21: return  $(\sigma \neq \perp \wedge \text{Ver}[\text{vk}, m, \sigma] \neq 0)$ 

```

Functionality 11: $\mathcal{F}\text{-NIZK}[\mathbf{P}, \mathcal{R}]$

```

Init()
1:  $\text{Ver}[*,*] \leftarrow \square$ 
P.Prove(x, w)
2: if  $\neg \mathcal{R}(x, w)$  then return  $\perp$ 
3:  $\pi \leftarrow \text{Sim}(\text{P.Prove}, x)$ 
4:  $\pi \leftarrow \text{San}[\text{Clean}](x, \pi, \text{Ver})$ 
5:  $\text{Ver}[x, \pi] \leftarrow 1$ 
6: return  $\pi$ 
P.Verify(x,  $\pi$ )
7: if  $\text{Ver}[x, \pi] \neq \square$  then return  $\text{Ver}[x, \pi]$ 
8:  $w \leftarrow \text{Sim}(\text{P.Verify}, x, \pi)$ 
9:  $\text{Ver}[x, \pi] \leftarrow \mathcal{R}(x, w)$ 
10: return  $\text{Ver}[x, \pi]$ 
Pred. Clean(x,  $\pi$ , Ver)
11: return  $(\pi \neq \perp \wedge \text{Ver}[x, \pi] \neq 0)$ 

```

functionalities. This is defined similarly to $\mathcal{F}\text{-NIZK}$, except that all calls to $\mathcal{R}$ in lines 2 and 9 are replaced by calls to a relation functionality $\mathcal{F}\text{-Rel}$, i.e., $\mathcal{F}\text{-Rel.P.Eval}(x, w)$. This technique enables us to construct proofs about the validity of relations which involve other functionalities, e.g., a relation that verifies a hash preimage when the hash is represented as an ideal functionality as opposed to a concrete implementation.

We give an example of $\mathcal{F}\text{-Rel}$ in Functionality 12 as employed for model checking.

Functionality 12: $\mathcal{F}\text{-Rel}[\mathbf{P}, \mathcal{R}]$

Building Blocks:

```

1:  $\mathcal{F}\text{-Hash}[\mathbf{P}] \parallel \mathcal{R}(x, w) := (x \in w)$ 
P.Eval((x, h), w)
2:  $h' \leftarrow \mathcal{F}\text{-Hash.P.Hash}(w)$ 
3: return  $(h' = h \wedge \mathcal{R}(x, w))$ 

```

Collision-resistant hash functions. $\mathcal{F}\text{-Hash}$ (Functionality 13) models collision-resistant hashing. The choice of outputs is left to the simulator, provided they are unique across all inputs.

Functionality 13: $\mathcal{F}\text{-Hash}[\mathbf{P}]$

```

Init()
1:  $\text{H}[*] \leftarrow \square$ 
P.Hash(x)
2:  $h \leftarrow \text{Sim}(\text{P.Hash}, x)$ 
3: if  $\text{H}[x] \neq \square$  then return  $\text{H}[x]$ 
4:  $h \leftarrow \text{San}[\text{Clean}](h, \text{H})$ 
5:  $\text{H}[x] \leftarrow h$ 
6: return  $h$ 
Pred. Clean(h, H)
7: return  $\neg(\exists x. \text{H}[x] = h)$ 

```

APPENDIX D GAME-BASED DEFINITIONS

(Multi-user) digital signatures. The concept of *multi-user digital signatures* was formalized in [37] to address the problem of security loss in the multi-user setting. More formally, a digital signature scheme $\text{DS} := (\text{Setup}, \text{Gen}, \text{Sig}, \text{Ver})$ consists of the following four algorithms.

$\text{Setup}(\lambda) \rightarrow \text{pp}$: The setup algorithm. It takes a security parameter λ and generates system parameters pp . We assume λ is hardwired into all algorithms.

$\text{Gen}(\text{pp}) \rightarrow (\text{sk}, \text{vk})$: The key generation algorithm. It takes the system parameters pp and generates a key pair (sk, vk) .

$\text{Sig}(\text{sk}, m) \rightarrow \sigma$: The signing algorithm. It takes a secret key sk and a message m and outputs a signature σ .

$\text{Ver}(\text{vk}, m, \sigma) \rightarrow b$: The verification algorithm. A deterministic algorithm that takes a verification key vk , a message m , and a signature σ , and outputs a bit b .

Signature correctness and (multi-user) security. We define correctness and n -user unforgeability through the (standard) game Game 2.⁸ In this game, the adversary can request public parameters, generate keys for parties, and corrupt parties to

⁸A noteworthy difference between property-based UC and Bellare–Rogaway style games is the use of $\text{Gen}(i)$ instead of $i.\text{Gen}()$, interpreting i as a party identifier.

Game 2: Multi-User Unforgeability

<u>Initialize()</u> 1: $\text{Sigs}[*] \leftarrow \mathbf{H} \leftarrow \mathbf{C} \leftarrow \emptyset$ 2: $\text{SK}[*] \leftarrow \text{VK}[*] \leftarrow \square$ 3: $\text{pp} \leftarrow \text{DS.Setup}$ 4: return pp <u>Gen(i)</u> 5: if $\text{VK}[i] = \perp$ then 6: $(\text{SK}[i], \text{VK}[i]) \leftarrow \text{DS.Gen}(\text{pp})$ 7: $\mathbf{H} \leftarrow \mathbf{H} \cup \{i\}$ 8: return $\text{VK}[i]$	<u>Corrupt(i)</u> 9: $\mathbf{C} \leftarrow \mathbf{C} \cup \{i\}$ 10: $\mathbf{H} \leftarrow \mathbf{H} \setminus \{i\}$ 11: return $\text{SK}[i]$ <u>Sign(i, m)</u> 12: check $i \in \mathbf{H}$ 13: $\sigma \leftarrow \text{DS.Sig}(\text{SK}[i], m)$ 14: $\text{Sigs}[i, m] \leftarrow \text{Sigs}[i, m] \cup \{\sigma\}$ 15: return σ	<u>n-UF-CMA(i, m, σ)</u> 16: check $\{i : \text{VK}[i] \neq \square\} \leq n$ 17: check $i \in \mathbf{H}$ 18: check $\text{Sigs}[i, m] = \emptyset$ 19: return $\text{DS.Ver}(\text{VK}[i], m, \sigma)$ <u>SIGCORRECTNESS(i, m, σ)</u> 20: check $i \in \mathbf{H}$ 21: check $\sigma \in \text{Sigs}[i, m]$ 22: return $\neg \text{DS.Ver}(\text{VK}[i], m, \sigma)$
--	---	--

obtain their secret keys. The adversary also has access to a signing oracle that generates signatures on messages of its choice for a given party. The game keeps track of generated signatures and signers. The n -UF-CMA property requires that no adversary is able to compute a new (m, σ) pair that verifies for any honest party i , given up to n queries. A standard guessing argument shows that the advantage in the n -UF-CMA game is bounded by n times that in the standard UF-CMA game, even under adaptive corruptions.

Collision-resistant hash functions. A concrete hash function $\text{Hash} : \{0, 1\}^m \rightarrow \{0, 1\}^l$ with $m > l$ is collision-resistant if it is computationally infeasible (or not known how [42]) to find distinct x and x' such that $\text{Hash}(x) = \text{Hash}(x')$. This is formalized in Game 3.

Game 3: Collision Resistance

<u>CR(x, x')</u> 1: check $x \neq x'$ 2: return $\text{Hash}(x) = \text{Hash}(x')$

Succinct proof systems. A (succinct) non-interactive proof (or argument) system for a relation $\mathcal{R}$ consists of a triple of PPT algorithms $\text{SNARK} := (\text{Setup}, \text{Prove}, \text{Verify})$:

Setup($\lambda, \mathcal{R}$) $\rightarrow$ crs: Generates a common (potentially structured) reference string crs for the relation $\mathcal{R}$.

Prove(crs, x, w) $\rightarrow \pi$: Computes a proof π that $(x, w) \in \mathcal{R}$.

Verify(crs, x, π) $\rightarrow b$: Outputs a bit b , indicating the proof's validity.

Completeness. Informally, a proof system is complete if it is infeasible to compute a proof for a valid statement-witness pair that does not verify. This property is formalized in Game 4 under the COMPLETENESS finalization procedure.

Knowledge soundness. The notion of *knowledge soundness* requires that for any malicious prover $\mathcal{A}$, there exists an efficient extractor $\text{Ext}_{\mathcal{A}}$ that can compute a witness whenever $\mathcal{A}$ produces a valid proof. The extractors considered here are non-black-box and possess full access to the prover's state, including its random coins. This property is formalized in Game 4 under the KNOWLEDGESOUNDNESS finalization procedure.

Game 4: SNARK Security

<u>Initialize()</u> 1: $\text{crs} \leftarrow \text{SNARK.Setup}(\mathcal{R})$ 2: return crs <u>COMPLETENESS(x, w)</u> 3: check $\mathcal{R}(x, w)$ 4: $\pi \leftarrow \text{SNARK.Prove}(\text{crs}, x, w)$ 5: return $\neg \text{SNARK.Verify}(\text{crs}, x, \pi)$ <u>KNOWLEDGESOUNDNESS(x, π)</u> 6: check $\text{SNARK.Verify}(\text{crs}, x, \pi)$ 7: $w \leftarrow \text{Ext}_{\mathcal{A}}(\text{crs}, \text{Coins}(\mathcal{A}))$ 8: return $\neg \mathcal{R}(x, w)$

Succinctness. We call π a Succinct Non-interactive ARgument of Knowledge (SNARK) if for any x and w , the length of the proof π is bounded by $|\pi| \in \text{poly}(\lambda) \cdot \text{polylog}(|x| + |w|)$.

APPENDIX E

LEDGER SYNTAX AND SECURITY

A. Ledger protocols

Below we describe various interfaces for a ledger protocol Π -Ledger associated with a ledger scheme. See Protocol 1 for the code.

P_{sid}.KGen(pp): On input public parameters pp, it runs KGen(pp) to obtain ledger maintenance keys, which are registered with a PKI, together with the initial stake of the party running this interface. The keys are locally stored. This interface is run once at round 0.

P_{sid}.Genesis(): Obtains pp from the CRS and the list of verification keys vk and corresponding stake values w from the PKI. It runs the Genesis algorithm to compute a genesis block G. This interface is run once at round 1.

P_{sid}.Submit(tx): On input a transaction tx, it obtains a list of protocol-specific messages m. It then runs the main blockchain maintenance algorithm BC on these inputs, to compute a new protocol message m and new state st. This interface is run once from round 2 onwards.

P_{sid}.Chain(): Computes the current chain by applying the Chain algorithm to the caller local state. This interface

can be run multiple times per round. This interface is needed to allow inspection of public, but ledger-protocol specific cryptographic data-structures such as VRF evaluations, headers, and consensus certificates. This is valuable for building low-level ledger protocol extensions, such as bridges.

$P_{\text{sid}}.\text{Ledger}()$: Computes the current ledger by using the L and Chain algorithms applied to the caller local state. This interface can be run multiple times per round.

$P_{\text{sid}}.\text{Header}()$: Computes the header for the current chain. This interface can be run multiple times per round.

$P_{\text{sid}}.\text{AChain}()$: Computes the agreed prefix of the current chain. This interface can be run multiple times per round.

B. Enabling smart contracts

Recall that our ledger schemes are formulated relative to a validity predicate Val . Our bridge protocols will rely on deploying contracts in order to bridge two ledgers. We present a concrete validity predicate Val that supports smart-contract functionality. At a high level, Val interprets transactions as a piece of contract code and some arguments. It then evaluates the code on the current ledger and the provided arguments. This is done iteratively, starting from the genesis block (which is processed using a dedicated initial contract-state computation procedure initCS). In this execution, the code of a contract can call other deployed contracts and request to transfer funds.

More precisely, a key-value store CS , which we call the *contract state*, contains, for each contract-id cID , a tuple $(\text{code}, \text{mem}, \text{funds})$ consisting of the contract code, its memory, and funds. Following standard practices, we assume that cID is computed via hashing the code . Predicate Val relies on an updateCS procedure that given a contract state CS and a transaction tx outputs an updated contract state (which may be $\perp$ indicating an invalid transaction). The updateCS procedure is applied iteratively to each of the transactions in a given ledger in order to compute the final contract state. Procedure contractState starts this iterative process using initCS , typically by reading off the list of ledger-maintenance keys and initial stakes from the genesis block. We also assume the genesis block encodes standard account contract code for the initial stakeholders, which initCS deploys.

In subsequent invocations, updateCS takes CS and parses a given transaction tx as $(\text{cID}, \text{method}, \text{args}; v, \text{sID})$ comprising a contract ID cID , a method of the contract method , and arguments args , a received-fund amount, and a fund-sender contract-id sID . For top-level contract calls it checks that the received value v is 0 and that the caller contract-id sID is the empty string.⁹ If the specified method is install , updateCS parses args further as code and sets $\text{CS}[\text{cID}]$ to $(\text{code}, \square, 0)$. (If a contract is already deployed at cID , the request is ignored.) We note that in this emulation CS is a *local* variable and is computed “from scratch” given a ledger (i.e., a list of transactions) L .

⁹We do not currently model gas costs, minting (besides genesis) or off-chain-to-contract transfers.

This emulation contains standard instructions that are applied to the current memory. Additionally, a contract can call a method method' of some other contract dID on args', v' . The call is implemented in the natural way by applying the updateCS to the current contract state CS and transaction $(\text{dID}, \text{method}', \text{args}'; v', \text{cID})$. The value v' indicates the amount of funds transferred between contracts. updateCS checks if the funds to be transferred are positive and that the sending contract cID has sufficient funds. If so, it goes on to reduce the funds of caller contract cID and increments those of the receiving contract dID by the given value. This design is inspired by Ethereum’s approach for transferring funds, though we have simplified it to fit the scope of the paper.

C. Ledger assumption example

In Algorithm 15, we formalize an example assumption, $\mathbb{A}_1[\rho]$, which formalizes a simple ρ -honesty assumption. This assumption is typically used in permissioned blockchains or fixed-committee settings.

D. Ledger security, an extended discussion

We define ledger security as properties of the execution trace of a UC protocol. Our definitions depend on the views of different parties at different rounds (e.g., regarding their local chains and ledgers). We define a “helper” function $\mathbb{C}(P, r)$ that returns the output of $P_{\text{sid}}.\text{Chain}()$ in round r from $\text{view}[r]$ as long as no $P_{\text{sid}}.\text{Submit}$ calls are placed after it (and $\perp$ otherwise). We define a matching $\mathbb{L}(P, r)$ function that returns the corresponding ledger. We describe these functions now and formally define them in Finalization 16.

We recall two simple correctness requirements already stated in the body of this work. Genesis correctness states that in every round, the genesis block returned by $P_{\text{sid}}.\text{Genesis}$ matches that returned by $\text{getGenesis}(C)$, where C is P ’s view of the chain. We also require ledger validity: no invalid transactions are included in any (honest) party’s (local) view of the ledger in any round. See Finalization 17.

We now restate *ledger safety*, *ledger δ -liveness*, and *agreed prefix* properties before stating various other ledger properties that are not already defined in the body of this work. We briefly discuss relations among these notions.

Ledger safety. We follow [49] in defining the safety property, which intuitively requires that the ledgers are *append-only*, and that the ledgers of any two honest parties are consistent in the sense that one is always a prefix of the other.

Ledger δ -liveness. Liveness requires that if an honest party P submits a transaction at some round r and this transaction *remains valid* for at least δ rounds in the view of honest parties who have not already included it in their ledger, then at most δ rounds after r any other honest party Q will report the transactions in its local view of the ledger.

Our definition follows that of PSS, but includes a validity check which seems to be missing from PSS’s definition [40, Definition 7.1]. Despite this, our liveness definition is also closer to PSS as GKL defines liveness assuming that the

Algorithm 14: Val with Contract Support

```

Val(G, L, tx)
1: L ← L : tx
2: return (contractState(G, L) ≠ ⊥)
contractState(G, L)
3: CS ← initCS(G)
4: for tx ∈ L do
5:   if tx.v ≠ 0 ∨ tx.sID ≠ ε then return ⊥
6:   // sID must be empty
7:   CS ← updateCS(CS, tx)
8:   if CS = ⊥ then return ⊥
9: return CS
initCS(G, w)
10: for (code[P], vk[P], w[P]) ∈ G do
11:   alD ← Hash(code[P])
12:   CS[alD] ← (code[P], ε, w[P])

updateCS(CS, tx)
13: (clD, method, args; v, sID) ← parse(tx)
14: if method = install then
15:   code ← parse(args)
16:   clD ← Hash(code)
17:   check CS[clD] = ⊥
18:   CS[clD] ← (code, ⊥, 0) // mem set to ε
19:   return CS
20: (code, _, _) ← CS[clD]
21: run code.method as follows:
22: on standard instruction ins:
23:   mem ← ins(CS[clD].mem, args; v, sID)
24:   if mem = ⊥ then return ⊥
25:   CS[clD].mem ← mem
26: on y ← dID.method'(args'; v')
27:   if v' > 0 then
28:     if CS[clD].funds - v' < 0 then return ⊥
29:     CS[clD].funds ← CS[clD].funds - v'
30:     CS[dID].funds ← CS[dID].funds + v'
31:   CS; y ← updateCS(CS, (dID, method', args'; v', clD))
32:   check CS ≠ ⊥
33:   CS[clD].mem.y ← y
34: on return(y)
35:   return CS; y
36: return CS

```

Algorithm 15: Ledger Assumption Example

```

A1[ρ](r; P, G, (C[s], L(P, s))P ∈ P, s ∈ [r])
1: return |C[r] ∩ P| ≤ ρ · |P|

```

environment Submits a transaction for a sufficiently long number of consecutive rounds to all honest parties. Instead we assume that the reliable distribution of transactions to other parties is part of the ledger protocol.

Agreed prefix. Our bridge constructions will need to report *agreed-upon* headers of a sending ledger on a receiving ledger. To make this formal, we define an *agreed prefix* property which requires that in every round all parties can output an agreed upon prefix of everyone’s chain by applying the `AChain` function to their chain. Moreover, the agreed prefix of the ledger at any round is required to contain the agreed prefix of any sufficiently old ledger. This is reminiscent of the “super-safety” property of [49, Definition 13]. A second security property is `APGROWTH` requiring that the agreed prefix grows with the ledger; that is, any party’s local chain (which due to liveness necessarily grows) is contained in the agreed prefix after some delay.

E. Other ledger properties

Self-persistence. This property requires that the local ledger of any party is *append-only* in the sense that if an entry is added to a specific position in a party’s ledger in some round, it will

persist in that position from that round onwards. See property `SELFPERISTENCE` in Finalization 17.

GKL ledger persistence. We formulate two alternative persistence notions that follow those in GKL and PSS. The first property, `GKLPERSISTENCE`, fully decouples persistence from liveness and requires that a valid transaction that is reported in one party’s ledger at a specific position is also reported in any other party’s ledgers in the same position, provided the position is not empty ($\perp$). Note that this property does *not* imply self-persistence, as (in a bad ledger) a valid transaction might switch back to $\perp$. However, when combined with the `APPENDONLY` property, self-persistence is implied.

PSS ledger persistence. The second property `PSSPERSISTENCE` is arguably closer to [40, Definition 7.2]. Here, ledger persistence under an assumption $\mathbb{A}$ posits that if a valid transaction is reported at some position in an honest party’s ledger, it will eventually be reported in this position after a sufficient number of rounds elapses. Thus, this definition has a flavor of liveness mixed into it, though it does not guarantee full liveness since it does not guarantee that a submitted transaction will indeed appear in the party’s local ledger. This notion of persistence implies `GKLPERSISTENCE`, but still it does not imply self-persistence (for essentially the same reason that `GKLPERSISTENCE` does not).

Collision resistance. This property states that headers form succinct fingerprints for a given chain. In general, a chain may be invalid, in which case the header algorithm `Header` may output $\perp$. But if not, we require headers to be distinct on distinct chains. We formalize this under `COLLISIONRESISTANCE` in

Finalization 16: Chain and Ledger Security Properties

Helper $C(P, r)$

```

1: for entry  $\in \text{view}[r]$  do
2:   if entry =  $(\_, P_{\text{sid}}.\text{Submit}, \_, \_)$  then
3:      $C \leftarrow \perp$ 
4:   if entry =  $(\_, P_{\text{sid}}.\text{Chain}, \_, C')$  then
5:      $C \leftarrow C'$ 
6: return  $C$  // last chain after last  $P_{\text{sid}}.\text{Submit}$  of round  $r$ 

```

Helper $L(P, r)$

```

7: return Ledger( $C(P, r)$ )

```

$A\text{-AGREEDPREFIX}(P, Q, r, r')$

```

8: check  $A(r')$ 
9: check  $r \leq r'$ 
10: check  $P \notin C[r] \wedge Q \notin C[r']$ 
11:  $C_P \leftarrow A\text{Chain}(C(P, r)); C_Q \leftarrow A\text{Chain}(C(Q, r'))$ 
12: return  $\neg(C_P \preceq C_Q)$  // if  $r = r'$ , then  $C_P = C_Q$ 

```

$(A, \delta)\text{-APGROWTH}(P, r, r')$

```

13: check  $A(r')$ 
14: check  $r + \delta \leq r'$ 
15:  $C \leftarrow C(P, r); C' \leftarrow A\text{Chain}(C(P, r'))$ 
16: return  $\neg(C \prec C')$ 

```

$A\text{-APPENDONLY}(P, r, r')$

```

17: check  $A(r')$ 
18: check  $r \leq r'$ 
19: check  $P \notin C[r']$ 
20:  $L \leftarrow L(P, r); L' \leftarrow L(P, r')$ 
21: return  $\neg(L \preceq L')$ 

```

$A\text{-LEDGERSAFETY}(P, Q, r, r')$

```

22: if  $A\text{-APPENDONLY}(P, r, r')$  then return 1
23: check  $A(r')$ 
24: check  $P \notin C[r] \wedge Q \notin C[r']$ 
25:  $L_P \leftarrow L(P, r); L_Q \leftarrow L(Q, r')$ 
26: return  $\neg(L_P \preceq L_Q \vee L_Q \preceq L_P)$ 

```

$(A, \delta)\text{-LEDGERLIVENESS}(P, Q, r, r', \text{tx})$

```

27: check  $A(r')$ 
28: check  $r + \delta \leq r'$ 
29: check  $P \notin C[r] \wedge Q \notin C[r']$ 
30: check  $\exists \text{tx}. (\_, P_{\text{sid}}.\text{Submit}, \text{tx}, \_) \in \text{view}[r]$ 
31: check  $\forall s \in [r, r + \delta]. \forall R \notin C[s]. \text{Val}(L(R, s), \text{tx}) \vee \text{tx} \in L(R, s)$ 
32: return  $\neg(\text{tx} \in L(Q, r'))$ 

```

Finalization 17. We note that for typical ledgers collision resistance is already implied by other ledger security properties (such as safety and liveness), though not in a fully black-box manner.

Self-liveness. This notion requires that if an honest party receives a transaction that is valid for a sufficiently long window of time, then it is either already included in that party's ledger or will be included at the end of the period. See property **SELFIVENESS** in Finalization 17.

Cross-liveness. This (arguably simple) liveness notion requires that the transactions in the ledger of any honest party appear in the ledger of all other honest parties after some fixed maximal delay. It is straightforward to see that **CROSSLIVENESS** and **SELFIVENESS** together imply **LEDGERLIVENESS**: one ensures that submitted transactions appear in every party's ledger; the other ensures that all other honest parties also have it, as required by liveness.

APPENDIX F

THE LEDGER FUNCTIONALITY

Model checking requires ideal building blocks. We present an idealization of a ledger. To simplify, we focus on defining a *ledger* rather than a chain, which is sufficient for our purposes since we focus on ledger properties such as transaction inclusion.

Our ledger functionality $\mathcal{G}\text{-Ledger}$ comes with the following interfaces.

- A $P_{\text{sid}}.\text{KGen}()$ interface for the preliminary stage of genesis generation where potentially some keys (for proof-of-stake protocols) are generated.
- A $P_{\text{sid}}.\text{Genesis}()$ interface generating, via the simulator, a genesis block G . This interface ensures agreement on G .

- A $P_{\text{sid}}.\text{Submit}(\text{tx})$ interface allowing parties in any session to submit transactions tx to the ledger (recall that the functionality is global and accessible from any session). The transaction and the submission times are recorded in buffer.
- A $P_{\text{sid}}.\text{Ledger}()$ interface allows parties in different sessions to read the current state of the ledger. Upon a read request, the functionality prepares a growing log of *valid* transactions submitted so far. All transactions older than Δ_{lat} rounds (the latency parameter) are included in the log. Then, for each party, the simulator picks a pointer to a prefix of this log. This pointer is sanitized to be increasing wrt. the party's last pointer, and to ensure a sufficiently recent prefix of the log. (Thus different parties' pointers are within a "neighborhood" of each other.) A slack parameter Δ_{slack} determines how far back from the current round the pointer can be, and thus how synchronized parties are. The validity of transactions is decided using the Val function.
- In the $P_{\text{sid}}.\text{ComSel}[c, \rho]()$ interface, we ask the simulator to output a committee sanitized to satisfy the properties defined in Finalization 4. See Functionality Extension 19 for the details.
- Similarly, in the $P_{\text{sid}}.\text{ALedger}[\Delta_{\text{apl}}]()$ interface, we ask the simulator for a pointer, which is sanitized to be at most the minimum of all parties' pointers, and at least lags from this by some delay parameter. See Functionality Extension 20 for the details.
- Interface $P_{\text{sid}}.\text{ALedgerHeader}()$ computes a collision-resistant header value for the agreed ledger. Its definition borrows ideas from $\mathcal{F}\text{-Hash}$. See Functionality Extension 21 for the details.

Finalization 17: Other Chain and Ledger Security Properties

LEDGERVALIDITY(P, r)

```
1: check  $P \notin C[r]$ 
2:  $L' : tx \leftarrow L(P, r)$ 
3: return  $\neg Val(L', tx)$ 
```

A-SELF PERSISTENCE(P, r, r', pos)

```
4: check  $A(r')$ 
5: check  $r \leq r'$ 
6: check  $P \notin C[r']$ 
7:  $tx \leftarrow L(P, r)[pos]$ ;  $tx' \leftarrow L(P, r')[pos]$ 
8: check  $tx \neq \perp$ 
9: return  $\neg(tx = tx')$ 
```

A-GKL PERSISTENCE(P, Q, r, r', pos)

```
10: check  $A(r')$ 
11: check  $r \leq r'$  // can be removed
12: check  $P \notin C[r] \wedge Q \notin C[r']$ 
13:  $tx \leftarrow L(P, r)[pos]$ ;  $tx' \leftarrow L(Q, r')[pos]$ 
14: check  $tx \neq \perp \wedge tx' \neq \perp$ 
15: return  $\neg(tx = tx')$ 
```

(A, δ)-PSS PERSISTENCE(P, Q, r, r', pos)

```
16: check  $A(r')$ 
17: check  $r + \delta \leq r'$ 
18: check  $P \notin C[r] \wedge Q \notin C[r']$ 
19:  $tx \leftarrow L(P, r)[pos]$ ;  $tx' \leftarrow L(Q, r')[pos]$ 
20: check  $tx \neq \perp$ 
21: return  $\neg(tx = tx')$ 
```

COLLISION RESISTANCE(C, C')

```
22: check  $C \neq C'$ 
23:  $h \leftarrow \text{Header}(C)$ ;  $h' \leftarrow \text{Header}(C')$ 
24: check  $h \neq \perp \wedge h' \neq \perp$  // if C is invalid
25: return  $\neg(h \neq h')$ 
```

GENESIS CORRECTNESS(P, r, G)

```
26: check  $P \notin C[r]$ 
27: check  $(\_, P_{sid}.Genesis, \_, G) \in \text{view}[1]$ 
28:  $G' \leftarrow \text{getGenesis}(C(P, r))$ 
29: return  $\neg(G = G')$ 
```

(A, δ)-SELF LIVENESS(P, r, r', tx)

```
30: check  $A(r')$ 
31: check  $r + \delta \leq r'$ 
32: check  $P \notin C[r]$ 
33: check  $\exists tx. (\_, P_{sid}.Submit, tx, \_) \in \text{view}[r]$ 
34: check  $\forall s \in [r, r + \delta]. Val(L(P, s), tx) \vee tx \in L(P, s)$ 
35: return  $\neg(tx \in L(P, r'))$ 
```

(A, δ)-CROSS LIVENESS(P, Q, r, r', tx)

```
36: check  $A(r')$ 
37: check  $r + \delta \leq r'$ 
38: check  $P \notin C[r] \wedge Q \notin C[r']$ 
39: check  $tx \in L(P, r)$ 
40: return  $\neg(tx \in L(Q, r'))$ 
```

For model checking, we use a concrete validity predicate. Moreover, since all components are idealized, like NIZKs with externalized relations, we externalize the validity relation in $\mathcal{G}$ -Ledger. This is done by replacing line 27 in $\mathcal{G}$ -Ledger by $b \leftarrow \mathcal{F}\text{-Val}.P_{sid^*}.Eval(log, tx)$.

We present a concrete one, used in the ATS bridge model checking, in Functionality 22. The implicit caller sid in $\mathcal{F}\text{-Val}$ is sid^* , the session id of $\mathcal{G}$ -Ledger. We note that this functionality only uses the ATS verification interface, which is realized by a deterministic, stateless algorithm (from an ATS scheme). Thus the approach of Camenisch et al. [10] can be applied.

APPENDIX G BRIDGE PROTOCOLS

We now describe the interfaces for a bridge protocol Π -Bridge (see Protocol 2) in more detail.

P.BKGen(): Obtains the common parameters via $\mathcal{F}\text{-CRS}$, generates bridge protocol keys using BKGen , and registers them with a PKI using $\mathcal{F}\text{-PKI}$. This interface is required to be run once in round r_0 , when the bridge parties intend to initialize a bridge.

P.RoT(): The data required to run the RoT algorithm is obtained from $\mathcal{F}\text{-CRS}$, $\mathcal{F}\text{-PKI}$, and the P.Chain interfaces of two ledgers. Note that we assume $\mathbf{P} \subseteq \mathbf{P}_i$ and $\mathbf{P} \subseteq \mathbf{P}_j$. This interface is required to be run once in round $r_0 + 1$ so that all bridge maintenance keys can be obtained from the $\mathcal{F}\text{-PKI}$. The deterministic nature of this algorithm

ensures that the same rot is obtained when executed on the same inputs. Thus, honest parties will agree on rot if RoT only relies on the agreed prefix of an input chain.

P.Bridge(): Bridge parties prepare data for bridge maintenance using the $\mathcal{F}\text{-Diffuse}$ network as well as the P.Chain interfaces of the two ledgers. They then run the Br algorithm and diffuse the protocol messages via $\mathcal{F}\text{-Diffuse}$ and transactions to the ledgers via P.Submit . Note that, once again, $\mathbf{P} \subseteq \mathbf{P}_i$ and $\mathbf{P} \subseteq \mathbf{P}_j$. This interface is required to be run once per round from round $r_0 + 2$ onward.

P.Certify(rot, x): This interface allows parties in $\mathbf{P}_i$ (a subset of whom are bridge operators) to prepare a property certificate, typically in the form of a proof, that a certain statement x holds with respect to the current state of the chain C_i obtained from the P.Chain interface of $\Pi\text{-Ledger}_i$. Algorithm Certify is used to compute the certificate/proof π . As part of this operation, the parties need access to the root-of-trust string rot . We make this an explicit input to the interface. The interpretation is that rot is distributed authentically via an off-band mechanism (e.g., through a web service that securely distributes the bridge software). Accordingly, security properties will hold only if this interface (and the P.Verify interface – see next) are run wrt. the correct rot . This interface can be called multiple times per round. We will implement Certify using SNARKs. However, more efficient (though also more restricted) solutions can be envisioned based on

Functionality 18: $\mathcal{G}$ -Ledger $[\mathbf{P}, \text{sid}^*, r_0, \text{Val}, \Delta_{\text{latency}}, \Delta_{\text{slack}}]$

Building Blocks:

```

1:  $\mathcal{G}$ -Clock $[\mathbf{P}']$  with  $\mathbf{P} \subseteq \mathbf{P}'$ 
Init() // state is shared across all sid's
2: buffer[*]  $\leftarrow \square$  // waiting tx's
3: log  $\leftarrow []$  // settled tx's
4: elD  $\leftarrow 0$ ; ptr[*,*]  $\leftarrow \perp$ ; G  $\leftarrow \square$ 
Psid.Init()
5:  $\mathcal{G}$ -Clock.Psid*.Reg()
Psid.KGen()
6:  $r \leftarrow \mathcal{G}$ -Clock.Psid*.Round()
7: if  $r \neq r_0$  then return  $\perp$ 
8: Sim(Psid.KGen)
9:  $\mathcal{G}$ -Clock.Psid*.OK()
Psid.Genesis()
10:  $r \leftarrow \mathcal{G}$ -Clock.Psid*.Round()
11: if  $r \neq r_0 + 1$  then return G
12: if G  $\neq \square$  then return G
13: G  $\leftarrow \text{Sim}(\text{P}_{\text{sid}}.\text{Genesis})$ 
14: G  $\leftarrow \text{San}[\text{Clean}_G](G)$ 
15:  $\mathcal{G}$ -Clock.Psid*.OK()
16: return G
Psid.Submit(tx) // any sid
17:  $r \leftarrow \mathcal{G}$ -Clock.Psid*.Round()
18: elD  $\leftarrow \text{elD} + 1$ 
19: buffer[elD]  $\leftarrow (\text{tx}, r)$ 
20: Sim(Psid.Submit, tx)
21:  $\mathcal{G}$ -Clock.Psid*.OK()
22: return ok

```

```

Psid.Ledger() // any sid
23: (Idx, p)  $\leftarrow \text{Sim}(\text{P}_{\text{sid}}.\text{Ledger})$ 
24:  $r \leftarrow \mathcal{G}$ -Clock.Psid*.Round()
25: Idx  $\leftarrow \text{SanIdx}(\text{Idx}, r)$ 
26: for  $i \in \text{Idx}$  do
27:   if buffer[i]  $\neq \square$  then
28:     b  $\leftarrow \text{Val}(\text{log}, \text{buffer}[i].\text{tx})$ 
29:     if b then
30:       log  $\leftarrow \text{log} : (\text{buffer}[i].\text{tx}, r)$ 
31:       buffer[i]  $\leftarrow \square$ 
32: p  $\leftarrow \text{San}_{\text{ptr}}(p, \text{ptr}[\text{P}_{\text{sid}}], r)$ 
33: if log = []  $\vee$  p =  $\perp$  then return []
34: ptr[Psid]  $\leftarrow$  p
35: return log[: ptr[Psid]]
SanIdx(Idx, r) // returns a list of pointers
36: for  $i \in \text{Idx}$  do
37:   if buffer[i] =  $\square$  then Idx  $\leftarrow \text{Idx} \setminus \{i\}$ 
38: for  $i \in [i : \text{buffer}[i] \neq \square \wedge i \notin \text{Idx}]$  do
39:   if  $r - \text{buffer}[i].r \geq \Delta_{\text{latency}}$  then
40:     Idx  $\leftarrow \text{Idx} : i$  // liveness: tx's get added
41: return Idx
Sanptr(p, p', r) // returns a pointer or  $\perp$ 
42: p*  $\leftarrow \max(\{i : r - \text{log}[i].r \geq \Delta_{\text{slack}}\} \cup \{p\})$ 
43: p*  $\leftarrow$  if  $p' = \perp$  then p* else max( $\{p^*, p'\}$ )
44: if p* < 0 then return  $\perp$ 
45: if p* > |log| - 1 then p*  $\leftarrow$  |log| - 1
46: return p*
Pred. CleanG(G)
9: return (G  $\neq \perp$ )

```

Functionality Extension 19: P_{sid}.ComSel $[c, \rho]$

```

Init() // extends  $\mathcal{G}$ -Ledger's Init
1: Com[*]  $\leftarrow \emptyset$ 
Psid.ComSel() // any sid
2:  $r \leftarrow \mathcal{G}$ -Clock.Psid*.Round()
3: Com  $\leftarrow \text{Sim}(\text{P}_{\text{sid}}.\text{ComSel})$ 
4: if Com[r]  $\neq \square$  then return Com[r]
5: Com  $\leftarrow \text{San}[\text{Clean}_{\text{Com}}[c, \rho]](\text{Com}, \mathbf{P}, \mathbf{C})$ 
6: Com[r]  $\leftarrow$  Com
7: return Com

```

```

Pred. CleanCom[c, ρ](Com, P, C)
// lists counted with multiplicity
10: bcor  $\leftarrow (|\text{Com}| = c \wedge \text{Com} \subseteq \mathbf{P})$ 
11: bsec  $\leftarrow (|\{P \in \text{Com} : P \in \mathbf{C}\}| \leq \rho \cdot |\text{Com}|)$ 
12: return bcor  $\wedge$  bsec

```

Merkle trees (which check the inclusion of transactions in headers).

P.Verify(rot, x, π): Parties maintaining ledger j can use this interface to verify statements about the state of the sending ledger. As part of this operation, the parties obtain the data required to run the Verify algorithm using the getVerData procedure. This procedure relies on the rot string and the ledger of the receiving ledger, which is computed by applying Ledger _{j} to C _{j} obtained via the P.Chain interface of the receiving ledger. This interface can be called multiple times per round.

APPENDIX H ANALYSES OF BRIDGES

A. Direct-observation bridge

Perhaps the simplest type of bridge is one where the set of parties $\mathbf{P}_j$ that operate the receiving ledger *directly observe* the state of the sending ledger, and accept a given header hash as part of a special bridge transaction if it indeed corresponds to that of an agreed prefix of the sending ledger. All parties maintaining the receiving ledger partake in bridge operations and thus, $\mathbf{P} = \mathbf{P}_j$. Furthermore, they need to observe the sending ledger and hence $\mathbf{P} \subseteq \mathbf{P}_i$. Typically, bridge operators

Functionality Extension 20: $P_{\text{sid}}.\text{ALedger}[\Delta_{\text{apl}}]$

Init()

1: $\text{apl}[*] \leftarrow \square$

$P_{\text{sid}}.\text{ALedger}()$ // any sid

2: $r \leftarrow \mathcal{G}\text{-Clock}.P_{\text{sid}}.\text{Round}()$

3: $p \leftarrow \text{Sim}(P_{\text{sid}}.\text{ALedger})$

4: **if** $\text{apl}[r] \neq \square$ **then return** $\text{apl}[r]$

5: $S \leftarrow \{\text{ptr}[P'_{\text{sid}'}] : \text{ptr}[P'_{\text{sid}'}] \neq \square\}$

6: **if** $S = \emptyset$ **then return** $\perp$

7: $p \leftarrow \text{San}[\text{Clean}_{\text{apl}}](p, S)$

8: $\text{apl}[r] \leftarrow p$

9: **return** $\log[:p]$

$\text{Pred. Clean}_{\text{apl}}(p, S)$

10: **return** $(p \in [\max(\{0, \min(S) - \Delta_{\text{apl}}\}).. \min(S)])$

Functionality Extension 21: $P_{\text{sid}}.\text{ALedgerHeader}()$

Init()

1: $H[*] \leftarrow \square$

$P_{\text{sid}}.\text{ALedgerHeader}()$ // any sid

2: $\log \leftarrow \mathcal{G}\text{-Ledger}.P_{\text{sid}}.\text{ALedger}()$

3: **if** $H[\log] \neq \square$ **then return** $H[\log]$

4: $h \leftarrow \text{Sim}(P_{\text{sid}}.\text{ALedgerHeader}, \log)$

5: $h \leftarrow \text{San}[\text{Clean}_H](h, H)$

6: $H[\log] \leftarrow h$

7: **return** h

$\text{Pred. Clean}_H(h, H)$

11: **return** $\neg(\exists \log. H[\log] = h)$

Functionality 22: $\mathcal{F}\text{-Val}[P]$

Building Blocks:

1: $\mathcal{F}\text{-ATS}[P', ..]$ with $P \subseteq P'$

Init()

2: $\text{mem}[*].(\text{hs}, \text{avk}) \leftarrow (\square, \square)$

$P.\text{contractState}(\text{clD})$

3: **return** $\text{mem}[\text{clD}]$

$P.\text{Eval}(\log, \text{tx})$

4: **if** $\text{tx} \in \log$ **then return** 0

5: **if** $\text{tx} = (\text{normal}, _, b)$ **then return** b // annotated tx's

6: **if** $\text{tx} = (\text{install}, \text{clD}, \text{code}) \wedge \forall \text{clD}'. (\text{install}, \text{clD}', \text{code}) \notin \log$ **then**

7: $\text{mem}[\text{clD}].\text{hs} \leftarrow []$ // list hash_out

8: $\text{mem}[\text{clD}].\text{avk} \leftarrow \perp$

9: **return** 1

10: **if** $\text{tx} = (\text{update}, m, \sigma) \wedge \exists \text{clD}, \text{code}. (\text{install}, \text{clD}, \text{code}) \in \log$ **then**

11: $\text{get}(\text{install}, \text{clD}, \text{code}) \in \log$

12: $\text{mem} \leftarrow \text{mem}[\text{clD}]$

13: $\text{avk} \leftarrow \text{mem}.\text{avk}$

14: **if** $\text{avk} = \perp$ **then**

15: $\text{avk} \leftarrow \text{code}$

16: $b \leftarrow \mathcal{F}\text{-ATS}.P.\text{AVer}(\text{avk}, m, \sigma)$

17: **if** b **then**

18: $(h, \text{avk}') \leftarrow \text{parse}(m)$

19: $\text{mem}.\text{hs} \leftarrow \text{mem}.\text{hs} : h$

20: $\text{mem}.\text{avk} \leftarrow \text{avk}'$

21: $\text{mem}[\text{clD}] \leftarrow \text{mem}$

22: **return** 1

23: **return** 0

24: **return** 0

need to read from the sending ledger and write onto the receiving one, thus implying that $P \subseteq P_i \cap P_j$.

To implement a direct-observation bridge, bridge parties observe the sending ledger and relay headers via bridge transactions. Moreover, the ledger validity rules of the receiving ledger must be configured so as to enable reaching consensus on the actual header hashes of the sending ledger. This entails a form of *soft* fork in the sense that all standard (non-bridge) transactions remain valid once the validity rule is upgraded to deal with bridge transactions, but bridge transactions that

contain invalid headers are not accepted by the new validity rule.¹⁰

In more detail, there are no keys in a direct-observation bridge, and the public parameters and the root of trust are simply a CRS for a proof system (see Bridge Scheme 2). The bridge algorithm Br in round $r_0 + 2$ prepares a contract $\text{DOBridge}[\text{rot}]$, with rot hardwired in, to be deployed on the

¹⁰That is, the old node software accepts everything the new node software accepts, but the new software does not accept header transactions that might pass for the old 'obsolete' validity predicate.

receiving ledger. We note that all (honest) parties attempt to deploy this contract, but only the fastest transaction will result in an actual deployment. Since all parties know the root of trust, they know the contract's ID, and thus can submit transactions to the correct contract.

This contract has two methods, `update` and `verify`. In rounds $r \geq r_0 + 3$, the `Br` algorithm prepares a transaction which submits (ℓ, h) to the `update` method of `DOBridge`. Here ℓ is the length of an agreed prefix and h is the header information at this length. This transaction is processed by an extension to the validity rule of the receiving ledger: it is marked as valid only if h matches that of the sending ledger at the given length ℓ (see algorithm `xcVal`). Note that as part of this, parties in $\mathbf{P}_j$ need to observe the sending ledger so that they can run `xcVal`.¹¹ It is important that the header of an agreed prefix is used here so that all parties store the same header value. The updated validity rule `val'_j` checks the validity predicate of the ledger and this extension. The `verify` method verifies proofs using the CRS in the contract memory. This is to give other contracts the ability to check the validity of statements through a uniform interface and without keeping track of headers.

In order to show that a certain statement x holds for the sending ledger L we will rely on a proof system to show that (1) x holds for *some* ledger L , and (2) that L has a fingerprint (hash value) that matches one that is included as part of a bridge transaction. For this to verify (so that liveness is not violated) we need the relation $\mathcal{R}$ to only look at the agreed prefix of a given chain. More formally, we assume that $\mathcal{R}$ takes the form

$$\mathcal{R}(x, C) := \mathcal{R}'(x, \text{AChain}_i(C)) ,$$

for some other relation $\mathcal{R}'$, where `AChaini` is the agreed-prefix computation algorithm. The corresponding zk-relation then is

$$\mathcal{R}_{zk}((x, h), C) := (\text{Header}_i(C, \text{apl}_i(C)) = h \wedge \mathcal{R}(x, C)) ,$$

where `Headeri` is the header algorithm for the sending chain and `apli` is the agreed-prefix algorithm.¹²

When verifying statements, we need to decide which header value will be used in the verification procedure of the proof system. To this end, we assume that the proofs include this header information. If the specified header information does not exist, the proof is rejected (see algorithm `Verify`).

More formally, on input (rot, x, C) the `Certify` algorithm computes a proof, with respect to $\mathcal{R}_{zk}$ defined above. To retrieve the data needed for verification, `getVerData` runs the `contractState` algorithm on its input ledger, and looks up the memory field `mem` from contract state `CS[cID]`, where `cID` is the contract ID of `DOBridge`. The `Verify` algorithm obtains the CRS from the memory `mem`. It then checks if the

¹¹This means that the operation of the header verification cannot be completed by data solely available on the receiving ledger, and thus it cannot be implemented via a smart contract.

¹²We note that proving $\mathcal{R}_{zk}$ efficiently is a matter of choosing an efficiently implementable $\mathcal{R}$ and an efficient authenticated data structure [45] for the header.

hash value h (included with the proof) appears in `mem`. If not, the proof is rejected; otherwise it is verified according to the verification algorithm of the proof system.

We next prove that the direct-observation bridge is sound and live as long as the sending and receiving ledger are secure (super-safe and live), the proof system is knowledge sound and complete, and the header hash is collision resistant.

Theorem 4 (Direct-Observation Bridge Soundness). *For any given trust assumptions $\mathbb{A}$ and $\mathbb{A}_i$ the direct-observation bridge (protocol `II-Bridge` wrt. `Bridge Scheme 2`) from a sending ledger `II-Ledgeri` to a receiving ledger `II-Ledgerj` provides `ABRIDGE` SOUNDNESS if:*

- Both ledgers satisfy perfect `LEDGER` VALIDITY;
- The bridge trust assumption implies the sending ledger trust assumption, i.e., $\mathbb{A} \Rightarrow \mathbb{A}_i$, and ledger protocol `II-Ledgeri` satisfies `ABRIDGE` PREFIX;
- The SNARK scheme satisfies `KNOWLEDGE` SOUNDNESS and `Headeri` satisfies `COLLISION` RESISTANCE.

Proof. Consider the event that an adversary $\mathcal{A}$ violates `BRIDGE` SOUNDNESS: it outputs a tuple $(P, Q, r, \text{rot}, x, \pi)$ such that (1) `rot` has also been output by an honest P ; (2) (x, π) is valid wrt. `mem` obtained from `Gj`, `rot`, `Lj`(`Q, r`); but (3) for any $r' \leq r$, there is a party P' honest in round r' with respect to which $\mathcal{R}(x, C_i[P', r'])$ is false. We proceed along four games to show that the probability of this event is negligible.

G₀: We start with the `BRIDGE` SOUNDNESS game:

$$\Pr[\text{BRIDGE} \text{ SOUNDNESS}] - \Pr[G_0] = 0 .$$

G₁ This game aborts (without $\mathcal{A}$ winning) if any transaction added to any ledger, and in particular to the ledger `Lj`(`Q, r`) or some `Li`(`P', r'`) is invalid.¹³ By perfect `LEDGER` VALIDITY we have that

$$\Pr[G_0] - \Pr[G_1] = 0$$

G₂: This game aborts if some header (ℓ, h) is submitted to the contract and added to its `mem.hs` but h does not match the header of the sending chain at length ℓ . Since the extended ledger validity rules mandate checking that submitted headers indeed match those of the sending chain, we have that

$$\Pr[G_1] - \Pr[G_2] = 0 .$$

G₃: This game extracts a chain C_i^* from any proof π that passes verification, where the extractor gets the coins of *both* the game and the adversary. If C_i^* does not verify wrt. $\mathcal{R}_{zk}$, the game aborts. A reduction to knowledge soundness shows that the probability of this event is small. In more detail, the reduction uses the CRS oracle from the knowledge soundness game as the CRS value in the bridge soundness game. It then runs the adversary faithfully in the

¹³This step is mostly to simplify our reasoning. An invalid transaction in `Lj`(`Q, r`) would already result in the adversary losing, due to `Verify` in line 6 failing for `mem` = $\perp$. Similarly, an invalid ledger `Li`(`P', r'`) does not seem to make it any easier to prove statements about it.

Bridge Scheme 2: Direct Observation

```

Setup( $\lambda$ )
1:  $\text{crs} \leftarrow \text{SNARK.Setup}(\lambda)$ 
2: return  $\text{crs}$ 
BKGen( $\text{crs}$ ) when  $r = r_0$ 
3: return  $(\varepsilon, \varepsilon)$ 
RoT( $\text{crs}, \text{bvk}, C_i, C_j$ ) when  $r = r_0 + 1$ 
4:  $\text{rot} \leftarrow \text{crs}$ 
5: return  $(\text{rot}, \varepsilon)$ 
Br( $r, \mathbf{m}, C_i, C_j; \text{bst}$ ) when  $r = r_0 + 2$ 
6:  $\text{rot} \leftarrow \text{bst.rot}$ 
7:  $\text{code} \leftarrow \text{DOBridge}[\text{rot}]$ 
8:  $\text{clD} \leftarrow \text{Hash}(\text{code})$ 
9:  $\text{btX}_j \leftarrow (\text{clD}, \text{install}, \text{code})$ 
10:  $\text{bst.clD} \leftarrow \text{clD}$ 
11: return  $(\perp, \text{btX}_j, \perp; \text{bst})$ 
Br( $r, \mathbf{m}, C_i, C_j; \text{bst}$ ) when  $r \geq r_0 + 3$ 
12:  $\text{clD} \leftarrow \text{bst.clD}; \ell \leftarrow \text{apl}_i(C_i)$ 
13:  $\mathbf{h} \leftarrow \text{Header}_i(C_i, \ell)$ 
14:  $\text{btX}_j \leftarrow (\text{clD}, \text{update}, (\ell, \mathbf{h}))$ 
15: return  $(\perp, \text{btX}_j, \perp; \text{bst})$ 
xcVal( $\text{rot}, C_i, \text{btX}$ )
16:  $(\text{clD}, \text{update}, (\ell, \mathbf{h})) \leftarrow \text{parse}(\text{btX})$ 
17: if  $\text{clD} = \text{Hash}(\text{DOBridge}[\text{rot}])$  then
18:   return  $(\mathbf{h} = \text{Header}_i(C_i, \ell))$ 
19: return 1

Val'_j( $L_j, \text{tx}; C_i$ )
20: return  $\text{Val}_j(L_j, \text{tx}) \wedge \text{xcVal}(\text{rot}, C_i, \text{tx})$ 
Certify( $\text{rot}, \mathbf{x}, C$ )
21:  $\text{crs} \leftarrow \text{rot}$ 
22:  $\mathbf{h} \leftarrow \text{Header}_i(C, \text{apl}_i(C))$ 
23:  $\pi' \leftarrow \text{SNARK.Prove}(\text{crs}, (\mathbf{x}, \mathbf{h}), C)$ 
24:  $\pi \leftarrow (\mathbf{h}, \pi')$ 
25: return  $\pi$ 
getVerData( $G, \text{rot}, L$ )
26:  $\text{CS} \leftarrow \text{contractState}_j(G, L)$ 
27:  $\text{clD} \leftarrow \text{Hash}(\text{DOBridge}[\text{rot}])$ 
28:  $(\_, \text{mem}, \_) \leftarrow \text{CS}[\text{clD}]$ 
29: return  $\text{mem}$ 
Verify( $\text{mem}, \mathbf{x}, \pi$ )
30:  $(\mathbf{h}, \pi') \leftarrow \pi$ 
31: check  $\mathbf{h} \in \text{mem.hs}$ 
32: return  $\text{SNARK.Verify}(\text{mem.crs}, (\mathbf{x}, \mathbf{h}), \pi')$ 
DOBridge[rot].update( $\ell, \mathbf{h}$ )
33:  $\text{mem.crs} \leftarrow \text{rot}$ 
34:  $\text{mem.hs} \leftarrow \text{mem.hs} : \mathbf{h} \text{ // ignore } \ell$ 
35: return ok
DOBridge[rot].verify( $\mathbf{x}, \pi$ )
36: return  $\text{Verify}(\text{mem}, \mathbf{x}, \pi)$ 

```

bridge soundness execution environment. Note that there is no need to simulate proofs as the adversary provides witnesses for statements of its choice. The abort event translates to winning the **KNOWLEDGESOUNDNESS** game as long as the correct rot is used:

$$\Pr[G_2] - \Pr[G_3] \leq \text{Adv}_{\text{SNARK}}^{\text{KNOWLEDGESOUNDNESS}}.$$

G₄: This game aborts if the agreed prefix of the extracted chain $\text{AChain}_i(C_i^*)$ is not equal to the chain that some honest bridge party P^* used to compute the header information in the bridge transaction in round $r^* \leq r$. Let $C_i(P^*, r^*)$ be this party's chain. Note that Verify checks the header of the proof $\mathbf{h}^* := \text{Header}_i(C^*, \text{apl}_i(C^*))$ against the header $\mathbf{h} = \text{Header}_i(C_i(P^*, r^*), \text{apl}_i(C_i(P^*, r^*)))$. A reduction to the **COLLISIONRESISTANCE** of the headers of the sending ledger shows that

$$\Pr[G_3] - \Pr[G_4] \leq \text{Adv}_{\Pi\text{-Ledger}_i}^{\text{COLLISIONRESISTANCE}}.$$

G₅: This game aborts if for some $P' \in \mathbf{H}[r^*]$ the agreed prefix $\text{AChain}_i(C_i(P', r^*))$ is not the same as $\text{AChain}_i(C_i(P^*, r^*))$. If this event arises, then the $\mathbb{A}_i$ -**AGREEDPREFIX** property of the sending ledger would be violated:

$$\Pr[G_4] - \Pr[G_5] \leq \text{Adv}_{\Pi\text{-Ledger}_i}^{\mathbb{A}_i\text{-AGREEDPREFIX}}.$$

Note there is no factor $|\mathbf{P}|$ loss since the agreed prefix property can be publicly checked.

G₆: This game replaces searching over all r' in the winning condition with the specific round r^* . This winning condition is more lenient (i.e. it is implied by the full winning condition). In particular G_5 states that the adversary only wins if $\mathcal{R}(\mathbf{x}, C_i(P', r'))$ is false for all r' , while G_6 states that the adversary wins if only $\mathcal{R}(\mathbf{x}, C_i(P', r^*))$ is false. Thus,

$$\Pr[G_5] - \Pr[G_6] \leq 0.$$

G₇: This game runs **BRIDGESOUNDNESS**, but now uses C_i^* in $\mathcal{R}$ instead of $C_i(P', r^*)$. At this point of the game $\text{AChain}_i(C_i^*) = \text{AChain}_i(C_i(P^*, r^*)) = \text{AChain}_i(C_i(P', r^*))$. Moreover, recall that relation $\mathcal{R}$ only depends on the agreed prefix of the input chain. Hence,

$$\Pr[G_6] - \Pr[G_7] = 0.$$

Now, observe that G_5 is not winnable as C_i^* always verifies wrt. relation $\mathcal{R}_{\text{zk}}$, and in particular it satisfies $\mathcal{R}$. Thus,

$$\Pr[G_7] = 0.$$

The theorem follows by collecting the above (in)equalities. $\square$

We next prove bridge liveness.

Theorem 5 (Direct-Observation Bridge Liveness). *For any given assumptions $\mathbb{A}_j$ and $\mathbb{A}$ and delay parameter δ the direct-observation bridge (protocol Π -Bridge wrt. Bridge Scheme 2) from a sending ledger protocol Π -Ledger to a receiving ledger protocol Π -Ledger provides $(\mathbb{A}, \delta)$ -**BRIDGELIVENESS** if:*

- Both ledgers satisfy perfect **LEDGERVALIDITY**;
- The bridge trust assumption implies the receiving ledger trust assumption, i.e., $\mathbb{A} \Rightarrow \mathbb{A}_j$, and ledger $\Pi\text{-Ledger}_j$ satisfies $(\mathbb{A}_j, \delta_j)\text{-LEDGERLIVENESS}$ for some $\delta_j \leq \delta$;
- Under assumption $\mathbb{A}$, there is at least one honest party on the sending chain. Formally, $\forall r. \mathbb{A}(r) \Rightarrow \mathbf{P}_i \setminus \mathbf{C}[r] \neq \emptyset$;
- The **SNARK** scheme satisfies **COMPLETENESS**.

(Note that the liveness of the sending ledger is not used as an assumption: the certified statement is assumed to be already true.)

Proof. Consider the event that an adversary $\mathcal{A}$ violates $(\mathbb{A}, \delta)\text{-BRIDGELIVENESS}$: it outputs a tuple $(P, Q, r, r', \text{rot}, x, \pi)$ where $\pi = (h, \pi')$ and such that (1) the statement x verifies wrt. relation $\mathcal{R}$ and $\mathbf{C}_i(P, r)$, which implies that x verifies wrt. $\mathbf{AChain}_i(\mathbf{L}_i(P, r))$ and relation $\mathcal{R}'$; and (2) proof π was output in round r by **Certify**;¹⁴ and (3) the correct rot is used to retrieve the mem from $\mathbf{L}_j(Q, r')$ but the proof does not verify. We show that the probability of this event is small.

G₀: This is the $(\mathbb{A}, \delta)\text{-BRIDGELIVENESS}$ game:

$$\Pr[(\mathbb{A}, \delta)\text{-BRIDGELIVENESS}] - \Pr[G_0] = 0.$$

G₁ This game aborts if any transaction added to any ledger is invalid. By perfect **LEDGERVALIDITY** we have that

$$\Pr[G_0] - \Pr[G_1] = 0.$$

G₂: This game aborts if the contract memory based on ledger $\mathbf{L}_j(Q, r')$ used in round $r' \geq r + \delta$ does not contain the bridge header value h provided in the proof $\pi = (h, \pi')$. There is at least one honest party in every round, and honest parties submit headers in all rounds. Thus an honestly computed header is submitted in round r , and by the (extended) ledger rules, remains valid until $r' \geq r + \delta \geq r + \delta_j$. A direct reduction to $(\mathbb{A}_j, \delta_j)\text{-LEDGERLIVENESS}$ of $\Pi\text{-Ledger}_j$ shows that

$$\Pr[G_1] - \Pr[G_2] \leq \text{Adv}_{\Pi\text{-Ledger}_j}^{(\mathbb{A}_j, \delta_j)\text{-LEDGERLIVENESS}}.$$

Note that since $\mathbb{A} \Rightarrow \mathbb{A}_j$, and adversary $\mathcal{A}$ respects $\mathbb{A}$, the reduction respects $\mathbb{A}_j$.

G₃: This game aborts if the proof π output by **Certify** does not verify under **Verify**. On $\pi = (h, \pi')$, **Verify** invokes **SNARK.Verify** with statement (x, h) and proof π' . Since relation $\mathcal{R}$ is checked to hold, the header value h and proof π' is correctly computed within **Certify**, and the honestly generated rot (and thus crs) is used, a reduction to **COMPLETENESS** of the **SNARK**, faithfully running the UC execution encompassing the adversary and the protocol, shows that

$$\Pr[G_2] - \Pr[G_3] \leq \text{Adv}_{\text{SNARK}}^{\text{COMPLETENESS}}.$$

¹⁴Note that a **Certify** output of $\pi = \perp$ can also be used to violate liveness.

Note that at this point both the checks that $h \in \text{mem.hs}$ and that **SNARK.Verify** verifies pass and we have now arrived at a game G_3 where

$$\Pr[G_3] = 0.$$

The theorem follows. $\square$

Remark. We note that the above reductions preserve the adaptivity of the adversary wrt. corruptions. That is, for an adaptively sound (resp. live) bridge we would need adaptive versions of the assumptions in the theorem statements.

Remark. We note that the **Verify** interface could also be directly implemented via direct observation, avoiding the need for a **SNARK**. Formally, this requires modifying the syntax of the **getVerData** algorithm so that it takes the sending chain as extra input. Using this, **Verify** can then directly verify statements about the sending chain. Moreover, if only specific types of information about the sending chain need to be verified, the proof system can be implemented in more efficient ways (e.g., using Merkle inclusion proofs).

B. Consensus-certificate bridges

From a high-level perspective, a number of bridge constructions presented herein rely on *consensus certificates for the headers* of the sending ledger. These are certificates that guarantee that honest parties agree (have consensus) on headers for a prefix of their chain. By virtue of the collision resistance of headers, these certificates also guarantee that parties agree on a prefix of local ledgers. Many ledger protocols *already* come equipped with such consensus certificates. For example, iterated-BFT-type protocols typically prepare quorum certificates on the tip of the chain. In this section, we formalize what it means for a ledger to offer secure consensus certificates. We then show how to use these to construct bridges.

We start by defining auxiliary algorithms associated with generating and verifying consensus certificates.

epoch(r) $\rightarrow e$: This is a deterministic algorithm that returns an epoch number for a given round number. For concreteness, we assume this function takes the form $\text{epoch}(r) := \lfloor r/R \rfloor$ for some integer $R \geq 1$. Thus, the smallest r for which $\text{epoch}(r) = e$ is eR , while the largest such r is $(e+1)R - 1$. We call this contiguous sequence of adjacent rounds an epoch.

conCert(C) $\rightarrow (\ell, h, \text{cert})$: This is a deterministic algorithm that, given a chain state C , returns the length ℓ of an agreed prefix, header information h for that prefix, and a consensus certificate cert .

conCertVer($(h, \text{cert}), (h', \text{cert}')$) $\rightarrow b$: This is a deterministic algorithm that, given two (presumably consecutive) pairs of headers and certificates, returns a decision indicating whether this link in the certificate chain is valid or not.

Note. We note that the certificates output by **conCert** are distinct from the proofs output by **Certify**: the former are used to guarantee consensus over a prefix of any party's ledger,

Finalization 23: Consensus Certificate Security

A-SECURECONCERT(P, r, certs)

```

1: check  $\mathbb{A}(r)$ 
2: check  $P \notin C[r]$ 
3:  $(\ell, h, \text{cert}) \leftarrow (1, \varepsilon, \text{getGenesis}(C(P, 1)))$ 
4: for  $(\ell', h', \text{cert}') \in \text{certs}$  do
5:   check  $\text{conCertVer}((h, \text{cert}), (h', \text{cert}'))$ 
6:    $(\ell, h, \text{cert}) \leftarrow (\ell', h', \text{cert}')$ 
7:  $h_P \leftarrow \text{Header}(\text{prefix}(C(P, r), \ell))$ 
8: check  $h_P \neq \perp$ 
9: return  $\neg(h_P = h)$ 

```

A-AVAILABLECONCERT(P, r)

```

10: check  $\mathbb{A}(r)$ 
11: check  $P \notin C[r]$ 
12:  $(h, \text{cert}) \leftarrow (\varepsilon, \text{getGenesis}(1, C(P, 1)))$ 
13: for  $s \in [r]$  do
14:    $(\_, h', \text{cert}') \leftarrow \text{conCert}(C(P, s))$ 
15:   if  $(h', \text{cert}') \neq (h, \text{cert})$  then
16:     if  $\neg \text{conCertVer}((h, \text{cert}), (h', \text{cert}'))$  then return 1
17:    $(h, \text{cert}) \leftarrow (h', \text{cert}')$ 
17: return 0

```

whereas the latter are used to prove specific properties about the prefix. In other words, conCert forms part of the bridge infrastructure, while Certify is part of the bridge's API.¹⁵

The intuitive security property expected from a secure consensus certificate is that if a chain of certificates starting from the genesis block verifies, then all other honest parties have the same chain header for the prefix up to length ℓ . This property is formalized in Finalization 23. We note that in the presence of secure checkpoints [31] the chain of certificates only needs to be verified until the latest checkpoint (as opposed to the genesis block).

Compatibility. In order to simplify the exposition, we assume that the headers in cert 's are *compatible* with a header function Header and the agreed-prefix function AChain as defined in Section III-A. That is, for any valid C if $(\ell, h, \text{cert}) = \text{conCert}(C)$, with $\text{cert} \neq \perp$, then $h = \text{Header}(C, \text{AChain}(C))$ and $|\text{AChain}(C)| = \ell$. This in particular means that the output headers are collision-resistant. It also allows us to define $\mathcal{R}_{zk}$ as in other bridges. Note that a secure consensus certificate for the chain C compatible with a collision-resistant header for the block length specified by the certificate implies that $\text{AChain}(C)$ is indeed a secure agreed prefix.

If the sending ledger already has secure consensus certificates, they can be used to build a secure bridge: a group of bridge operators $\mathbf{P} \subseteq \mathbf{P}_i$ observe the sending chain and post its consensus certificates to the receiving ledger (thus, $\mathbf{P} \subseteq \mathbf{P}_j$ as well). The receiving ledger's contract verifies the certificates and appends the header to the contract memory. Bridge soundness follows from the security of consensus certificates. Bridge liveness relies on the availability of the consensus certificates and on the minimal trust assumption that there is at least *some* honest party active in the bridge committee. This bridge is shown in Bridge Scheme 3. The Certify , getVerData , and Verify algorithms as well as the verify method of this bridge are implemented analogously to other bridges and thus omitted.

Theorem 6 (ConCert Bridge Soundness). *For any given assumptions $\mathbb{A}$ and $\mathbb{A}_i$ the consensus-certificate bridge (pro-*

tocol II-Bridge wrt. Bridge Scheme 3) from a sending ledger II-Ledger_i to a receiving ledger II-Ledger_j provides A-BRIDGE SOUNDNESS if:

- Both ledgers satisfy perfect LEDGERVALIDITY and II-Ledger_i satisfies perfect GENESISCORRECTNESS;
- $\mathbb{A} \Rightarrow \mathbb{A}_i$ and conCert_i satisfies $\mathbb{A}_i$ -SECURECONCERT;
- Scheme SNARK satisfies KNOWLEDGE SOUNDNESS and Header_i satisfies COLLISIONRESISTANCE.

Proof sketch. The proof of this theorem proceeds analogously to those of the direct observation bridge, thanks in part to our compatibility requirement on the headers:

G₀: We start with the BRIDGE SOUNDNESS game:

$$\Pr[\text{BRIDGE SOUNDNESS}] - \Pr[G_0] = 0.$$

G₁ This game aborts if any transaction added to any ledger is invalid or if the genesis block returned by $\text{getGenesis}(C_i(P, r))$ is incorrect for any party P or round r . By perfect LEDGERVALIDITY and GENESISCORRECTNESS we have that

$$\Pr[G_0] - \Pr[G_1] = 0.$$

G₂ This game aborts if a chain of certificates verifies but there is some honest party whose header at length ℓ is valid ($\neq \perp$) but different to the header provided in the final header in the chain of certificates. The probability of this event can be upper-bounded by

$$\Pr[G_1] - \Pr[G_2] \leq \text{Adv}_{\text{II-Ledger}_i}^{\mathbb{A}_i\text{-SECURECONCERT}}.$$

Games G₃–G₇: These games proceed analogously to those in the soundness proof of the direct-observation bridge by first extracting a chain satisfying the zero-knowledge relation via the KNOWLEDGE SOUNDNESS property, then showing that this extracted chain matches that used to compute headers via the COLLISIONRESISTANCE of headers. Note that we do not invoke the AGREEDPREFIX property of the sending ledger to show that the (agreed prefix) ledger used in the bridge transaction matches that of the parties P' of the winning condition of soundness as this is already implied by $\mathbb{A}_i$ -SECURECONCERT and COLLISIONRESISTANCE. Thus G₅ is lossless.

¹⁵Metaphorically, Certify corresponds to the manufacturing of vehicles crossing the bridge.

Bridge Scheme 3: Consensus Certificate

```

Setup( $\lambda$ )
1: return SNARK.Setup( $\lambda$ )
BKGen(pp) when  $r = r_0$ 
2: return ( $\varepsilon, \varepsilon$ )
RoT(pp, bvk,  $C_i, C_j$ ) when  $r = r_0 + 1$ 
3:  $G_i \leftarrow \text{getGenesis}_i(C_i)$ 
4:  $\text{rot} \leftarrow (\text{crs}, G_i)$ 
5: return ( $\text{rot}, \varepsilon$ )
Br( $r, \mathbf{m}, C_i, C_j; \text{bst}$ ) when  $r = r_0 + 2$ 
6:  $\text{rot} \leftarrow \text{bst.rot}$ 
7:  $\text{code} \leftarrow \text{CCBridge}[\text{rot}]$ 
8:  $\text{cID} \leftarrow \text{Hash}(\text{code})$ 
9:  $\text{bst.cID} \leftarrow \text{cID}$ 
10:  $\text{btX}_j \leftarrow (\text{cID}, \text{install}, \text{code})$ 
11: return ( $\perp, \text{btX}_j, \perp; \text{bst}$ )

Br( $r, \mathbf{m}, C_i, C_j; \text{bst}$ ) when  $r \geq r_0 + 3$ 
12: ( $\_, \mathbf{h}, \text{cert}$ )  $\leftarrow \text{conCert}_i(C_i)$ 
13: if ( $\mathbf{h}, \text{cert}$ )  $\neq \text{head}(\text{certs})$  then
14:    $\text{certs} \leftarrow \text{certs} : (\mathbf{h}, \text{cert})$ 
15: return ok
Br( $r, \mathbf{m}, C_i, C_j; \text{bst}$ ) when  $r \geq r_0 + 3$  and  $r \equiv 0 \pmod{R}$ 
16:  $\text{cID} \leftarrow \text{bst.cID}$ 
17:  $\text{btX}_j \leftarrow (\text{cID}, \text{update}, \text{certs})$ 
18: return ( $\perp, \text{btX}_j, \perp; \text{bst}$ )
CCBridge[rot].update(certs)
19: ( $\mathbf{h}, \text{cert}$ )  $\leftarrow (\text{head}(\text{mem.hs}), \text{mem.cert})$ 
20: if  $\text{cert} = \perp$  then
21:   ( $\text{crs}, G_i$ )  $\leftarrow \text{parse}(\text{rot})$ 
22:   ( $\mathbf{h}, \text{cert}$ )  $\leftarrow (\varepsilon, G_i)$ 
23: for ( $\mathbf{h}', \text{cert}'$ )  $\in \text{certs}$  do
24:   check  $\text{conCertVer}_i((\mathbf{h}, \text{cert}), (\mathbf{h}', \text{cert}'))$ 
25:    $\text{mem.hs} \leftarrow \text{mem.hs} : \mathbf{h}'$ 
26:    $\text{mem.cert} \leftarrow \text{cert}'$ 
27: return ok

```

This game puts us in a position to argue that the chain used in header calculation by the bridge parties using `conCert` matches that of any honest party up to the length ℓ as specified by `conCert`. From this point onwards, the rest of the proof proceeds analogously to that of the FC bridge. $\square$

Theorem 7 (ConCert Bridge Liveness). *For any given assumptions $\mathbb{A}$, $\mathbb{A}_i$, and $\mathbb{A}_j$ the consensus-certificate bridge (protocol Π -Bridge wrt. Bridge Scheme 3) from a sending ledger $\Pi\text{-Ledger}_i$ to a receiving ledger $\Pi\text{-Ledger}_j$ provides $(\mathbb{A}, \delta)$ -BRIDGELIVENESS if:*

- Both ledgers satisfy perfect LEDGERVALIDITY;
- Under $\mathbb{A}$, there is at least one honest bridge operator (existential honesty):

$$\forall r. \mathbb{A}(r) \Rightarrow |\mathbf{P} \cap \mathbf{C}[r]| < |\mathbf{P}|;$$

- $\mathbb{A} \Rightarrow \mathbb{A}_i$ and consensus-certificate algorithm `conCerti` satisfies $\mathbb{A}_i$ -AVAILABLECONCERT;
- $\mathbb{A} \Rightarrow \mathbb{A}_j$ and ledger protocol $\Pi\text{-Ledger}_j$ satisfies $(\mathbb{A}_j, \delta_j)$ -LEDGERLIVENESS for some $\delta_j \leq \delta$;
- Scheme SNARK satisfies COMPLETENESS.

Proof sketch. In order to prove liveness, we introduce an extra game hop:

G₀: This is the $(\mathbb{A}, \delta)$ -BRIDGELIVENESS game:

$$\Pr[(\mathbb{A}, \delta)\text{-BRIDGELIVENESS}] - \Pr[G_0] = 0.$$

G₁ This game aborts if any transaction added to any ledger is invalid. By perfect LEDGERVALIDITY we have that

$$\Pr[G_0] - \Pr[G_1] = 0.$$

G_{2.1}: The game aborts if there exists an (epoch) $e \geq 1$ such that all headers of the bridge transactions submitted before round eR are included in `mem.hs` but no bridge

transaction submitted in round eR is valid on $\Pi\text{-Ledger}_j$ for the full interval $[eR, eR + \delta_j]$ (unless already included in the ledger). Thus, this game aborts only if newly submitted certificates of epoch e are invalid on ledger j with respect to `conCertVer` and the contract's memory. The probability of this event can be upper bounded by the $\mathbb{A}_i$ -AVAILABLECONCERT property of the sending chain:

$$\Pr[G_1] - \Pr[G_{2.1}] \leq \text{Adv}_{\Pi\text{-Ledger}_i}^{\mathbb{A}_i\text{-AVAILABLECONCERT}}.$$

G_{2.2} This game aborts if there exists an epoch $e \geq 1$ such that no bridge transactions submitted in round eR are valid until round $eR + \delta$ (or are already included in the ledger) but the headers of their certificates are not included in `mem.hs` by round $eR + \delta$. A direct reduction to $(\mathbb{A}_j, \delta_j)$ -LEDGERLIVENESS of $\Pi\text{-Ledger}_j$ shows that

$$\Pr[G_{2.1}] - \Pr[G_{2.2}] \leq \text{Adv}_{\Pi\text{-Ledger}_j}^{(\mathbb{A}_j, \delta_j)\text{-LEDGERLIVENESS}}.$$

Note that since $\mathbb{A} \Rightarrow \mathbb{A}_j$, and adversary $\mathcal{A}$ respects $\mathbb{A}$, the reduction respects $\mathbb{A}_j$.

G_{2.3} This game aborts if the contract memory based on ledger $\mathbb{L}_j(Q, r')$ used in round $r' \geq r + \delta$ does not contain the bridge header value $\mathbf{h}$ provided in the proof $\pi = (\mathbf{h}, \pi')$ produced in round r .

This follows from existential honesty which implies that in every epoch at least one bridge transaction is submitted and from an inductive argument over the events of $G_{2.1}$ and $G_{2.2}$ that imply that this transaction will be included in the ledger. The base condition of the argument is that the genesis block is always a valid consensus certificate. The abort conditions of $G_{2.1}$ and $G_{2.2}$ imply that the headers of the bridge transaction of epoch e are added to `mem.hs` if the headers of all previous epochs have been

added (for epoch 1 this is the empty header of the genesis block). Thus

$$\Pr[G_{2.2}] - \Pr[G_{2.3}] = 0.$$

G_{2.4}: This game aborts if a header computed by the certifying party is not included in the header mem.hs upon verification. This contradicts the perfect compatibility of conCert with AChain.

$$\Pr[G_{2.3}] - \Pr[G_{2.4}] = 0.$$

G₃: This game is analogous to those in the proof of Theorem 5, relying on the COMPLETENESS of the SNARK scheme. $\square$

C. Fixed-committee bridge

A second type of bridge design is one where each member of a *fixed* group of bridge operators $\mathbf{P}$ observe the state of the sending ledger (thus, $\mathbf{P} \subseteq \mathbf{P}_i$) and submit a signature on the hash of the agreed prefix of the sending ledger to the receiving ledger (thus, $\mathbf{P} \subseteq \mathbf{P}_j$ as well). A contract deployed on the receiving ledger collects the submitted signatures. When a threshold number of signatures is reached, the hash value is declared as accepted. This is a simple bridge design and has been used in the Ronin [43] and Wormhole bridge [50]. However, fixing the committee makes it brittle and vulnerable to committee attacks [43].

Nevertheless, and due to its popularity, we formalize this bridge in this section. The public parameters include a CRS for a proof system, and bridge maintenance keys are a pair of standard signature keys (see Bridge Scheme 4). The root of trust includes the CRS and the list of all bridge maintenance verification keys.¹⁶

The Br algorithm in round $r_0 + 2$ prepares and deploys a contract FCBridge[rot] on the receiving ledger, which has the root of trust data rot hardwired. This is similar to the direct-observation bridge except that FCBridge is implemented differently. From rounds $r_0 + 3$ onward, the Br algorithm computes a digital signature σ on the header of the agreed prefix and its verification key, and prepares it as an argument to be submitted to the update method of the contract. The update method checks the validity of σ with respect to vk on message (h, vk). If ok, it adds (h, vk) to its memory mem. It then checks if greater than a ρ fraction of bridge operator keys appear in memory for a specific h. If so, the contract finalizes h in its memory.

The second method of FCBridge, verify, is implemented analogously to that in the direct-observation bridge. The Certify algorithm also operates similarly to that in the direct-observation bridge with respect to the same zk-relation $\mathcal{R}_{zk}$. Algorithm getVerData extracts the memory of FCBridge. Algorithm Verify retrieves the CRS from mem, checks that its input h is in its memory, and applies the verification procedure of the SNARK. As these are similar to those in the direct observation bridge, we omit their formal descriptions.

¹⁶Alternatively, one can include a Merkle root of the verification keys, and later include proofs of inclusion wrt. this root with the signatures.

We next prove that the fixed-committee bridge is sound and live as defined in Section III-B as long as the sending and receiving ledgers are secure (persistent and live), at most a ρ fraction of bridge parties are corrupt. For this, the proof system needs to be knowledge sound and complete, and the headers need to be collision-resistant.

Theorem 8 (Fixed-Committee Bridge Soundness). *For any given assumptions $\mathbb{A}_i$ and $\mathbb{A}$ and any ρ the ρ -fixed-committee bridge (protocol II-Bridge wrt. Bridge Scheme 4) from a sending ledger II-Ledger_i to a receiving ledger II-Ledger_j satisfies $(\mathbb{A}, \delta)$ -BRIDGESOUNDNESS if:*

- Both ledgers satisfy perfect LEDGERVALIDITY;
- $\mathbb{A} \Rightarrow \mathbb{A}_i$ and ledger protocol II-Ledger_i satisfies $\mathbb{A}_i$ -AGREEDPREFIX;
- Under $\mathbb{A}$, there are at most $\rho \cdot |\mathbf{P}|$ corrupt bridge operators. In other words, $\mathbb{A} \Rightarrow \mathbb{A}_1[\rho]$ defined in Algorithm 15;
- Scheme SNARK satisfies KNOWLEDGESOUNDNESS, DS satisfies UF-CMA, and Header_i satisfies COLLISIONRESISTANCE.

Proof sketch. The proof of this theorem mostly follows that of Theorem 4. The main difference is that we use the unforgeability of the underlying digital signature scheme and the fact that at most a ρ fraction of the parties are corrupt to argue that at least one honest party must have signed an accepted header. In slightly more detail, we proceed along the following games, where game G_{1.5} is the new step in this proof.

G₀ and G₁: As in the proof of Theorem 4, we start with BRIDGESOUNDNESS game and transition, using LEDGERVALIDITY to a setting where all transactions added to the ledgers are valid.

G₂: This game aborts if a header h is added to mem.hs but no honest party signed h. A reduction to the multi-user UF-CMA security shows that

$$\Pr[G_1] - \Pr[G_2] \leq \text{Adv}_{\text{DS}}^{n\text{-UF-CMA}} \leq n \cdot \text{Adv}_{\text{DS}}^{\text{UF-CMA}}.$$

where $n := |\mathbf{P}|$. To see this, note that h is added to mem.hs if there are strictly greater than $\rho \cdot |\mathbf{P}|$ pairs (h, vk) in mem.hvk such that each (h, vk) came with a valid signature σ wrt. vk $\in$ bvk. If the said event arises, then at least one of the keys is honest and, by assumption, did not sign h yet the adversary came up with a valid signature σ for it. The reduction, who knows the corruption status and can verify signatures, can find this party and output the signature as a forgery. Note that a standard guessing argument shows that the advantage in the multi-user unforgeability game is upper bounded by n times advantage in the single user game, even under adaptive corruptions.

G₃–G₇: These games proceed analogously to those in the soundness proof of the direct-observation bridge by first extracting a chain satisfying the zero-knowledge relation via the KNOWLEDGESOUNDNESS property, then showing that this extracted chain matches that used in computing

Bridge Scheme 4: ρ -Fixed Committee

```

Setup( $\lambda$ )
1: return SNARK.Setup( $\lambda$ )
BKGen(pp) when  $r = r_0$ 
2: return DS.Gen()
RoT(pp, bvk,  $C_i, C_j$ ) when  $r = r_0 + 1$ 
3:  $\text{rot} \leftarrow (\text{pp}, \text{bvk})$ 
4: return (rot,  $\varepsilon$ )
Br( $r, \mathbf{m}, C_i, C_j; \text{bst}$ ) when  $r = r_0 + 2$ 
5:  $\text{rot} \leftarrow \text{bst.rot}$ 
6:  $\text{code} \leftarrow \text{FCBridge}[\text{rot}]$ 
7:  $\text{clD} \leftarrow \text{Hash}(\text{code})$ 
8:  $\text{bst.clD} \leftarrow \text{clD}$ 
9:  $\text{btX}_j \leftarrow (\text{clD}, \text{install}, \text{code})$ 
10: return ( $\perp, \text{btX}_j, \perp; \text{bst}$ )

Br( $r, \mathbf{m}, C_i, C_j; \text{bst}$ ) when  $r \geq r_0 + 3$ 
11:  $\text{clD} \leftarrow \text{bst.clD}$ 
12:  $\mathbf{h} \leftarrow \text{Header}_i(C_i, \text{apl}_i(C_i))$ 
13:  $(\text{sk}, \text{vk}) \leftarrow (\text{bst.bvk}, \text{bst.bsk})$ 
14:  $\sigma \leftarrow \text{DS.Sig}(\text{sk}, (\mathbf{h}, \text{vk}))$ 
15:  $\text{btX}_j \leftarrow (\text{clD}, \text{update}, (\mathbf{h}, \text{vk}, \sigma))$ 
16: return ( $\perp, \text{btX}_j, \perp; \text{bst}$ )
FCBridge[rot].update( $\mathbf{h}, \text{vk}, \sigma$ )
17: (pp, bvk)  $\leftarrow$  parse(rot)
18: check  $\text{vk} \in \text{bvk}$ 
19: check  $\text{DS.Ver}(\text{vk}, (\mathbf{h}, \text{vk}), \sigma)$ 
20:  $\text{mem.hvk} \leftarrow \text{mem.hvk} : (\mathbf{h}, \text{vk})$ 
21: if  $|\{(\mathbf{h}, \text{vk}) : (\mathbf{h}, \text{vk}) \in \text{mem.hvk}\}| > \rho \cdot |\mathbf{P}|$  then
22:    $\text{mem.hs} \leftarrow \text{mem.hs} : \mathbf{h}$ 
23:    $\text{mem.hvk} \leftarrow []$ 
24: return ok

```

headers via the COLLISIONRESISTANCE of headers, and finally invoking the AGREEDPREFIX property of the sending ledger to show that the (agreed prefix) ledger used in the bridge transaction matches that of the party which is used in the winning condition of soundness.

The theorem follows, with an additional term in the final bound due to the extra game hop above. $\square$

Theorem 9 (Fixed-Committee Bridge Liveness). *For any given assumptions $\mathbb{A}_j$ and $\mathbb{A}$ and any ρ , the ρ -fixed-committee bridge (protocol Π -Bridge wrt. Bridge Scheme 4) from a sending ledger Π -Ledger $_i$ to a receiving ledger Π -Ledger $_j$ satisfies $(\mathbb{A}, \delta)$ -BRIDGELIVENESS if:*

- Both ledgers satisfy perfect LEDGERVALIDITY;
- $\mathbb{A} \Rightarrow \mathbb{A}_j$ and ledger protocol Π -Ledger $_j$ satisfies $(\mathbb{A}_j, \delta_j)$ -LEDGERLIVENESS for some $\delta_j \leq \delta$;
- Under $\mathbb{A}$, there are greater than $\rho \cdot |\mathbf{P}|$ honest bridge operators, i.e.,

$$\forall r. \mathbb{A}(r) \Rightarrow |\mathbf{P} \cap \mathbf{H}[r]| > \rho \cdot |\mathbf{P}|,$$

where $\mathbf{H}[r] := \mathbf{P} \setminus \mathbf{C}[r]$.

- SNARK satisfies COMPLETENESS and DS is CORRECT.

Proof sketch. By our assumption that there are at least $\rho \cdot |\mathbf{P}|$ honest parties in all rounds who submit a transaction that is then guaranteed to be added to the mem.

G₀ and G₁: These games are analogous to those in the proof of direct observation liveness. G₁ relies on the COMPLETENESS of the SNARK scheme.

G_{2.1}: This game aborts if one of the signatures created by an honest party does not verify. A simple reduction to a multi-user analogue of correctness game (that is breaking correctness for one out of many keys), together with a guessing argument, shows that

$$\Pr[G_1] - \Pr[G_{2.1}] \leq n \cdot \text{Adv}_{\text{DS}}^{\text{CORRECT}}.$$

We note that $\text{Adv}_{\text{DS}}^{\text{CORRECT}}$ is typically zero.

G_{2.2}: This game aborts if mem.hvk recovered from ledger Ledger $_j$ [Q, r'] used in round $r' \geq r + \delta$ does not contain at least $\rho \cdot |\mathbf{P}|$ pairs $(\mathbf{h}, \text{vk})$ for the agreed header of round r .

By our assumptions, there are at least $\rho \cdot |\mathbf{P}|$ honest parties in every round, and honest parties submit $(\mathbf{h}, \text{vk}, \sigma)$ in every round. Since the header is honestly computed and submitted in some round r , and remains valid until $r' \geq r + \delta \geq r + \delta_j$, a reduction to $(\mathbb{A}_j, \delta_j)$ -LEDGERLIVENESS shows that

$$\Pr[G_{2.1}] - \Pr[G_{2.2}] \leq \text{Adv}_{\Pi\text{-Ledger}_j}^{(\mathbb{A}_j, \delta_j)\text{-LEDGERLIVENESS}}.$$

Once again, since $\mathbb{A} \Rightarrow \mathbb{A}_j$, and the adversary respects $\mathbb{A}$ in an execution, the reduction respects $\mathbb{A}_j$.

G_{2.2}: This game aborts if the contract memory mem.hs based on ledger $\mathbb{L}_j(Q, r')$ used in round $r' \geq r + \delta$ does not contain the bridge header value $\mathbf{h}$ provided in the proof $\pi = (\mathbf{h}, \pi')$. Header $\mathbf{h}$ is the header of round r , and by the aborts G_{2.1}, mem.hvk must contain at least $\rho \cdot |\mathbf{P}|$ pairs $(\mathbf{h}, \text{vk})$. Thus by the code of FCBridge[rot].update, $\mathbf{h}$ is added to hs.

Since

G₃: This game is analogous to those in the proof of Theorem 5, relying on the completeness of the SNARK scheme.

We note that as this point both the checks that $\mathbf{h} \in \text{mem.hs}$ and that SNARK.Verify verifies pass and we have now arrived at a game G₃ where

$$\Pr[G_3] = 0.$$

The theorem follows. $\square$

Remark. To achieve both liveness and soundness we must have, from soundness, that $|\mathbf{P} \cap \mathbf{C}[r]| \leq \rho \cdot |\mathbf{P}|$ and from liveness that $|\mathbf{P} \cap \mathbf{C}[r]| < |\mathbf{P}| - \rho \cdot |\mathbf{P}| = (1 - \rho) \cdot |\mathbf{P}|$. These imply that $|\mathbf{P} \cap \mathbf{C}[r]| < |\mathbf{P}|/2$. In other words, we would need $\rho < 1/2$ to achieve both liveness and soundness.

An analogous bridge can be implemented using a threshold signature scheme with aggregated signatures to reduce on-chain computation and memory footprint. That is, an aggregator collects a threshold number of individual signatures, aggregates them, and submits to a contract for verification. Our ATS bridge scheme follows this idea, but additionally derives the required trust assumptions from the sending chain.

APPENDIX I SUPPLEMENTARY ATS MATERIAL

A. The ATS ideal functionality

To facilitate our model checking of the ATS bridge, which relies on *ideally secure* building blocks, we give a definition of an ideal functionality for ATS schemes. To the best of our knowledge, this functionality is novel, and although complex, had been thoroughly checked for flaws. We give an outline of this functionality below, and describe how its design prevents (ideal-world) adversaries from breaking its correctness and unforgeability properties. This functionality is potentially of interest in other contexts involving threshold signatures.

Functionality $\mathcal{F}\text{-ATS}$ comprises interfaces that match the algorithms for an ATS scheme, except for setup, which compatible with UC approaches, is provided by independent idealized building blocks formulated via setup assumptions. We give an overview below and present its full specification in Functionality 24.

Interfaces P.Gen , P.Sign , and P.Ver are defined similarly to a standard signature ideal functionality $\mathcal{F}\text{-Sig}$. In particular, individual verification keys that are generated (via the simulator) are sanitized so that there are neither collisions nor pre-forgeries wrt. them. The generated signatures are also sanitized so that they will always verify. Verification ensures that the output bit is overwritten by 0 if the signer is honest and the input signature was not generated through the P.Sign interface.

Interface P.AKey aggregates a given list of keys (possibly with repetitions). For correctness, it ensures that, independently of what the simulator outputs, honestly generated individual keys always successfully aggregate. Interface P.ACheck checks if an aggregate verification key is correct essentially through recomputation. These two interfaces keep track of lists of verification keys and their aggregates for consistency.

Interface P.ASign returns an aggregate signature τ . It proceeds by computing an avk associated with the input list of verification keys, returning $\perp$ if the simulator rejects the vk list. It then checks if there are enough verifying σ in $\mathbf{S}$, returning $\perp$ if this is not the case. Verification bits are computed as usual via the simulator. Finally, a sanitized aggregate signature is returned, which is ensured to pass aggregate verification.

Interface P.AVer is designed so that a list of individual verification keys vk^* corresponding to a given avk is computed *via the simulator*. The interface counts, *with multiplicity*, if the total number of non-honest keys that have signed the message, corrupt keys, and other keys that appear in vk^* is less than the threshold t . In this case, the interface overwrites the simulator's bit with 0. This ensures unforgeability.

In this functionality, we keep track of the maliciously generated keys in a set CVK , which P.Gen interface avoids. This allows the functionality to distinguish honest and malicious keys, which is then used to define correctness of signature aggregation.

A quick glance at the functionality reveals that there are many details, spanning over 100 lines of code. This, in turn, gives rise to the possibility of bugs. We have designed the functionality in tandem with model-checking, where we have a variety of trace and state properties. Indeed, our initial attempts at formulating $\mathcal{F}\text{-ATS}$ had subtle bugs, some of which were arguably confusing. For instance, $\mathcal{F}\text{-ATS}$ maintains two lists AVK and VK that intuitively keep a correspondence between lists of individual keys and aggregate verification keys. Due to the simulators resumption capabilities, the presence of maliciously generated keys, and the fact that the lists are populated under different interfaces, bugs were present. We used Quint's output trace vectors, to debug the functionality. For example, this led us to keep track of malicious keys in a list CVK and avoid them. In its final formulation, Quint reports that $\mathcal{F}\text{-ATS}$ passes a number of properties, including correctness and unforgeability for both individual and aggregate signatures; see Section V.

B. ATS correctness

In Game 5, we extend Game 1 so that in addition to unforgeability, we also define three correctness properties. We require that 1) any set of keys that are aggregated, even if some of the keys are not honestly generated, pass ACheck (AVKCORRECTNESS); 2) individual signatures pass (individual) signature verification (SIGCORRECTNESS); and 3) ATS aggregation of any set of $t + 1$ individual vk /signatures pairs that verify with some vk , verifies against wrt verification procedure ($t\text{-ASIGCORRECTNESS}$) subject to the condition that vk in the provided pairs appear in a unique position in vk . Note that this still allows for duplicate entries in vk to account e.g. for weighted votes, but ensures that the provided signatures do not exceed their specified quota. Thus, "ad-hocness" refers to the fact that this set can be chosen in an ad-hoc manner after the signatures are generated. We note that this property must hold even with respect to malicious keys. In particular, and surprisingly, the correctness notion defined in the original work [28] is *not* sufficient for the liveness of our ATS bridge.

As shown in [28, Section 5], an ATS based on a multi-signature scheme from [8] (which also includes Merkle proofs of inclusion in a Merkle commitment to a set of verification keys) achieves $t\text{-UF-CMA}$ security under adaptive corruptions. Our adaptive notions of correctness (where some malicious keys may be included in the aggregate verification key) are also satisfied. Indeed, AVKCORRECTNESS relies on AKey being deterministic, and SIGCORRECTNESS on the perfect correctness of the underlying multi-signature scheme and Merkle proofs, which also hold for maliciously generated keys. The $t\text{-ASIGCORRECTNESS}$ property is met because of the soundness of Merkle trees and perfect correctness of the underlying multi-signature scheme.

Functionality 24: $\mathcal{F}$ -ATS[P, t]

```

Init()
1:  $VK[*] \leftarrow Ver[*] \leftarrow AVK[*] \leftarrow AVer[*] \leftarrow \square$ 
2:  $VKs[*] \leftarrow Signers[*] \leftarrow CVK[*] \leftarrow \emptyset$ 
P.Gen()
3:  $vk \leftarrow Sim(P.Gen)$ 
4: if  $VK[P] \neq \square$  then return  $VK[P]$ 
5:  $vk \leftarrow San[Clean_{vk}](vk, VK, CVK, Ver)$ 
6:  $VK[P] \leftarrow vk$ 
7: return  $vk$ 
P.AKey(vk)
8:  $avk \leftarrow Sim(P.AKey, vk)$ 
9: if  $AVK[vk] \neq \square$  then return  $AVK[P]$ 
10:  $avk \leftarrow San[Clean_{avk}](avk, vk, AVK, AVer, VK, C)$ 
11:  $AVK[vk] \leftarrow avk$ 
12: if  $avk \neq \perp$  then
13:    $VKs[avk] \leftarrow vk$ 
14: for  $vk \in vk$  with  $vk \notin VK[*]$  do
15:    $CVK \leftarrow CVK \cup \{vk\}$ 
16: return  $avk$ 
P.ACheck(avk, vk)
17:  $avk' \leftarrow Sim(P.ACheck, avk, vk)$ 
18: if  $AVK[vk] \neq \square$  then return  $(avk = AVK[P])$ 
19:  $avk' \leftarrow San[Clean_{avk}](avk', vk, AVK, AVer, VK, C)$ 
20:  $AVK[vk] \leftarrow avk'$ 
21: if  $avk \neq \perp$  then
22:    $VKs[avk'] \leftarrow vk$ 
23: for  $vk \in vk$  with  $vk \notin VK[*]$  do
24:    $CVK \leftarrow CVK \cup \{vk\}$ 
25: return  $(avk = avk')$ 

```

```

P.Sign(m)
26: if  $VK[P] = \square$  then return  $\perp$ 
27:  $\sigma \leftarrow Sim(P.Sign, m)$ 
28:  $vk \leftarrow VK[P]$ 
29:  $\sigma \leftarrow San[Clean_{\sigma}](vk, m, \sigma, Ver)$ 
30:  $Signers[m] \leftarrow Signers[m] \cup \{vk\}$ 
31:  $Ver[vk, m, \sigma] \leftarrow 1$ 
32: return  $\sigma$ 
P.Ver(vk, m,  $\sigma$ )
33:  $b \leftarrow Sim(P.Ver, vk, m, \sigma)$ 
34: if  $Ver[vk, m, \sigma] \neq \square$  then
35:   return  $Ver[vk, m, \sigma]$ 
36: if  $\exists P' \in P \setminus C. VK[P'] = vk$  then
37:    $b \leftarrow 0$ 
38:  $Ver[vk, m, \sigma] \leftarrow b$ 
39: if  $vk \notin VK[*]$  then
40:    $CVK \leftarrow CVK \cup \{vk\}$ 
41: return  $b$ 
Pred. Cleanvk(vk, VK, CVK, Ver)
42: return  $(vk \neq \perp \wedge vk \notin VK[*] \cup CVK \wedge Ver[vk, *, *] \neq 1)$ 
Pred. Cleanavk(avk, vk, AVK, AVer, VK, C)
43: if  $avk \neq \perp$  then
44:   return  $(avk \notin AVK[*] \vee AVer[avk, *, *] \neq 1)$ 
45: return  $(\exists vk \in vk. \forall P \notin C. vk \neq VK[P])$ 
Pred. Clean $\sigma$ (vk, m,  $\sigma$ , Ver)
46: return  $(\sigma \neq \perp \wedge Ver[vk, m, \sigma] \neq 0)$ 

```

C. Security of the ATS bridge

Theorem 10 (ATS Bridge Soundness). *For any assumptions $\mathbb{A}_i, \mathbb{A}'_i, \mathbb{A}_j, \mathbb{A}$ and parameters (c, ρ) , the ATS bridge (protocol Π -Bridge wrt. Bridge Scheme 1) from a sending ledger Π -Ledger _{i} to a receiving ledger Π -Ledger _{j} provides $\mathbb{A}$ -BRIDGESOUNDNESS if:*

- Both ledgers satisfy perfect LEDGERVALIDITY;
- $\mathbb{A} \Rightarrow \mathbb{A}_i$ and Π -Ledger _{i} satisfies $\mathbb{A}_i$ -AGREEDPREFIX;
- $\mathbb{A} \Rightarrow \mathbb{A}'_i$ and ledger protocol Π -Ledger _{i} satisfies $(\mathbb{A}'_i, c, \rho, 2R)$ -SECURECOMSEL;
- $\mathbb{A} \Rightarrow \mathbb{A}_j$ and ledger protocol Π -Ledger _{j} satisfies $(\mathbb{A}_j, \delta_j)$ -LEDGERLIVENESS for some δ_j such that $R \geq \Delta + \delta_j + 3$;
- Scheme ATS satisfies ρc -UF-CMA and ρc -ASIGCORRECTNESS;
- Scheme SNARK satisfies KNOWLEDGESOUNDNESS and Header _{i} satisfies COLLISIONRESISTANCE.

Proof. The proof of this theorem is analogous to that for the fixed-committee bridge modulo modifications for selection of committees and a reduction to the unforgeability of ATS. We give an overview below, highlighting the differences.

$\underline{G}_0$: This is the BRIDGESOUNDNESS game.

$\underline{G}_1$ This game aborts if any transaction added to any ledger, and in particular to the ledger $L_j(Q, r)$ or some $L_i(P', r')$ is invalid.¹⁷ By perfect LEDGERVALIDITY we have that

$$\Pr[G_0] - \Pr[G_1] = 0$$

$\underline{G}_{2.1}$: This game aborts if in some epoch, **Com** is not of size c , or not a subset of P_i , or more that ρc of its members are corrupt in some round, either during the epoch **Com** was formed, or the following one. This game is justified using the SECURECOMSEL _{i} property:

$$\Pr[G_1] - \Pr[G_{2.1}] \leq \lfloor r_{max}/R \rfloor \cdot \text{Adv}_{\Pi\text{-Ledger}_i}^{(\mathbb{A}, c, \rho)\text{-SECURECOMSEL}}.$$

Here r_{max} is an upper bound on the maximum number of rounds that the adversary runs.

$\underline{G}_{2.2}$: This game aborts if during the calculation of chain headers (line 19) for some $P, P' \in H[r^*]$ the agreed prefix $\text{AChain}_i(C_i(P', r^*))$ is not the same as $\text{AChain}_i(C_i(P^*, r^*))$. If this event arises, then the

¹⁷This step is mostly to simplify our reasoning. An invalid transaction in $L_j(Q, r)$ would already result in the adversary losing, due to Verify in line 6 failing for $mem = \perp$. Similarly, an invalid ledger $L_i(P', r')$ doesn't seem to make it any easier to prove statements about it.

Functionality 25: $\mathcal{F}$ -ATS[P, t], Continued

P.ASign(m, vk, S)

```

47: (avk, b,  $\tau$ )  $\leftarrow$  Sim(P.ASign, m, vk, S)
48: if  $b \notin \{0, 1\}^{|S|}$  then  $b \leftarrow 0^{|S|}$ 
49: if  $AVK[vk] \neq \square$  then
50:   avk  $\leftarrow$  AVK[vk]
51: else // compute avk for vk
52:   avk  $\leftarrow$  San[Cleanavk](avk, vk, AVK, AVer, VK, C)
53:   if avk  $\neq \perp$  then
54:     VKs[avk']  $\leftarrow$  vk
55:   for vk  $\in$  vk with vk  $\notin$  VK[*] do
56:     CVK  $\leftarrow$  CVK  $\cup$  {vk}
    // at this point we have an avk
57: if (avk =  $\perp$ )  $\vee$   $\neg(\forall i. \exists! j. S[i].vk = vk[j])$  then
58:   return  $\perp$ 
    // check if there are enough verifying sigs in S
59: for  $i \in [|S|]$  do
60:   (vk,  $\sigma$ )  $\leftarrow$  S[i]
61:   if Ver[vk, m,  $\sigma$ ]  $\neq \square$  then
62:     b[i]  $\leftarrow$  Ver[vk, m,  $\sigma$ ]
63:   else if  $\exists P' \in P \setminus C. VK[P'] = vk$  then
64:     b[i]  $\leftarrow$  0
65:   Ver[vk, m,  $\sigma$ ]  $\leftarrow$  b[i]
66:   if vk  $\notin$  VK[*] then
67:     CVK  $\leftarrow$  CVK  $\cup$  {vk}
68: if  $|\{i \in [|S|] : b[i] = 1\}| \leq t$  then return  $\perp$ 
    // only here we know we need a  $\tau$ 
69:  $\tau \leftarrow$  San[Clean $_{\tau}$ ](avk, m,  $\tau$ , AVer)
70: AVer[avk, m,  $\tau$ ]  $\leftarrow$  1
71: return  $\tau$ 

```

P.AVer(avk, m, τ)

```

72: (b, vk*)  $\leftarrow$  Sim(P.AVer, avk, m,  $\tau$ )
73: if AVer[avk, m,  $\tau$ ]  $\neq \square$  then
74:   return AVer[avk, m,  $\tau$ ]
75: if VKs[avk]  $\neq \square$  then
76:   vk*  $\leftarrow$  VKs[avk]
    // even if VKs[avk] is unset, AVK[vk*] may be set
    // to a value  $\neq$  avk. Sim is not allowed to pick a vk*
    // that's already taken.
77: if  $\perp \in vk^* \vee (AVK[vk^*] \neq \square \wedge AVK[vk^*] \neq$ 
    avk)  $\vee \neg$  Cleanavk(avk, vk*, AVK, AVer, VK, C)
    then
78:   AVer[avk, m,  $\tau$ ]  $\leftarrow$  0
79:   return 0
80: AVK[vk*]  $\leftarrow$  avk
81: VKs[avk]  $\leftarrow$  vk*
82: for vk  $\in$  vk* with vk  $\notin$  VK[*] do
83:   CVK  $\leftarrow$  CVK  $\cup$  {vk}
84: VKc  $\leftarrow$  {VK[P'] : P'  $\in$  C}
85: q  $\leftarrow$  0
86: for vk  $\in$  vk* do
87:   if vk  $\notin$  VK[*]  $\vee$  vk  $\in$  (Signers[m]  $\cup$  VKc) then
88:     q  $\leftarrow$  q + 1
89: if q  $\leq t$  then
90:   b  $\leftarrow$  0
91: AVer[avk, m,  $\tau$ ]  $\leftarrow$  b
92: return b
Pred. Clean $_{\tau}$ (avk, m,  $\tau$ , AVer)
93: return ( $\tau \neq \perp \wedge$  AVer[avk, m,  $\tau$ ]  $\neq$  0)

```

Game 5: ATS Correctness

AVKCORRECTNESS(vk)

```

20: avk  $\leftarrow$  ATS.AKey(vk)
21: check  $\forall i. \exists j. vk[i] = VK[j]$ 
22: return  $\neg$ ATS.ACheck(avk, vk)

```

SIGCORRECTNESS(i, m, σ)

```

23: check VK[i]  $\in$  H
24: check  $\sigma \in$  Sigs[i, m]
25: return ATS.Ver(VK[i], m,  $\sigma$ )

```

t-ASIGCORRECTNESS(m, vk, S, avk) // S a list of (vk, σ)

```

26: check  $\forall i \in |S|. \exists! j. S[i].vk = vk[j]$ 
27: check  $|\{i \in |S| : \text{ATS.Ver}(S[i].vk, m, S[i].\sigma)\}| > t$ 
28: check ATS.ACheck(avk, vk)
29:  $\tau \leftarrow$  ATS.ASig(m, vk, S)
30: return  $\neg$ ATS.AVer(avk, m,  $\tau$ )

```

$\mathbb{A}_i$ -AGREEDPREFIX property of the sending ledger would be violated:

$$\Pr[G_{2.1}] - \Pr[G_{2.2}] \leq \text{Adv}_{\Pi\text{-Ledger}_i}^{\mathbb{A}_i\text{-AGREEDPREFIX}}.$$

Note there is no factor $|P|$ loss since the agreed prefix property can be publicly checked.

G_{2.3}: This game aborts if there is no honest party who has signed a header that appears in mem.hs. Note that by the

previous transition, there are at most ρc corrupt parties in each epoch that take part in an ATS signature. Moreover, we set $t := \rho c$ in the ATS scheme. A reduction to t -UF-CMA of ATS in each epoch shows that we get that

$$\Pr[G_{2.2}] - \Pr[G_{2.3}] \leq \lfloor r_{\max}/R \rfloor \cdot \text{Adv}_{\text{ATS}}^{\rho c\text{-UF-CMA}}.$$

Here we rely on a multi-avk version of ATS where the same public parameter is used for multiple avk's in each of the $\lfloor r_{\max}/R \rfloor$ epochs, and a successful forgery can be wrt. any of the avk's.

G_{2.4}: We abort if an honest certificate fails to be created by round offset $\Delta + 2$. This directly contradicts ATS being ρc -ASIGCORRECTNESS.

$$\Pr[G_{2.3}] - \Pr[G_{2.4}] \leq \lfloor r_{\max}/R \rfloor \cdot \text{Adv}_{\text{ATS}}^{\rho c\text{-ASIGCORRECTNESS}}.$$

G_{2.5}: This game aborts if the certificate created at round offset $\Delta + 2$ does not appear on Π -Ledger_j by round $\Delta + \delta_j + 3$. Intuitively, this signifies the end of validity for the current avk, and by extension of the outgoing committee. Our assumption on committee selection only guarantees security for the first $2R$ rounds after the committee is selected, so this update is necessary for safety

as well as liveness. By the $(\mathbb{A}_j, \delta_j)$ -LEDGERLIVENESS of Π -Ledger_j we have

$$\Pr[G_{2.4}] - \Pr[G_{2.5}] \leq \text{Adv}_{\Pi\text{-Ledger}_j}^{(\mathbb{A}_j, \delta_j)\text{-LEDGERLIVENESS}}.$$

The previous game puts us in a position where it is guaranteed that some honest party signed a header. The rest of the proof can now proceed analogously to that of the FC bridge, and are omitted to avoid repetition.

They involve extracting a chain satisfying the zero-knowledge relation via the KNOWLEDGESOUNDNESS property, showing that this extracted chain matches that used in computing headers via the COLLISIONRESISTANCE of headers, and invoking the AGREEDPREFIX property of the sending ledger to show that the (agreed prefix) ledger used in the bridge transaction matches that of the party which is used in the winning condition of soundness.

G₃: This game extracts a chain C_i^* from any proof π that passes verification, where the extractor gets the coins of *both* the game and the adversary, except those used to generate the CRS. If C_i^* does not verify wrt. $\mathcal{R}_{zk}$, the game aborts. A reduction to knowledge soundness shows that the probability of this event is small. In more detail, the reduction uses the CRS oracle from the knowledge soundness game as the CRS value in the bridge soundness game. It then runs the adversary faithfully in the bridge soundness execution environment. Note that there is no need to simulate proofs as the adversary provides witnesses for statements of its choice. The abort event translates to winning the KNOWLEDGESOUNDNESS game as long as the correct rot is used:

$$\Pr[G_2] - \Pr[G_3] \leq \text{Adv}_{\text{SNARK}}^{\text{KNOWLEDGESOUNDNESS}}.$$

G₄: This game aborts if the agreed prefix of the extracted chain $\text{AChain}_i(C_i^*)$ is not equal to the chain that some honest bridge party P^* used to compute the header information in the bridge transaction in round $r^* \leq r$. Let $C_i(P^*, r^*)$ be this party's chain. Note that Verify checks the header of the proof $h^* := \text{Header}_i(C^*, \text{apl}_i(C^*))$ against the header $h = \text{Header}_i(C_i(P^*, r^*), \text{apl}_i(C_i(P^*, r^*)))$. A reduction to the COLLISIONRESISTANCE of the headers of the sending ledger shows that

$$\Pr[G_3] - \Pr[G_4] \leq \text{Adv}_{\Pi\text{-Ledger}_i}^{\text{COLLISIONRESISTANCE}}.$$

G₅: This game replaces searching over all r' in the winning condition with the specific round r^* . This winning condition is more lenient (i.e, it is implied by the full winning condition). In particular G_5 states that the adversary only wins if $\mathcal{R}(x, C_i(P', r'))$ is false for all r' , while G_6 states that the adversary wins if only $\mathcal{R}(x, C_i(P', r^*))$ is false. Thus,

$$\Pr[G_4] - \Pr[G_5] \leq 0.$$

G₇: This game runs BRIDGESOUNDNESS, but now uses C_i^* in $\mathcal{R}$ instead of $C_i(P', r^*)$. At this point of the game $\text{AChain}_i(C_i^*) = \text{AChain}_i(C_i(P^*, r^*)) =$

$\text{AChain}_i(C_i(P', r^*))$. Moreover, recall that relation $\mathcal{R}$ only depends on the agreed prefix of the input chain. Hence,

$$\Pr[G_5] - \Pr[G_6] = 0.$$

Now, observe that G_5 is not winnable as C_i^* always verifies wrt. relation $\mathcal{R}_{zk}$, and in particular it satisfies $\mathcal{R}$. Thus,

$$\Pr[G_6] = 0.$$

The theorem follows by collecting the above (in)equalities. $\square$

Theorem 11 (ATS Bridge Liveness). *For any assumptions $\mathbb{A}_j, \mathbb{A}'_i, \mathbb{A}$, and parameters (c, ρ) , the ATS bridge (protocol Π -Bridge wrt. Bridge Scheme 1) from a sending ledger Π -Ledger_i to a receiving ledger Π -Ledger_j provides $(\mathbb{A}, \delta)$ -BRIDGELIVENESS if:*

- Both ledgers satisfy perfect LEDGERVALIDITY;
- $\mathbb{A} \Rightarrow \mathbb{A}_i$ and ledger protocol Π -Ledger_i satisfies $\mathbb{A}_i$ -AGREEDPREFIX;
- $\mathbb{A} \Rightarrow \mathbb{A}_j$ and ledger protocol Π -Ledger_j satisfies $(\mathbb{A}_j, \delta_j)$ -LEDGERLIVENESS for some δ_j such that $\Delta + \delta_j + 3 \leq \delta$;
- $\mathbb{A} \Rightarrow \mathbb{A}'_i$ and ledger protocol Π -Ledger_i satisfies $(\mathbb{A}'_i, c, \rho, 2R)$ -SECURECOMSEL;
- Scheme ATS satisfies ρc -UF-CMA and ρc -ASIGCORRECTNESS;
- Scheme SNARK satisfies COMPLETENESS.

Proof sketch. Curiously, the liveness proof follows the soundness proof above, with the goal of reaching a state where we can guarantee the security of the headers in mem.hs . The need to regularly update the receiving ledger (via mem.hs) about the *future* committee means that soundness and liveness are linked. The rest of the proof continues as in the liveness proofs of the other bridges.

G₀: This is the BRIDGELIVENESS game.

G₁-G_{2.5}: The games are identical to the ones in the ATS BRIDGESOUNDNESS proof.

G₃: This game aborts if the proof π output by Certify is $\perp$ or does not verify under Verify . On $\pi = (h, \pi')$, Verify invokes SNARK.Verify with statement (x, h) and proof π' . Since relation $\mathcal{R}$ is checked to hold, the header value h and proof π' is correctly computed within Certify , and the honestly generated rot (and thus crs) is used, a reduction to COMPLETENESS of the SNARK, faithfully running the UC execution encompassing the adversary and the protocol, shows that

$$\Pr[G_2] - \Pr[G_3] \leq \text{Adv}_{\text{SNARK}}^{\text{COMPLETENESS}}.$$

We note that as this point both the checks that $h \in \text{mem.hs}$ and that SNARK.Verify verifies pass and we have now arrived at a game G_3 where

$$\Pr[G_3] = 0.$$

The theorem follows. $\square$

D. Light-client-based and ZK bridges

Cross-chain bridges and light clients are intimately connected, and many real-world bridges are (or claim to be) based on light clients [9], [20]. In this work, we show a connection that (to a limited extent) also goes the other direction, at least in terms of definitions. Namely, we define the security features required from a low-level light client in terms of a bridge to a local one-party ledger¹⁸ corresponding to the client’s device that can locally verify the statements about the ledger that the client is aware of.

Local ledgers. We define the local ledger scheme DLS_0 wrt. Val_0 to be a ledger scheme where Setup_0 , AGen_0 , KGen_0 and Genesis_0 return the empty string ε . Algorithms $\text{Chain}_0(\text{st}) \rightarrow \mathcal{C}$ and $\text{Ledger}_0(\mathcal{C}) \rightarrow \mathcal{L}$ are identity maps. Algorithm $\text{BC}_0^{\text{Val}_0}(r, \mathbf{m}, \text{tx}; \text{st})$ ignores the input round r and messages $\mathbf{m}$ and operates by calling Val_0 to check the validity of transactions before appending the list of valid transactions in tx to its state st .

Note that $\text{Ledger}_0(\text{Chain}_0(\text{st}))$ is simply the list of accepted transactions. We define the local ledger protocol associated with a party P and Val_0 to be $\Pi\text{-Ledger}[P, \mathbf{w}, \delta, \text{DLS}_0]$, where $P := \{P\}$, $\mathbf{w}_P := 0$, $\delta := 0$, and DLS_0 is the local ledger scheme associated with Val_0 . This ledger satisfies $(\mathbb{A}_0, 0)\text{-LEDGERPERSISTENCE}$ and $(\mathbb{A}_0, 0)\text{-LEDGERLIVENESS}$ under $\mathbb{A}_0 := \text{True}$, i.e., under no trust assumptions.¹⁹ For light-client-based bridges, we need the local ledger, i.e., its predicates Val_0 , to support smart contracts, e.g., as defined in Algorithm 14.

Light-client schemes and protocols. We define a *light-client scheme* LC simply as a bridge scheme that is used to build a bridge protocol from some sending ledger $\Pi\text{-Ledger}_i$ to a local receiving ledger $\Pi\text{-Ledger}_0$. We call the resulting bridge protocol a *light-client protocol* $\Pi\text{-LC}$. Note that this results in a notion of a light client supporting a relation $\mathcal{R}$. This conceptualization of light clients immediately leads to the notions of $\mathbb{A}\text{-LIGHTCLIENTSOUNDNESS} := \mathbb{A}\text{-BRIDGE SOUNDNESS}$ and $(\mathbb{A}, \delta)\text{-LIGHTCLIENTLIVENESS} := (\mathbb{A}, \delta)\text{-BRIDGELIVENESS}$ where $\mathbb{A}$ and δ need to be sufficient to bridge to the local ledger. The protocol part of a light client, which supplies the bridge transactions, is seen as the light-client *server*, and the local ledger (including any contracts deployed) is seen as the light client.

The work of Xie et al. [51] introduces informal notions of liveness and consistency for light clients. Briefly, liveness requires blockchain headers to be eventually included in the light-client state, and consistency essentially requires that the header list of any light client node is a prefix of that of any full node. Setting the bridge relation $\mathcal{R}(x, C)$ to one that checks if x matches the header of C , we recover formal notions that

imply these. Indeed, liveness (in our sense) means that the header must be included so that the relation can be checked, and soundness precludes the inclusion of any incorrect header as this would result in an incorrect statement that successfully verifies. Note that this corresponds to a ‘bare’ light client which needs to be dressed up to verify, e.g., transaction inclusion. This extended light client can be seen as a bridge (in our syntax) where the bridge relation is $\mathcal{R}(x, C) := (x \in \text{Ledger}(C))$.

Succinctness. One desirable aspect of practical light clients is *succinctness*: in each invocation, they require at most $O(1)$ communication and computation to synchronize their internal state [51]. In our formulation of light clients, succinctness means that the underlying bridge protocol uses $O(1)$ communication to communicate with the local ledger (though the bridge parties may have a higher communication complexity) and further that the local ledger uses $O(1)$ time to update its state, the non-trivial component of which is Val_0 (e.g., by memoizing all but the last transaction).

Instantiating light-clients from bridges. A light-client scheme can be instantiated using any of the bridge schemes defined in the previous sections except for the direct-observation bridge. This is because the direct-observation bridge requires the predicate Val_0 to observe the sending chain, which a local ledger does not support (or else if it is extended to support it, the resulting light client would become a full node).

LC-based bridges. Let $\Pi\text{-Ledger}_i$ and $\Pi\text{-Ledger}_j$ be two ledger protocols, and let LC be a light-client scheme for $\Pi\text{-Ledger}_i$ wrt. $\mathcal{R}$ (i.e., a bridge scheme from $\Pi\text{-Ledger}_i$ to the local ledger).²⁰ We construct a bridge scheme BBS from $\Pi\text{-Ledger}_i$ to $\Pi\text{-Ledger}_j$ using LC supporting the same relation $\mathcal{R}$. Informally, this bridge operates a single “on-chain party” running the local ledger scheme on the receiving ledger. Note that the local ledger is non-interactive as it does not rely on any incoming network messages. We establish the bridge by bridging $\Pi\text{-Ledger}_i$ to this local ledger using the light-client scheme.

Instead of explicitly running the local ledger on the receiving ledger – which even if $O(1)$ can be costly as it might require verifying a constant but large number of signatures from a committee – we attach succinct proofs of correct transition of the local ledger state. The details of this protocol are shown in Bridge Scheme 5.

Theorem 12 (ZK Bridge Soundness). *For any given assumptions $\mathbb{A}_i$, $\mathbb{A}_{lc}$ and $\mathbb{A}$ the ZK bridge (protocol $\Pi\text{-Bridge}$ wrt. Bridge Scheme 5) from a sending ledger $\Pi\text{-Ledger}_i$ to a receiving ledger $\Pi\text{-Ledger}_j$ based on a light-client LC satisfies $\mathbb{A}\text{-BRIDGE SOUNDNESS}$ if:*

- Both ledgers satisfy perfect LEDGERVALIDITY ;
- $\mathbb{A} \Rightarrow \mathbb{A}_i$ and ledger protocol $\Pi\text{-Ledger}_i$ satisfies $\mathbb{A}_i\text{-AGREEDPREFIX}$;
- $\mathbb{A} \Rightarrow \mathbb{A}_{lc}$ and light-client protocol $\Pi\text{-LC}$ satisfies $\mathbb{A}_{lc}\text{-LIGHTCLIENTSOUNDNESS}$;

¹⁸Conceptually, this is the same type of centralized ledger as kept, e.g., by central banks as well as banks and other commercial institutions, although not used in that way here.

¹⁹Note that if the single (honest) party is corrupted both properties become vacuous and trivially hold.

²⁰We build a bridge protocol using the lower-level light-client scheme rather than a full fledged light-client, which is a higher-level protocol.

Bridge Scheme 5: ZK

```

Setup( $\lambda$ )
1:  $pp_0 \leftarrow \text{Setup}_0(\lambda)$ 
2:  $crs \leftarrow \text{SNARK.Setup}(\lambda)$ 
3: return ( $pp_0, crs$ )

BKGen( $(pp_0, crs)$ )
4: return BKGen $_0(pp_0)$ 

RoT( $(pp_0, crs), vk, C_i, C_j$ ) when  $r = r_0 + 1$ 
5:  $vk_0 \leftarrow vk; C_0 \leftarrow \varepsilon$ 
6:  $(rot_0, aux_0) \leftarrow \text{RoT}_0(pp_0, vk_0, C_i, C_0)$ 
7:  $rot \leftarrow (crs, rot_0); aux \leftarrow aux_0$ 
8: return ( $rot, aux$ )

Br( $r, m, C_i, C_j; bst$ ) when  $r = r_0 + 2$ 
9:  $rot \leftarrow bst.rot$ 
10:  $code \leftarrow \text{ZKBridge}[rot]$ 
11:  $cID \leftarrow \text{Hash}(code)$ 
12:  $bst.cID \leftarrow cID$ 
13:  $btX_j \leftarrow (cID, \text{install}, code)$ 
14: return ( $\perp, btX_j, \perp; bst$ )

Br( $r, m, C_i, C_j; bst$ ) when  $r > r_0 + 2$ 
15:  $st_0 \leftarrow bst.st_0; C_0 \leftarrow \text{Chain}_0(st_0)$ 
16:  $(btX_i, btX_0, m; bst) \leftarrow \text{Br}_0(r-1, m; C_i, C_0; bst)$ 
17:  $st \leftarrow bst.st_0$ 
18:  $(\_, st') \leftarrow \text{BC}_0^{\text{val}_0}(\_, \_, btX_0; st)$ 
19:  $bst.st_0 \leftarrow st'$ 
20:  $(crs, \_) \leftarrow \text{parse}(bst.rot)$ 

21:  $\pi \leftarrow \text{SNARK.Prove}(crs, (st, st'), btX_0)$ 
    //  $\mathcal{R}((st, st'), btX) := (\_, st') = \text{BC}_0^{\text{val}_0}(\_, \_, btX; st)$ 
22:  $btX_j \leftarrow (cID, \text{update}, (st', \pi))$ 
23: return ( $btX_i, btX_j, m; bst$ )

zk-bridge
getVerData( $G, rot, L$ )
24:  $CS \leftarrow \text{contractState}_j(G, L)$ 
25:  $cID \leftarrow \text{Hash}(\text{ZKBridge}[rot])$ 
26:  $(\_, mem, \_) \leftarrow CS[cID]$ 
27:  $L_0 \leftarrow \text{Ledger}_0(\text{Chain}_0(mem.st_0))$ 
28:  $(\_, rot_0) \leftarrow \text{parse}(rot); G_0 \leftarrow \varepsilon$ 
29:  $mem \leftarrow \text{getVerData}_0(G_0, rot_0, L_0)$ 
30: return  $mem$ 

Certify( $rot, x, C$ )
31:  $(\_, rot_0) \leftarrow \text{parse}(rot)$ 
32: return  $\text{Certify}_0(rot_0, x, C)$ 

Verify( $mem, x, \pi$ )
33: return  $\text{Verify}_0(mem, x, \pi)$ 

ZKBridge[ $rot$ ].update( $st', \pi$ )
34:  $(crs, \_) \leftarrow \text{parse}(rot)$ 
35:  $st \leftarrow mem.st_0$ 
36: check  $\text{SNARK.Verify}(crs, (st, st'), \pi)$ 
37:  $mem.st_0 \leftarrow st'$ 
38: return ok

```

- Scheme SNARK satisfies KNOWLEDGESOUNDNESS.

Proof sketch. We give an overview of the structure of the proof.

G₀: This is the BRIDGESOUNDNESS game.

G₁: This game aborts if an invalid transaction is included in ledger $\Pi\text{-Ledger}_j$. In particular a transaction for which SNARK.Verify or Verify fail. The probability of this event is upper bounded by LEDGERVALIDITY_j as in previous bridge soundness proofs.

G₂: For every bridge update transaction $btX_j = (cID, \text{update}, (st', \pi))$, this game runs the knowledge extractor for the SNARK to obtain a bridge transaction btX_0 for the local ledger, and aborts if the extracted transactions do not result in the submitted state st' when run with respect to state st . The probability of this event is bounded by KNOWLEDGESOUNDNESS. Note that we have to run the extractor for each update transaction. This implies the need for either straight-line extraction or tightly secure extraction, that allows to recursively build extractors from extractors without a blow up in the extractor's running time.

G₃: This game runs with respect to the extracted witnesses. Since the effect of witnesses are the same as the submitted states and proofs, we have that $\Pr[G_3] = \Pr[G_2]$.

We now show that an adversary that violates BRIDGESOUNDNESS in G_3 can be used to win LIGHTCLIENTSOUNDNESS, the soundness of the light

client; that is,

$$\Pr[G_3] \leq \Pr[\text{LIGHTCLIENTSOUNDNESS}] .$$

To see this, note that the bridge construction simply runs the Certify and Verify procedures of the light client, and by rules of G_3 , the state of the ledger on chain evolved according to the bridge transactions submitted to it. Thus any violation of $\mathbb{A}$ -BRIDGESOUNDNESS directly gives rise to a violation of $\mathbb{A}_{lc}$ -LIGHTCLIENTSOUNDNESS. $\square$

Theorem 13 (ZK Bridge Liveness). *For any given assumptions $\mathbb{A}_j$, $\mathbb{A}_{lc}$ and $\mathbb{A}$ the ZK bridge (protocol Π -Bridge wrt. Bridge Scheme 5) from a sending ledger $\Pi\text{-Ledger}_i$ to a receiving ledger $\Pi\text{-Ledger}_j$ based on a light-client LC satisfies $(\mathbb{A}, \delta)$ -BRIDGELIVENESS if:*

- Both ledgers satisfy perfect LEDGERVALIDITY;
- $\mathbb{A} \Rightarrow \mathbb{A}_j$ and ledger protocol $\Pi\text{-Ledger}_j$ satisfies $(\mathbb{A}_j, \delta_j)$ -LEDGERLIVENESS;
- $\mathbb{A} \Rightarrow \mathbb{A}_{lc}$ and light-client protocol $\Pi\text{-LC}$ satisfies $(\mathbb{A}_{lc}, \delta_{lc})$ -LIGHTCLIENTLIVENESS;
- $\delta \geq \delta_j + \delta_{lc}$;
- Scheme SNARK satisfies COMPLETENESS.

Proof sketch. We provide an overview of the proof.

G₀: This is the $(\mathbb{A}, \delta)$ -BRIDGELIVENESS game.

G₁: This game aborts if the Certify algorithm outputs $\perp$. Since this algorithm runs Certify₀, and the relation

is checked to be valid on the sending ledger, this event would lead to the break of `LIGHTCLIENTLIVENESS`:

$$\Pr[G_0] - \Pr[G_1] \leq \text{Adv}_{\Pi\text{-LC}}^{(\mathbb{A}_{lc}, \delta_{lc})\text{-LIGHTCLIENTLIVENESS}} .$$

G₂: This game extracts `btx0` from the proofs and updates the state using these, rather than the submitted `st'` and proof `π`. Reducing to (recursive) knowledge soundness we get:

$$\Pr[G_1] - \Pr[G_2] \leq \text{Adv}_{\text{SNARK}}^{\text{KNOWLEDGESOUNDNESS}} .$$

G₃: If the LC does not run correctly on the receiving ledger, this game aborts. The liveness of the receiving ledger shows that:

$$\Pr[G_2] - \Pr[G_3] \leq \text{Adv}_{\Pi\text{-Ledger}_j}^{(\mathbb{A}_j, \delta_j)\text{-LEDGERLIVENESS}} .$$

G₄: This game aborts if `Verify` does not return 1 on a submitted proof. Since `Verify` simply runs `Verify0` of the light client, once again we have that:

$$\Pr[G_3] - \Pr[G_4] \leq \text{Adv}_{\Pi\text{-LC}}^{(\mathbb{A}_{lc}, \delta_{lc})\text{-LIGHTCLIENTLIVENESS}} .$$

We note that $\Pr[G_4] = 0$, and the theorem follows by collecting the above terms. $\square$